

Architecture et programmation d'accélérateurs matériels

Cours 1 : Architecture et modèles de programmation

Julien Jaeger
julien.jaeger@cea.fr

Introduction

- **Thématique : accélérateurs**
 - Etat de l'art
 - Architecture des accélérateurs
 - Programmation des accélérateurs
 - Optimisations caractéristiques
- **Pré-requis**
 - Notion de base architecture
 - Programmation impérative
 - Notion de programmation parallèle

Plan général

- But :
 - Introduction aux accélérateurs
 - Expertise en programmation CUDA
- Plan :
 - Architecture
 - Modèle de programmation
 - Introduction à CUDA
- Intervenant
 - Julien Jaeger (julien.jaeger@cea.fr)
 - Maël Martin (mael.martin@cea.fr)

Plan du cours

- Panorama des accélérateurs
- Architecture de GPGPU
 - NVIDIA Kepler
 - NVIDIA Volta
 - NVIDIA Ampere
 - NVIDIA Grace+Hopper
- Modèles de programmation pour accélérateurs
- NVIDIA CUDA (partie 1)

Panorama des accélérateurs

Accélérateurs

- Définition

- Ressource distincte (située sur la même puce que le processeur principal ou non) définie pour un ensemble différents d'objectifs que le processeur central

- En clair :

- Processeur dédié à une classe spéciale d'applications
- Permet d'obtenir de meilleures performances dans son domaine de prédilection

Accélérateurs – Interconnexion

- Les accélérateurs sont reliés au(x) processeur(s) principal(aux) grâce à une *interconnexion*
- Comportement similaire à un périphérique
 - Le processeur central contrôle l'accès
 - Il exécute également la partie principale du code
- Plusieurs types d'interconnexion
 - Accélérateurs déportés
 - Accélérateurs sur la puce principale

Types de connexions

- Via le bus système

- GPGPU nVidia, MIC Intel

- Avantages

- Faible coût
- Intégration aisée

- Inconvénients

- Performances modestes dûes à la généricité du bus
- Bande passante et latence sous-optimale

- Via le bus du processeur

- IBM Cell , APU AMD, Grace ARM + Hopper Nvidia

- Accès rapide
- Possibilité de partager des ressources avec le CPU

- Inconvénients

- Intégration complexe
- Dépendance avec le processeur

Architecture des accélérateurs

- **Microarchitecture**
 - Dirigée par la diversité des applications à exécuter ainsi que du domaine de prédilection
 - Simplification par rapport aux CPUs
 - Exécution dans le désordre, prédicteur de branchements, ...
- **Parallélisme exploité**
 - Généralement à grain très fin
 - La plupart exploite une forme de *Single Instruction Multiple Data*
- **Mais**
 - Une partie *importante* du travail est laissée à la pile logicielle et au développeur de codes

Architecture des accélérateurs

- Hiérarchie mémoire

- Mémoire contrôlée par le logiciel/programmeur (*scratchpad memory*)
- Cache géré par le matériel (comme sur un CPU généraliste)
- Interconnexion matérielle entre les différents processeurs de calcul
- Bande passante importante entre l'accélérateur et la mémoire globale

- Résumé

- Possibilités intéressantes pour le calcul haute performance
- Mais beaucoup d'aspects à gérer à *la main*.
- Ceci influence directement la façon de programmer les accélérateurs

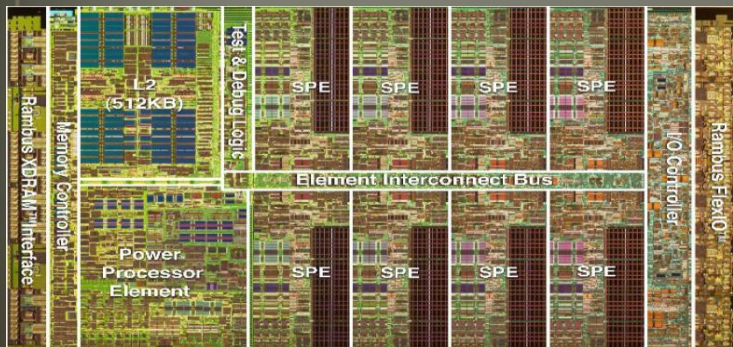
Programmation des accélérateurs

- Vision d'un accélérateur comme un périphérique *worker*.
- Le processeur principal contrôle l'accès aux ressources de calculs spécialisées
 - Processeur principal (CPU) : *hôte* ou *host*
 - Accélérateur : *device*
- Concepts généraux
 - Transferts des données
 - Exécution d'un code déporté dans un langage différent
 - Gestion manuel de l'exécution
 - Interrogation de l'accélérateur sur son état
 - Rapatriement des données sur l'hôte

Panorama des accélérateurs

- Accélérateurs dédiés
 - Cell (IBM)
 - MIC ou Xeon Phi (Intel)
- Processeurs graphiques
 - GPGPUs (General Purpose Graphics Processing Units)
 - NVIDIA
 - AMD/ATI
- Architectures reconfigurables
 - FPGA

Le processeur CELL (IBM)



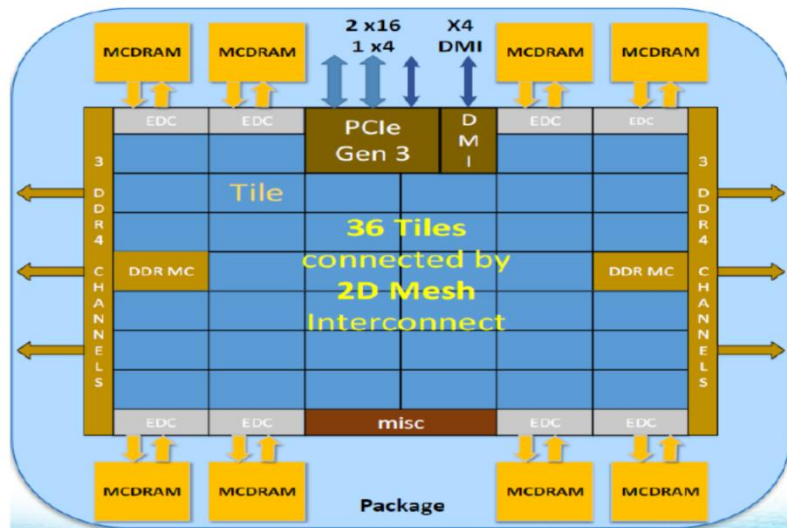
- Développé conjointement par IBM, Sony et Toshiba
- 8 cœurs 64-bit, à calculs flottants
 - Appelés *Synergistic Processor Elements* (SPEs).
- Un processeur maître PowerPC 64-bit capable d'exécuter deux threads
- SPEs peuvent traiter des opérandes 128-bit operands, décomposées en quatre mots 32-bits (SIMD).
- Le banc de registres peut stocker 128 opérandes.

MIC (Intel)

- Accélérateur proposé par Intel
 - Many-Integrated Core (MIC)
- Architecture
 - Coeurs x86
 - Support de l'*hyperthreading*
 - Cohérence de cache
- Première version: KNF
- Dernière version : KNL

KNL

KNL Overview



TILE

2 VPU	CHA	2 VPU
Core	1MB L2	Core

Chip: 36 Tiles interconnected by **2D Mesh**

Tile: 2 Cores + 2 VPU/core + 1 MB L2

Memory: MCDRAM: 16 GB on-package; High BW

DDR4: 6 channels @ 2400 up to 384 GB

IO: 36 lanes PCIe* Gen3. 4 lanes of DMI for chipset

Node: 1-Socket only

Fabric: Intel® Omni-Path Architecture on-package (not shown)

Vector Peak Perf: 3+TF DP and 6+TF SP Flops

Scalar Perf: ~3x over Knights Corner

Streams Triad (GB/s): MCDRAM : 400+; DDR: 90+

MCDRAM
~5X Higher BW
than DDR

- Source Intel: All products, computer systems, dates and figures specified are preliminary based on current expectations, and are subject to change without notice. KNL data are preliminary based on current expectations and are subject to change without notice. 1Binary Compatible with Intel Xeon processors using Haswell Instruction Set (except TSX). 2Bandwidth numbers are based on STREAM-like memory access pattern when MCDRAM used as flat memory. Results have been estimated based on internal Intel analysis and are provided for informational purposes only. Any difference in system hardware or software design or configuration may affect actual performance. *Other names and brands may be claimed as the property of others.

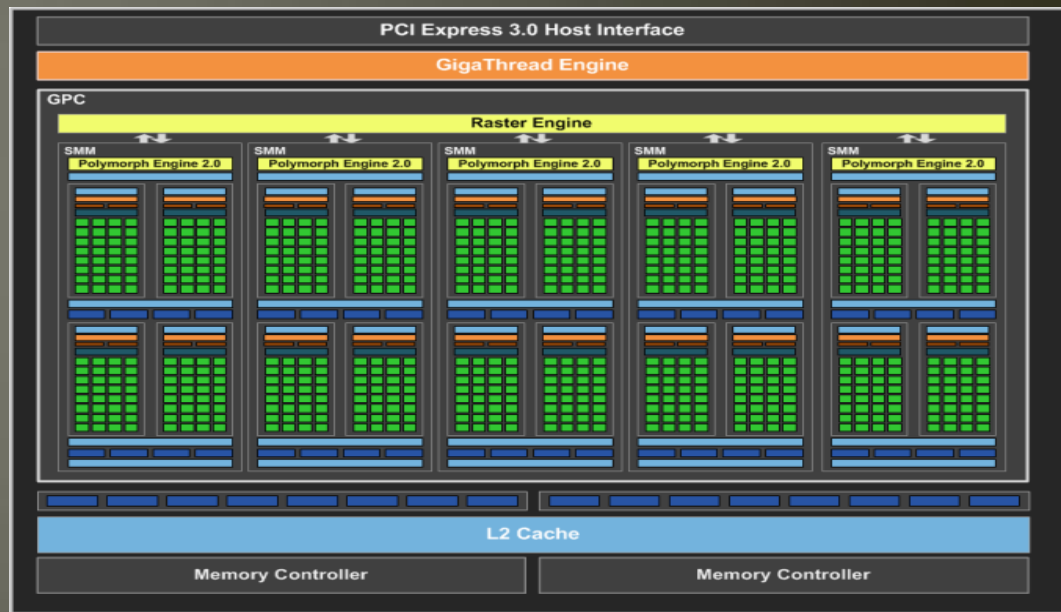
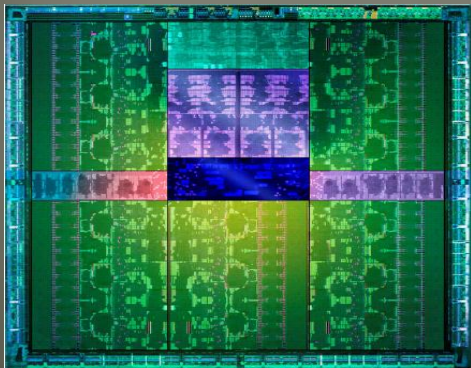
Processeurs graphiques

- Cartes graphiques initialement pour l'affichage 2D et 3D sur écran
 - Processeur(s) graphique(s) dédié(s)
- Capacités de calcul impressionnantes
 - Optimisation pour le rendu d'image (objectifs différents des processeurs généralistes)
 - Idéal pour le traitement sur des données régulières
- Utilisation pour le calcul scientifique
 - GPGPU (General Purpose Graphics Processing Unit)

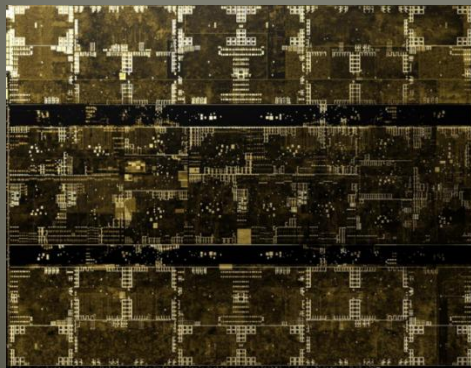
GPGPU

- Cartes graphiques sur un bus système
 - Interconnexion sur la carte mère
 - Communications via le port PCI-express
- Possibilité de connecter plusieurs cartes graphiques
 - Programmation Multi-GPU
 - Utilisé actuellement dans les supercalculateurs hybrides

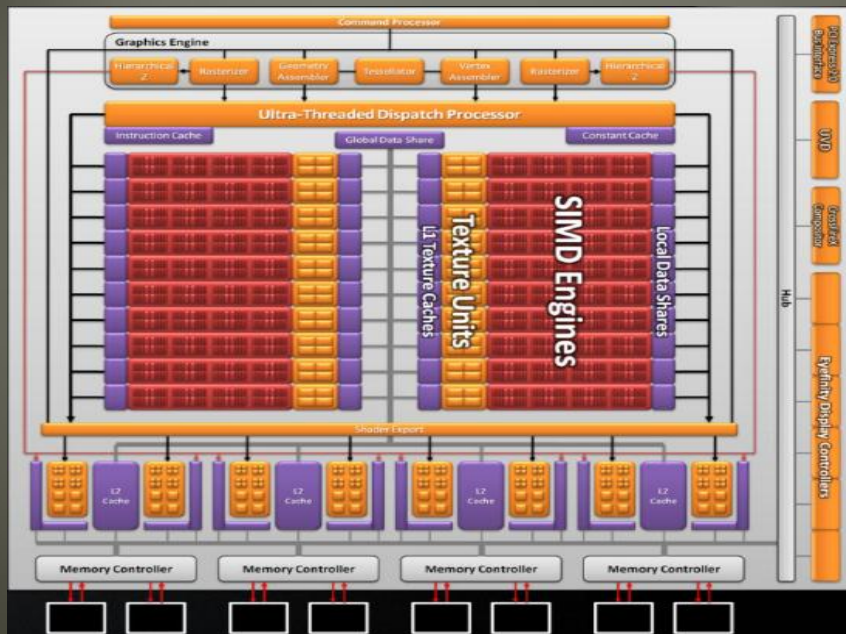
Architecture Maxwell NVIDIA



Volta

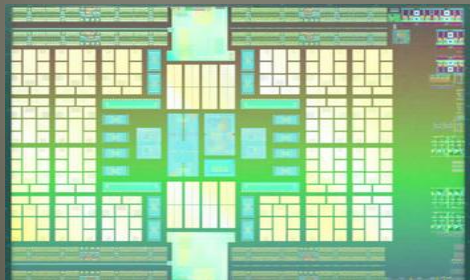


Architecture ATI Cypress



- Ensemble d'unités vectorielles
 - SIMD Engines
- Hiérarchie mémoire
 - Cache L2
- Performances crêtes
 - 2,72 TFlops SP
 - 0.54 TFlops DP

Architecture ATI Radeon Navi



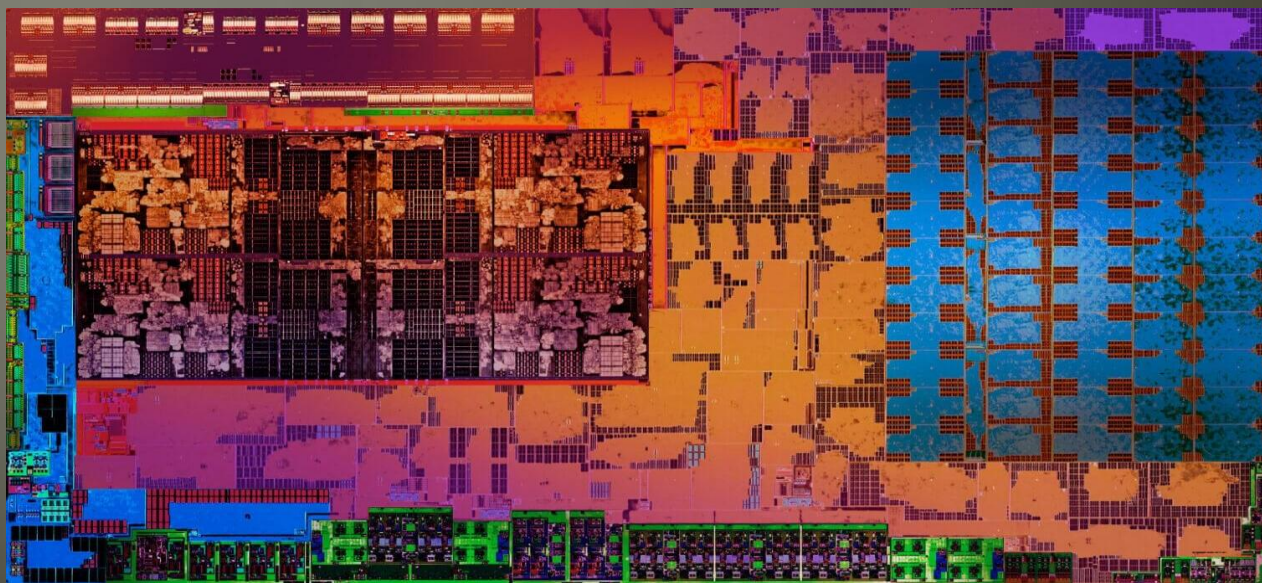
Architecture Fusion (Llano)



- Nom de code de l'architecture d'AMD/ATI : Fusion
- Design d'un APU (Accelerator Processing Unit)
- Sur la même puce :
 - CPU
 - Accélérateur de type GPU
- Première génération disponible en 2011
 - Llano avec technologie Bulldozer ou Bobcat
- <http://fusion.amd.com>

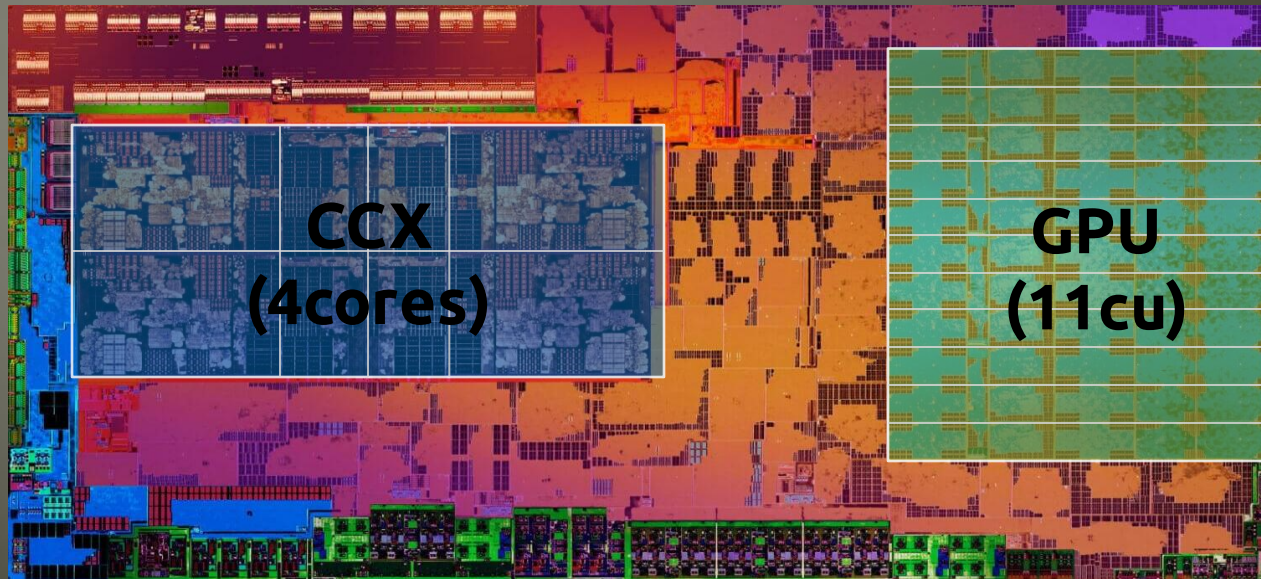
Architecture Fusion (Picasso)

- Ryzen-bridge 2019



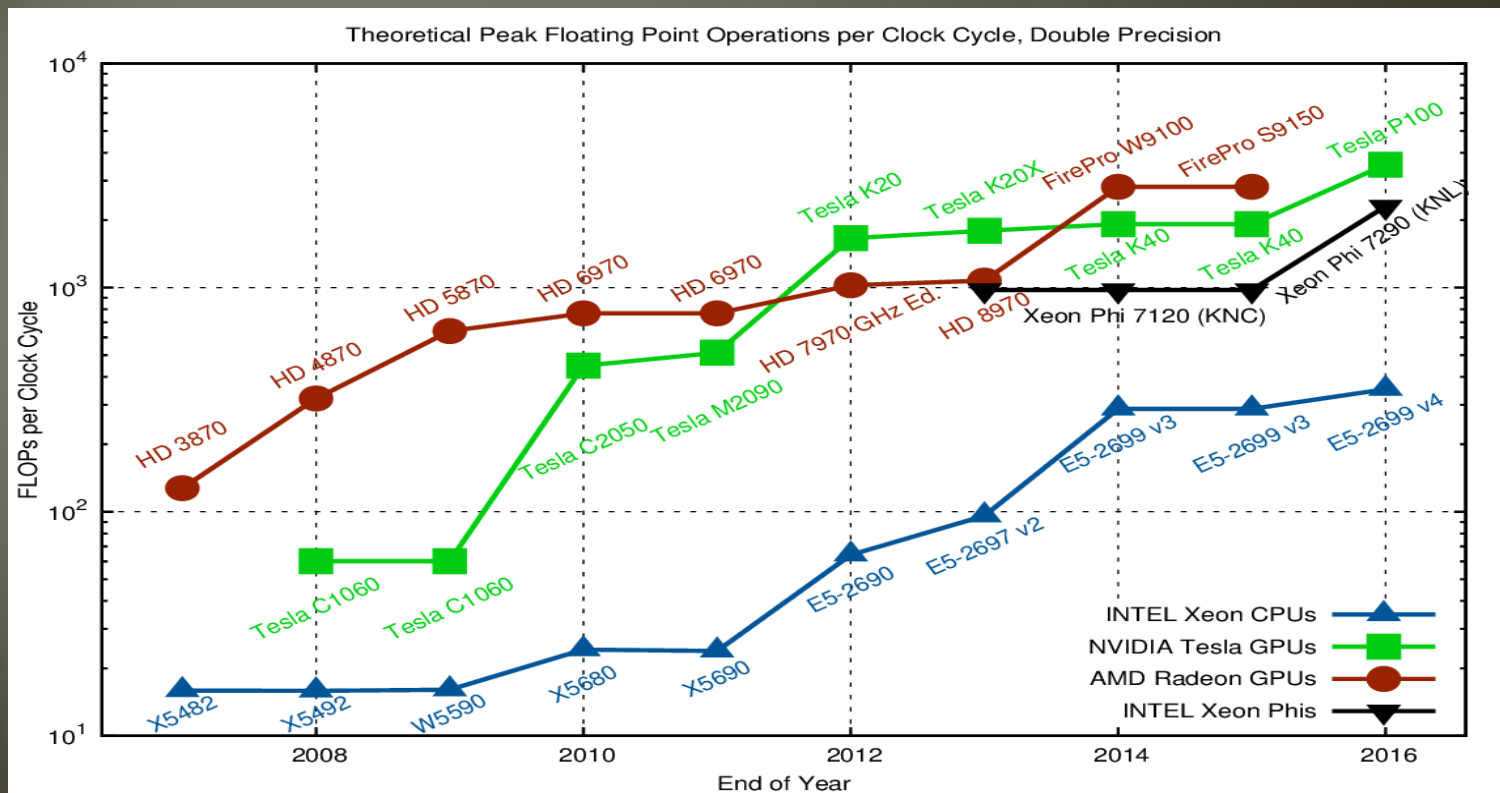
Architecture Fusion (Picasso)

- Ryzen-bridge 2019



Architecture de cartes graphiques NVIDIA

Evolution architecture NVIDIA – Performances flottantes



Architecture NVIDIA GeForce

- Brique de base
 - Coeur de calcul très simple
 - Ensemble de coeurs de calcul regroupés dans un *stream multiprocessor*
- Pourquoi *stream* ?
 - Fait pour fonctionner sur un flux de données continu
 - Langage synchrone sur chaque multicoeurs
- Design pour le parallélisme à grain fin

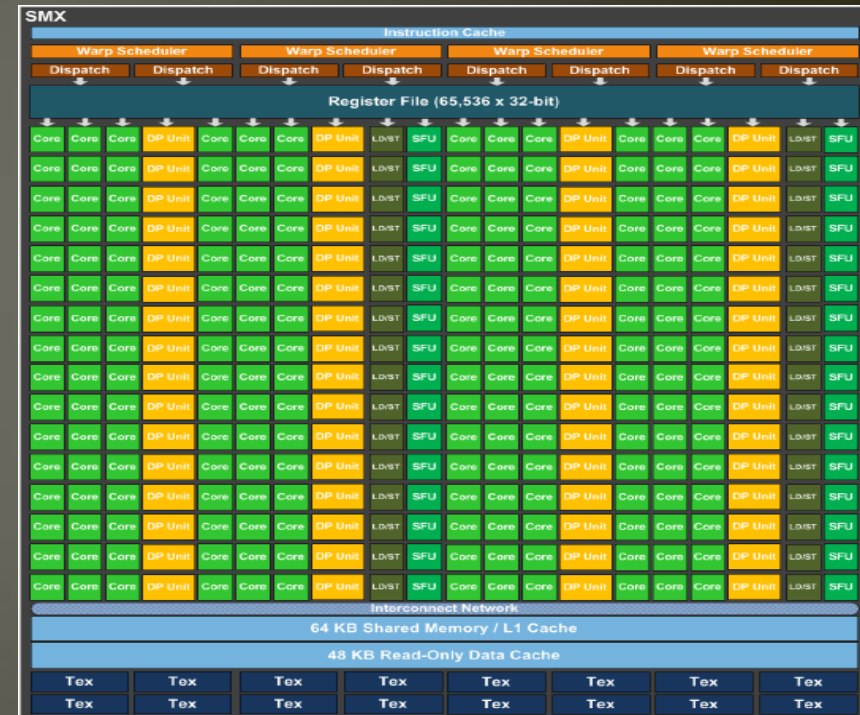
Architecture Kepler NVidia

- 15 Streaming Multiprocessors (SMs)
- Arrangement perpendiculaire au cache commun L2
- Légende
 - Vert → cœur de calcul
 - Unité double précision en orange
 - Orange → ordonnanceur et dispatcher
 - Bleu clair → banc de registres et cache L1



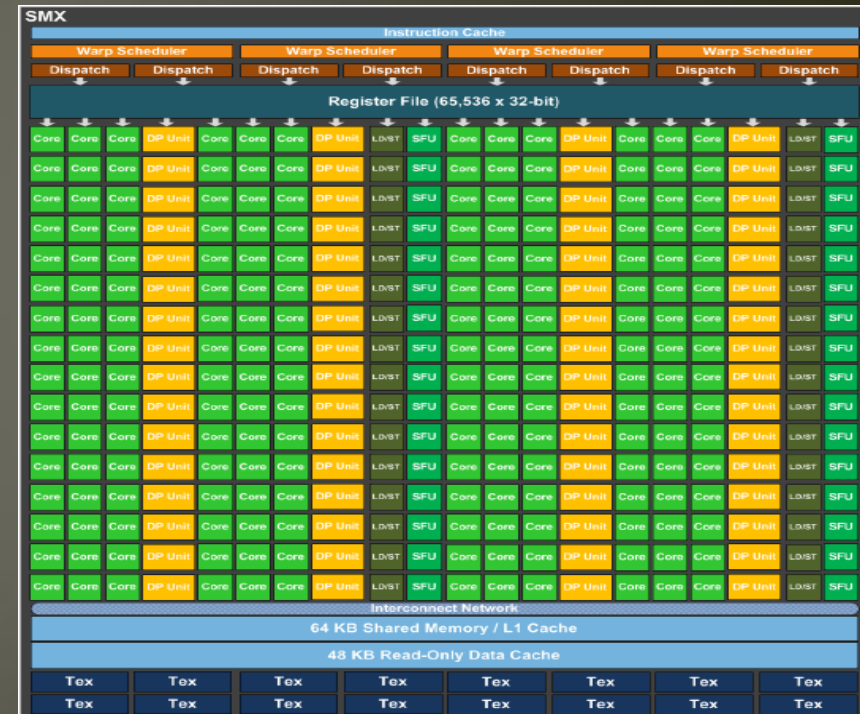
Kepler : Streaming Multiprocessor

- Streaming Multiprocessor (SMX)
- 192 cœurs compatibles CUDA
 - Unité ALU
 - Unité flottante simple précision
- 64 unités flottantes double précision
- 32 unités spéciales (SFU)
- 32 unités *load/store*



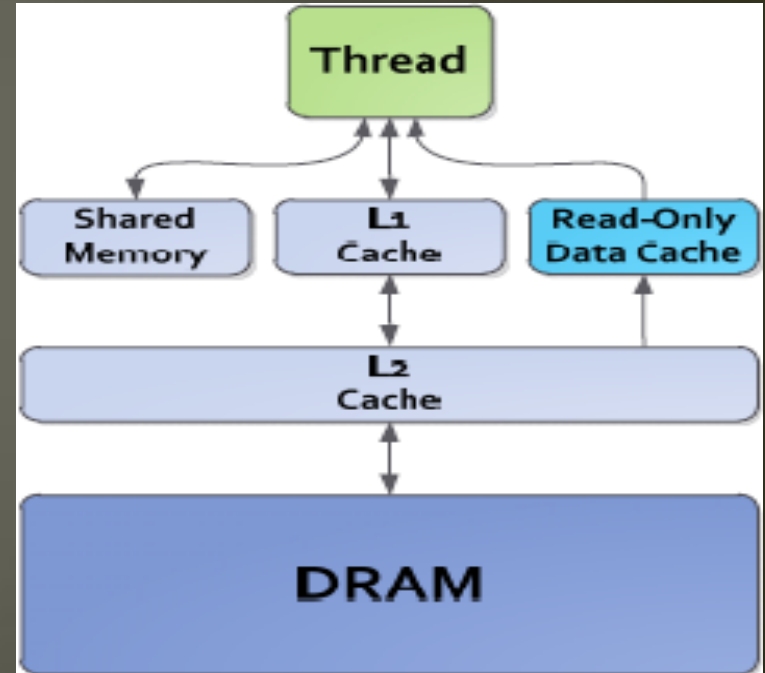
Kepler : Streaming Multiprocessor

- Exécution de type SIMT
 - Single-Instruction Multiple Thread
- Similaire au SIMD
- Exécution synchrone des cœurs de calcul
- Chaque SMX gère et exécute les threads de calcul par paquets de 32
 - Notion de warp
 - 4 ordonnanceurs de warps par SMX



Kepler : hiérarchie mémoire

- Thread ordonnancé sur un coeur de calcul sur un SMX
- Accès mémoire via instruction *load* ou *store*
- Deux possibilités
 - Accès mémoire partagé → pas de cache
 - Accès à la mémoire globale du device → passage par les 2 niveaux de cache
- Physiquement, la même mémoire est utilisée pour le cache L1 et la shared memory
- Apparition d'un read-only cache (48 Ko)



Volta



- 6 GPCs (Gpus Processing clusters)
- 7 TPCs (Texture Processing clusters) / GPC
- 2 SM par TPCs
- $\rightarrow 2 * 7 * 6 = 84 \text{ SM!}$

Volta



- 84 Streaming Multiprocessor
- 5376 unités flottantes simple précision (14 Tflops)
- 5376 unités entières simple précision (14 Tflops)
- 2688 unités flottantes double précision (7 Tflops)
- 672 cœurs tenseur (112 Tflops)
- 336 unités de texture

Ampere



- 8 GPCs (Gpus Processing clusters)
- 8 TPCs (Texture Processing clusters) / GPC
- 2 SM par TPCs
- $\rightarrow 2 * 8 * 8 = 128 \text{ SM!}$

Ampere: SM view

- 4 processings blocks / SM
- 16 unités int $\rightarrow$ 8192 (21 Tops)
- 16 unités flottantes simple précision $\rightarrow$ 8192 (21 Tflops)
- 8 unités flottantes double précision $\rightarrow$ 4096 (10 Tflops)
- 1 cœur tenseur (112 Tflops)



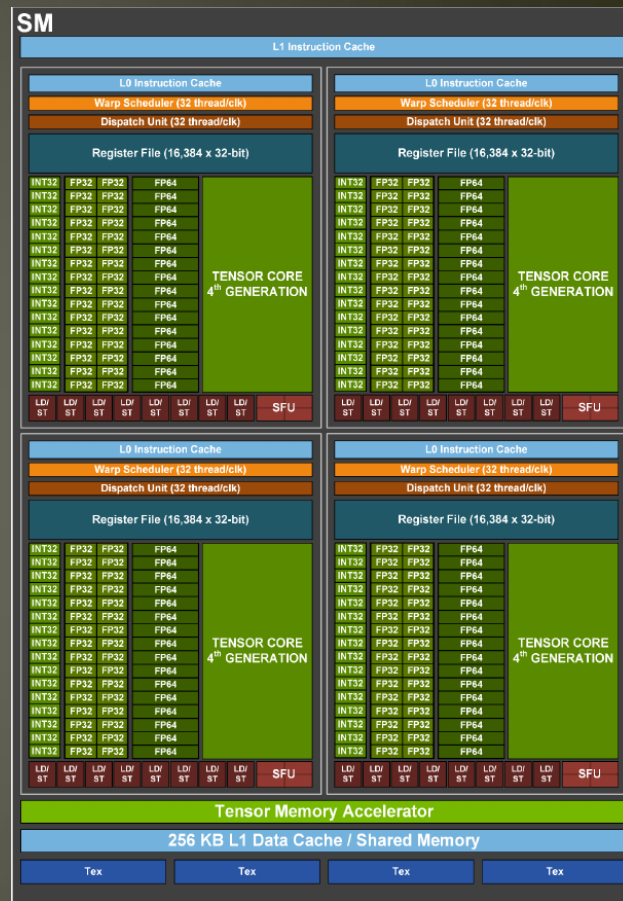
Hopper : Bienvenue en 2023

- 8 × Graphic Processing Clusters (GPC)
- 72 × Texture Processing Clusters (TPC)
 - 9 TPCs / GPC
- 144 (132?) × Ampere Streaming Multiprocessors (SM)
 - 2 SMs / TPC
 - 18 SMs / GPC
- 12 × Contrôleurs mémoire 512-bits
- 6 × High Bandwidth Memory (HBM2) Stacks
- 60MB L2 Cache



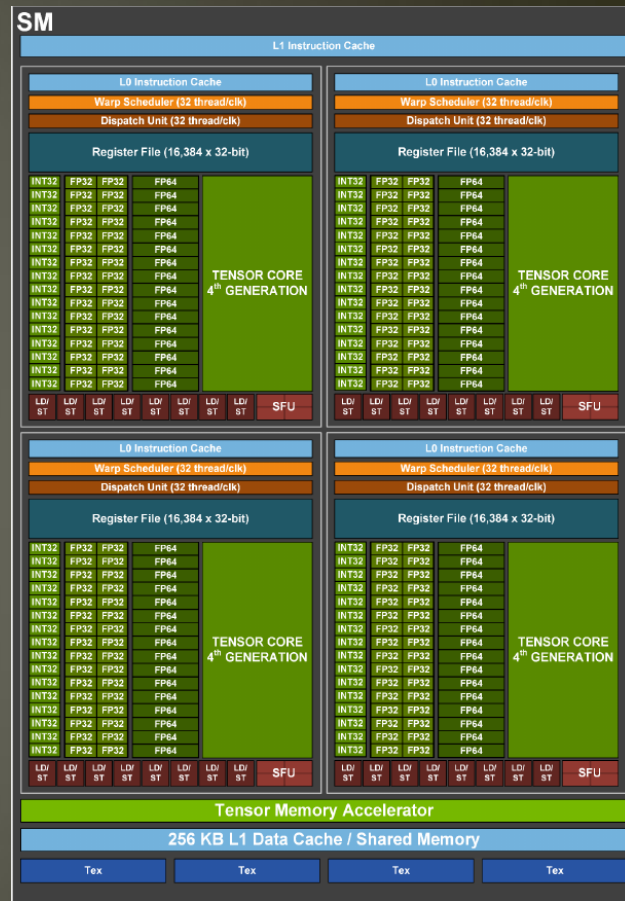
Hopper : SM insiders

- 4 × Processing Blocks
- 16 Unités INT32
- 32 Unités Simple Précision
- 16 Unités Double Précision
- 8 Unités Load/Store
- 1 Warp Scheduler
- 1 Dispatch Unit
- 64 warps actifs maximum



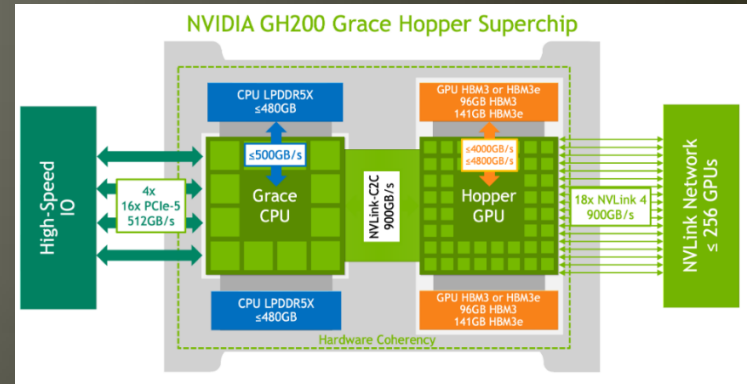
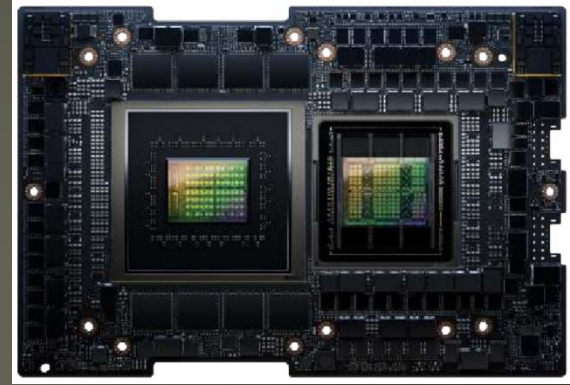
Hopper performances

- « classical » computing
 - FP32 : 60 Tflops
 - FP64 : 30 Tflops
- « IA » computing (tensor cores)
 - FP64 : 60 Tflops
 - TF32 : 1 Pflops
 - FP16 : 2 Pflops
 - FP8 : 4flops
- Difference between FP32 and TF32
 - FP32 : 8 bits exponent / 23 bits mantissa
 - TF32 : 8 bits exponent / 10 bits mantissa
 - FP16 : 5 bits exponent / 10 bits mantissa



Grace-Hopper superchip : l'avenir ?

- Processeur « Grace »
 - Architecture ARM Neoverse V2
 - 72 cœurs
- CPU et GPU sur le même die pour accélérer les transferts de données
 - 2x 450GB/s
 - Mémoire unifiée « native » garantie par un système de cohérence mémoire (C2C)



Modèles de programmation pour accélérateurs

Modèles de programmation

- Langages dédiés
 - Close To Metal (CTM) /
 - HIP/ROCm
 - CUDA
- Langage portables
 - OpenCL / SYCL
- Directives de parallélisation
 - OpenMP
 - OpenACC
- Langages haut niveau / DLS
 - Kokkos / Legion / RAJA

Programmation ATI

- **Close to Metal (CTM)**
 - Très bas niveau
 - Peu utilisé à présent
- **CAL/IL**
 - Compute Abstract Layer
 - Interaction entre le processeur principal et l'accélérateur
 - Intermediate Language
 - Langage dédié à l'exécution sur accélérateur
- **ROCm**
 - Offers toolkit to write and translate CUDA codes
- **OpenCL/SYCL**
 - Support des standards OpenCL et SYCL
 - AMD/ATI fait partie du consortium

Programmation NVIDIA

- Langage CUDA
 - Suite...

Directives de parallélisation

- **Proposition de programmation à base de directives**
 - Ajout de `#pragma` (C,C++) dans le code séquentiel
 - Possibilité d'ajouter ces directives dans un code déjà parallèle (MPI, OpenMP, ...) : ceci dépend du modèle
- **Solutions proposées**
 - OpenACC
 - OpenMP 4+
- **Avantages**
 - Portabilité en fonction des accélérateurs
 - La sémantique originale (voire séquentielle) peut être préservée
- **Inconvénients**
 - Souvent besoin de finir à *la main*
 - Spectre parfois limité

NVIDIA CUDA

Vue d'ensemble

- Programmation de *kernel*
- Deux parties : programme hôte et noyaux de calcul
- Nécessité de transférer les données
- Chaine de compilation dédiée à cause des noyaux de calcul (langage différent)
- Lien avec un *runtime* (en espace utilisateur) et un *driver* (en espace noyau)

Schéma de programmation

1. Initialisation

- Allocation mémoire
- Lecture des entrées
- ...

2. Allocation mémoire sur le GPU

- Gestion manuelle !

3. Transfert hôte → GPU

4. Exécution d'un noyau de calcul

- Possibilité de lancer plusieurs noyaux

5. Transferts GPU → hôte

6. Libération mémoire GPU

7. Libération mémoire CPU

Comment compiler ?

- **Structure du code**
 - Partie pour l'hôte
 - Partie pour le *device*
- En pratique, ces codes peuvent être mélangés dans le même fichier
- **Programmation hôte**
 - C ou C++
- **Programmation *device***
 - CUDA (sur-ensemble du C norme 99)
- **Chaîne de compilation : NVCC**
 - En tant que programmeur : utilisation du compilateur `nvcc`
 - N'hésitez pas à regarder la documentation livrée avec le SDK

Exemple de programme

- Exemple : addition de 2 vecteurs
- Opération de base :
 - $c[i] = a[i] + b[i]$
- Comment programmer ce noyau de calcul en séquentiel ?
 - Simple boucle
 - Application de l'opération de base sur chaque élément

```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

Portage de notre exemple

```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

Portage de notre exemple

```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

Transfert du
tableau c

Transfert du
tableau a

Transfert du
tableau b

Transfert de
l'entier N

Portage de notre exemple

```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

Transfert du
tableau c

Transfert du
tableau a

Transfert du
tableau b

Transfert de
l'entier N

Exécution sur
le GPU
en parallèle

??

Portage de notre exemple

```
void vectAdd(  
    double * a, double * b,  
    double * c, int N) {  
    int i ;  
    for (i=0;i<N;i++) {  
        c[i] = a[i] + b[i]  
    }  
}
```

Transfert du
tableau c

Transfert du
tableau a

Transfert du
tableau b

Transfert de
l'entier N

Rapatriement du
tableau c

Exécution sur
le GPU
en parallèle

??

Programmation sur l'hôte

- Gestion de l'hôte (CPU central)
 - Besoin de gérer la mémoire du GPU
 - Besoin de transférer les données
 - Donne l'ordre d'exécution du kernel
- API CUDA de l'hôte
 - CUDA Haut niveau / Bas niveau
 - `cudaMalloc` : allouer des données sur le *device*
 - `cudaMemcpy` : transférer des données vers le *device* ou vers l'hôte

Programme hôte

```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a,  
N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
N*sizeof(double));  
  
    vectAddKernel(d_a, d_b, d_c, N);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

```
    cudaMemcpy(d_a, a, N*sizeof(double),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(d_c, c, N*sizeof(double),  
cudaMemcpyHostToDevice);  
  
    vectAddKernel(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

Programme hôte

```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```



Pointeurs
sur le
device

```
    cudaMalloc((void **)&d_a,  
N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
N*sizeof(double));
```

```
    cudaMemcpy(d_a, a, N*sizeof(double),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(d_c, c, N*sizeof(double),  
cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

```
    cudaMemcpy(c, d_c, N*sizeof(double),  
cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

```
}
```

Programme hôte

```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```

Pointeurs
sur le
device

```
    cudaMalloc((void **)&d_a,  
N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
N*sizeof(double));
```

Allocation
mémoire
sur le GPU

```
    cudaMemcpy(d_a, a, N*sizeof(double),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(d_c, c, N*sizeof(double),  
cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

```
    cudaMemcpy(c, d_c, N*sizeof(double),  
cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

```
}
```

Programme hôte

```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```

Pointeurs
sur le
device

```
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

Allocation
mémoire
sur le GPU

Transferts
vers le
GPU

```
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_c, c, N*sizeof(double),  
        cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

```
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

```
}
```

Programme hôte

```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```

Pointeurs
sur le
device

```
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

Allocation
mémoire
sur le GPU

Exécution
du *kernel*

```
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_c, c, N*sizeof(double),  
        cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

```
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

```
}
```

Transferts
vers le
GPU

Programme hôte

```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```

Pointeurs
sur le
device

```
    cudaMalloc((void **)&d_a,  
N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
N*sizeof(double));
```

Allocation
mémoire
sur le GPU

Exécution
du *kernel*

```
    cudaMemcpy(d_a, a, N*sizeof(double),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
cudaMemcpyHostToDevice);  
    cudaMemcpy(d_c, c, N*sizeof(double),  
cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

```
    cudaMemcpy(c, d_c, N*sizeof(double),  
cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

```
}
```

Transferts
vers le
GPU

Transferts
vers l'hôte

Programme hôte

```
void vectAdd(  
    double * a, double * b, double * c,  
    int N) {
```

```
    double * d_a ;  
    double * d_b ;  
    double * d_c ;
```

```
    cudaMalloc((void **)&d_a,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_b,  
        N*sizeof(double));  
    cudaMalloc((void **)&d_c,  
        N*sizeof(double));
```

Pointeurs
sur le
device

Exécution
du *kernel*

```
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);
```

```
    vectAddKernel(d_a, d_b, d_c, N);
```

```
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);
```

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

Transferts
vers l'hôte

Allocation
mémoire
sur le GPU

Transferts
vers le
GPU

Libération
mémoire
sur le GPU

```
}
```

Modèle d'exécution

- Le *device* n'a plus qu'à exécuter le code en lui-même : l'addition de 2 vecteurs
- Comment programmer sur le *device* ?

```
for (i=0;i<N;i++)  
    c[i]=a[i]+b[i];
```

- Le modèle **CUDA** définit la notion de thread
 - Chaque thread lance la fonction *kernel*
 - Un peu comme si de nombreux threads étaient créés avec `pthread_create` en passant la fonction kernel en fonction à exécuter
- La programmation **CUDA** est alors par défaut *multithreadée*
 - Il faut raisonner par thread et non pas par boucle ou par itération de boucles
 - Différent d'une programmation séquentielle classique
- **Code du kernel** dans notre cas :

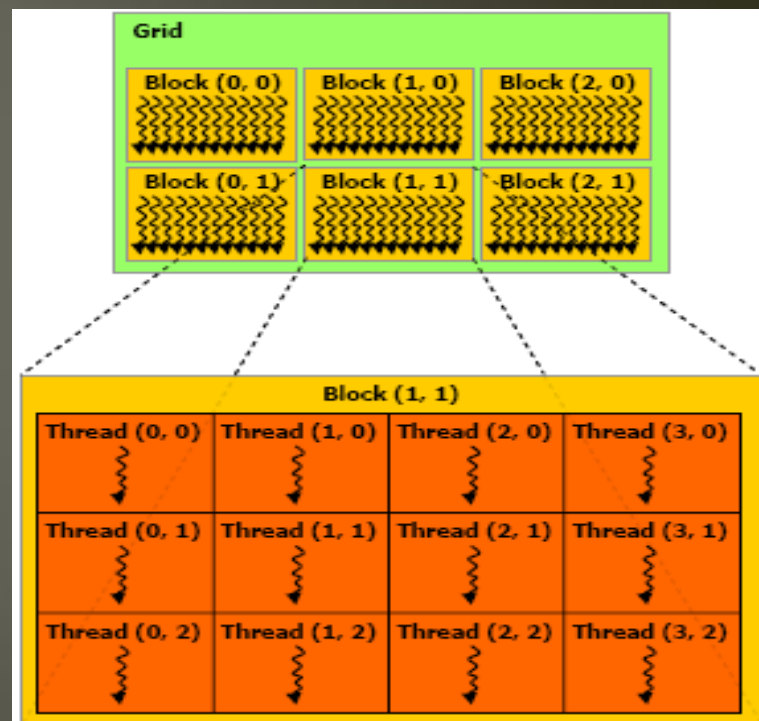
```
int i ;  
i= index; /* Update i with thread index */  
c[i] = a[i] + b[i];
```

Hiérarchie de threads

- Comment connaître l'identifiant d'un thread ?
 - Ceci est lié à l'organisation des threads au niveau du modèle d'exécution
- CUDA définit la notion de bloc (*block*) et grille (*grid*) pour organiser les threads de façon hiérarchique
- Une grille
 - Contient des blocs
 - Qui contiennent des threads
- La grille et les blocs peuvent avoir de 1 à 3 dimensions (représentation logique et non physique)
- Concrètement ?

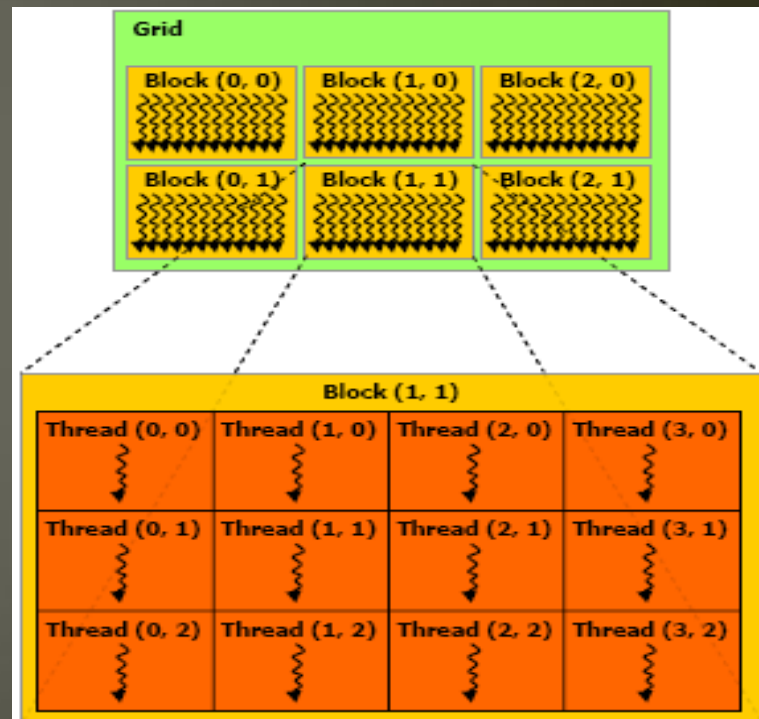
Hiérarchie de threads

- Exemple avec
 - 1 grille à 2 dimensions
 - 6 blocs à 2 dimensions
- Il y a 6 blocs dans la grille et 12 threads par bloc
- ➔ 72 threads au total
- Calcul d'un indice de thread ?
 - Linéarisation de tableau à n-D



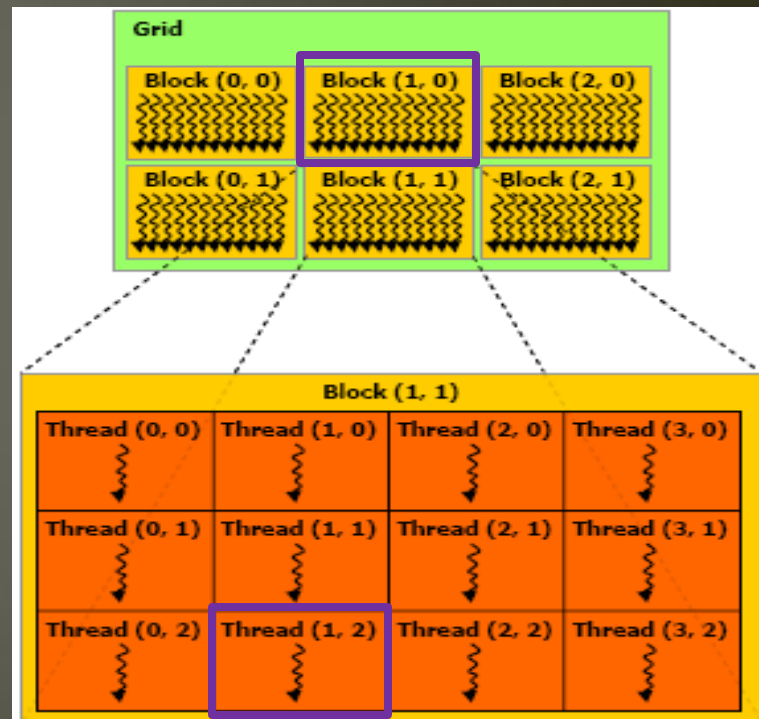
Hiérarchie de threads

- **Dimensions de la grille**
 - `dim3 gridDim`
 - Contient le nombre de blocs dans les 3 dimensions de la grille
- **Indice d'un bloc dans la grille**
 - `dim3 blockIdx`
 - Contient les coordonnées du bloc courant dans la grille
- **Dimensions des blocs**
 - `dim3 blockDim`
 - Contient le nombre de threads dans les 3 dimensions des blocs
- **Indice du thread dans le bloc**
 - `uint3 threadIdx`
 - Contient l'indice du thread courant dans le bloc courant (3 dimensions)
- **Taille d'un warp**
 - `int warpSize`



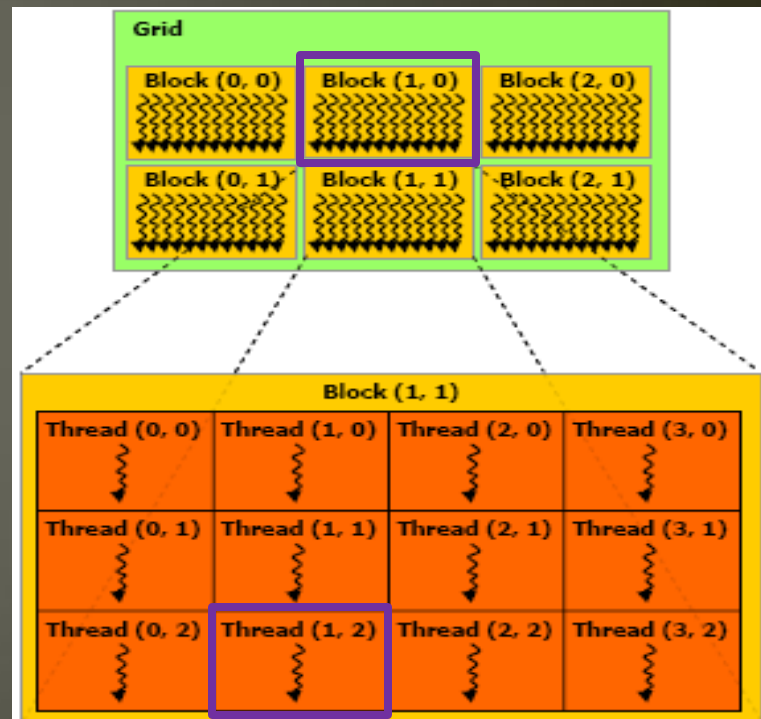
Hiérarchie de threads

- **Dimensions de la grille**
 - `dim3 gridDim`
 - Contient le nombre de blocs dans les 3 dimensions de la grille
- **Indice d'un bloc dans la grille**
 - `dim3 blockIdx`
 - Contient les coordonnées du bloc courant dans la grille
- **Dimensions des blocs**
 - `dim3 blockDim`
 - Contient le nombre de threads dans les 3 dimensions des blocs
- **Indice du thread dans le bloc**
 - `uint3 threadIdx`
 - Contient l'indice du thread courant dans le bloc courant (3 dimensions)
- **Taille d'un warp**
 - `int warpSize`



Hiérarchie de threads

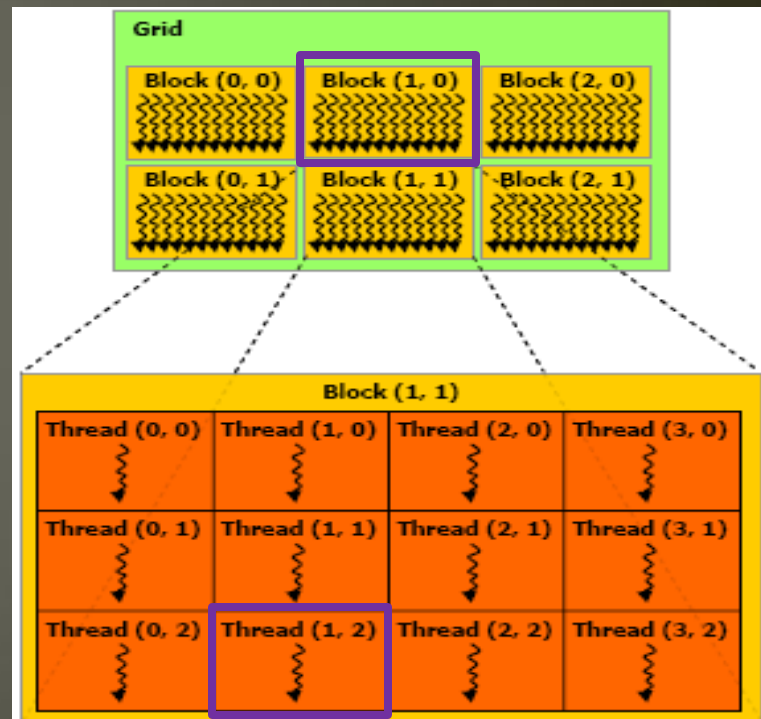
- Calculons l'ID global du thread encadré



Hiérarchie de threads

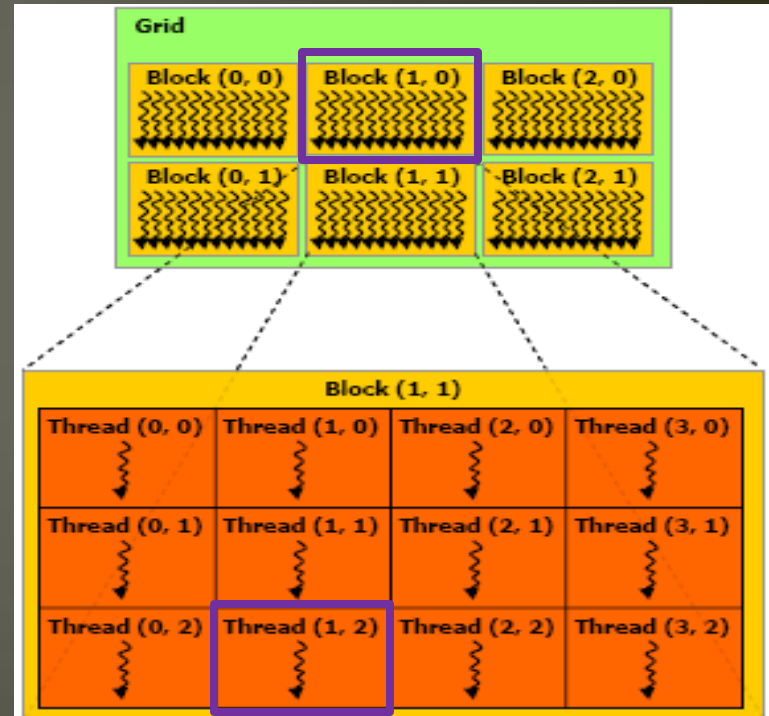
- Calculons l'ID global du thread encadré

- BlockIdx.x = 1
- BlockIdx.y = 0
- GridDim.x = 3
- GridDim.y = 2



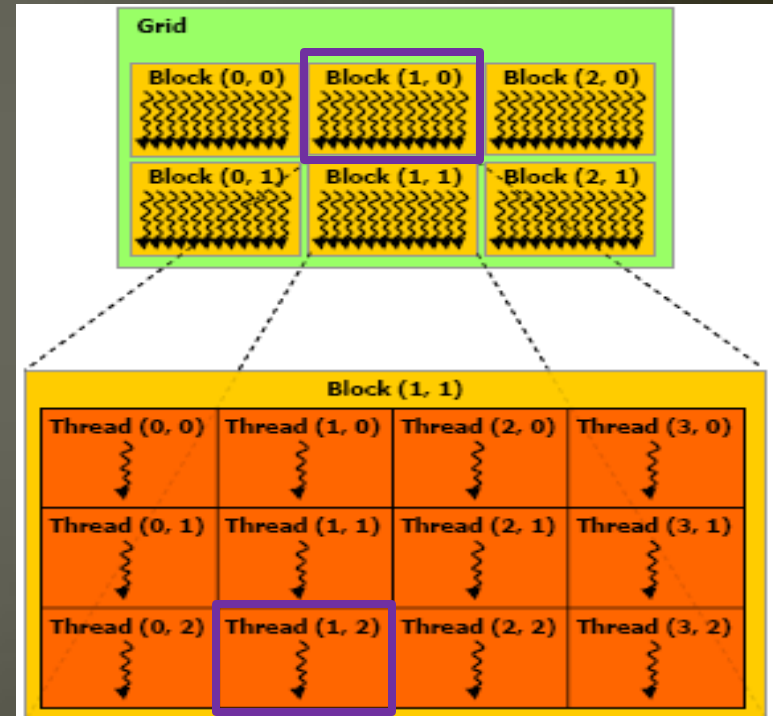
Hiérarchie de threads

- Calculons l'ID global du thread encadré
 - $\text{BlockIdx.x} = 1$
 - $\text{BlockIdx.y} = 0$
 - $\text{GridDim.x} = 3$
 - $\text{GridDim.y} = 2$
- $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$



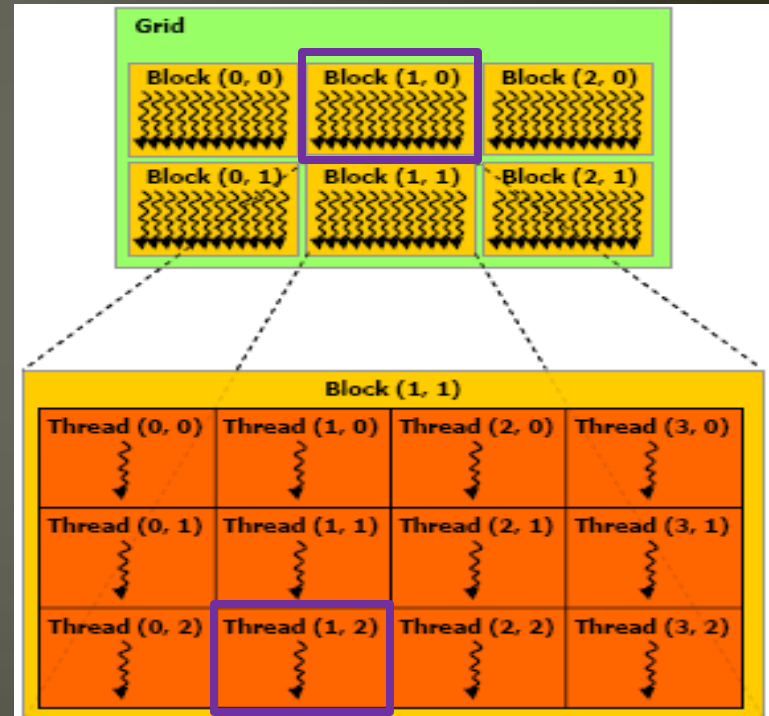
Hiérarchie de threads

- Calculons l'ID global du thread encadré
 - $\text{BlockIdx.x} = 1$
 - $\text{BlockIdx.y} = 0$
 - $\text{GridDim.x} = 3$
 - $\text{GridDim.y} = 2$
- $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
 - $\text{BlockDim.x} = 4$
 - $\text{BlockDim.y} = 3$



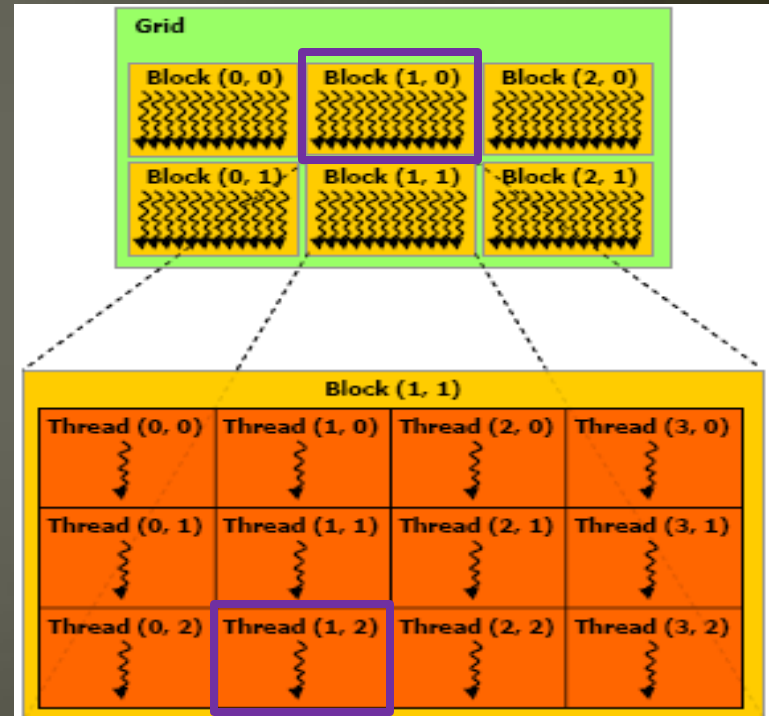
Hiérarchie de threads

- Calculons l'ID global du thread encadré
 - $\text{BlockIdx.x} = 1$
 - $\text{BlockIdx.y} = 0$
 - $\text{GridDim.x} = 3$
 - $\text{GridDim.y} = 2$
- $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
 - $\text{BlockDim.x} = 4$
 - $\text{BlockDim.y} = 3$
- $\text{Nb_threads_per_block} = \text{BlockDim.x} \times \text{BlockDim.y} = 4 \times 3 = 12$



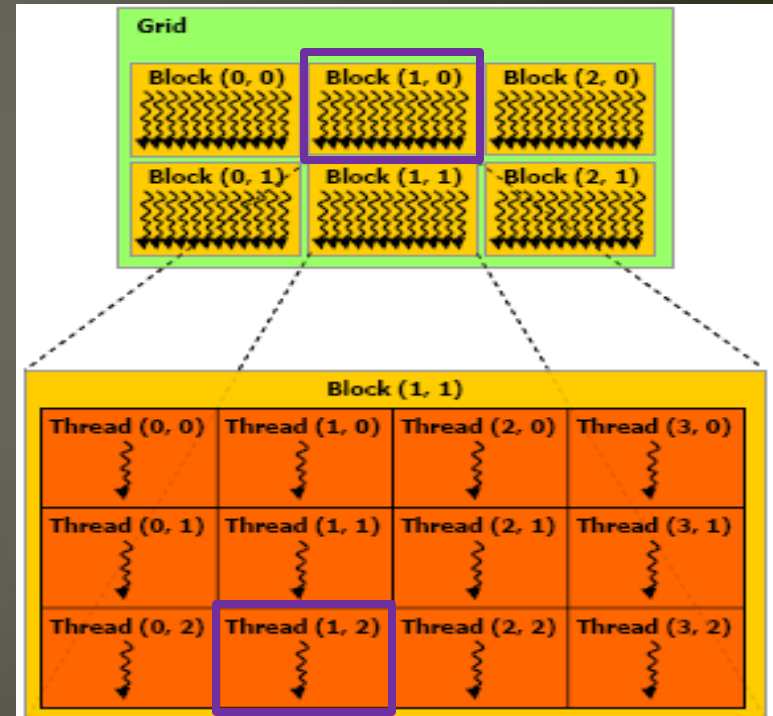
Hiérarchie de threads

- Calculons l'ID global du thread encadré
 - $\text{BlockIdx.x} = 1$
 - $\text{BlockIdx.y} = 0$
 - $\text{GridDim.x} = 3$
 - $\text{GridDim.y} = 2$
- $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
 - $\text{BlockDim.x} = 4$
 - $\text{BlockDim.y} = 3$
- $\text{Nb_threads_per_block} = \text{BlockDim.x} \times \text{BlockDim.y} = 4 \times 3 = 12$
 - $\text{threadIdx.x} = 1$
 - $\text{threadIdx.y} = 2$
 - $\text{BlockDim.x} = 4$



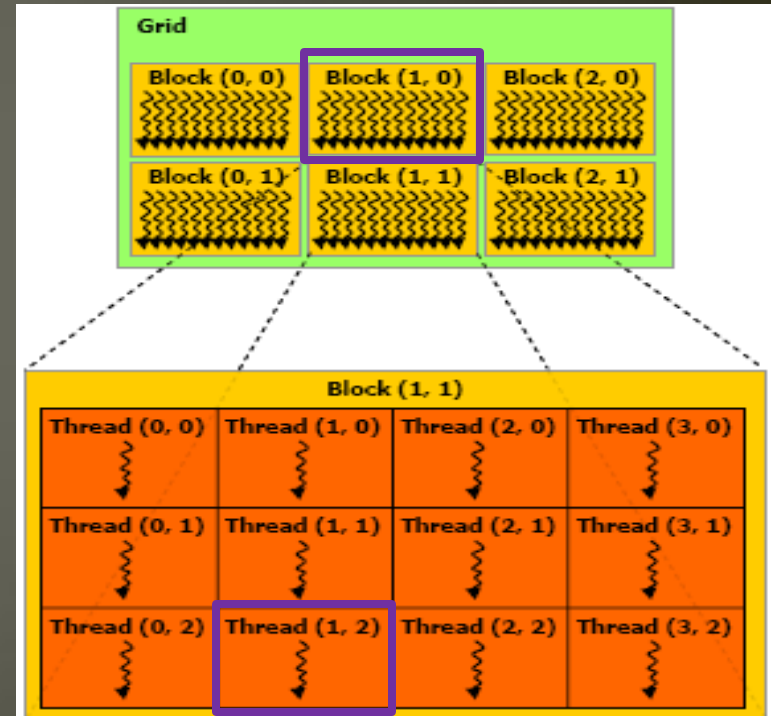
Hiérarchie de threads

- Calculons l'ID global du thread encadré
 - $\text{BlockIdx.x} = 1$
 - $\text{BlockIdx.y} = 0$
 - $\text{GridDim.x} = 3$
 - $\text{GridDim.y} = 2$
- $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
 - $\text{BlockDim.x} = 4$
 - $\text{BlockDim.y} = 3$
- $\text{Nb_threads_per_block} = \text{BlockDim.x} \times \text{BlockDim.y} = 4 \times 3 = 12$
 - $\text{threadIdx.x} = 1$
 - $\text{threadIdx.y} = 2$
 - $\text{BlockDim.x} = 4$
- $\text{Thread_loc_id} = \text{threadIdx.y} \times \text{BlockDim.x} + \text{threadIdx.x} = 2 \times 4 + 1 = 9$



Hiérarchie de threads

- Calculons l'ID global du thread encadré
 - $\text{BlockIdx.x} = 1$
 - $\text{BlockIdx.y} = 0$
 - $\text{GridDim.x} = 3$
 - $\text{GridDim.y} = 2$
- $\text{Block_glob_id} = \text{BlockIdx.y} \times \text{GridDim.x} + \text{BlockIdx.x} = 0 \times 3 + 1 = 1$
 - $\text{BlockDim.x} = 4$
 - $\text{BlockDim.y} = 3$
- $\text{Nb_threads_per_block} = \text{BlockDim.x} \times \text{BlockDim.y} = 4 \times 3 = 12$
 - $\text{threadIdx.x} = 1$
 - $\text{threadIdx.y} = 2$
 - $\text{BlockDim.x} = 4$
- $\text{Thread_loc_id} = \text{threadIdx.y} \times \text{BlockDim.x} + \text{threadIdx.x} = 2 \times 4 + 1 = 9$
- $\text{Thread_glob_id} = \text{Block_glob_id} \times \text{Nb_threads_per_block} + \text{Thread_loc_id} = 1 \times 12 + 9 = 21$



Programmation CUDA

- **Déclaration d'un kernel**
 - Mot clé `__global__`
- **Calcul de l'indice du thread en fonction de la géométrie de la grille**
 - Exemple avec une grille à 1 dimension et des blocs à 1 dimensions
- **Test pour s'assurer que les threads n'écrivent pas en dehors du tableau**
 - Correspond à la borne de la boucle d'origine

```
__global__ void vecAddKernel(  
    double *a,  
    double *b,  
    double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x  
    + threadIdx.x ;  
  
    if ( i < N ) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Intégration des *kernels*

- Comment se forment les blocs et la grille ?
- Besoin de le préciser lors de l'exécution du kernel
 - Le programme hôte est en charge de définir les dimensions du calcul
- Syntaxe d'un appel à un noyau de calcul du *device*
`my_kernel<<<Dg, Db>>>(arg1, arg2, arg3 );`
 - Dg : dimensions et taille de la grille (type dim3)
 - Db : dimensions et taille des blocs de la grille (type dim3)
- Nombre total de blocs ?
- Nombre de threads par bloc ?
- Nombre total de threads ?

Programme CUDA complet

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_c, c, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i<N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```

Programme CUDA complet

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i<N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```

Programme CUDA complet

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i<N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```

Programme CUDA complet

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i<N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```

Programme CUDA complet

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i<N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```

Programme CUDA complet

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i < N ) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Programme CUDA complet

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i<N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```

Programme CUDA complet

HOST

```
void vectAdd(  
    double * a, double * b, double * c, int N) {  
  
    double * d_a ;  
    double * d_b ;  
    double * d_c ;  
  
    cudaMalloc((void **)&d_a, N*sizeof(double));  
    cudaMalloc((void **)&d_b, N*sizeof(double));  
    cudaMalloc((void **)&d_c, N*sizeof(double));  
  
    cudaMemcpy(d_a, a, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
    cudaMemcpy(d_b, b, N*sizeof(double),  
        cudaMemcpyHostToDevice);  
  
    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);  
  
    cudaMemcpy(c, d_c, N*sizeof(double),  
        cudaMemcpyDeviceToHost);  
  
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);  
}
```

DEVICE

```
__global__ void vecAddKernel( double  
    *a, double *b, double *c, int N) {  
  
    int i ;  
  
    i = blockIdx.x * blockDim.x +  
        threadIdx.x ;  
  
    if ( i<N ) {  
        c[i] = a[i]+b[i];  
    }  
}
```


Programme CUDA complet

HOST

```
void vectAdd(
    double * a, double * b, double * c, int N) {

    double * d_a ;
    double * d_b ;
    double * d_c ;

    cudaMalloc((void **)&d_a, N*sizeof(double));
    cudaMalloc((void **)&d_b, N*sizeof(double));
    cudaMalloc((void **)&d_c, N*sizeof(double));

    cudaMemcpy(d_a, a, N*sizeof(double),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N*sizeof(double),
        cudaMemcpyHostToDevice);

    vectAddKernel<<<32,64>>>(d_a, d_b, d_c, N);

    cudaMemcpy(c, d_c, N*sizeof(double),
        cudaMemcpyDeviceToHost);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
}
```

DEVICE

```
__global__ void vecAddKernel( double
    *a, double *b, double *c, int N) {

    int i ;

    i = blockIdx.x * blockDim.x +
        threadIdx.x ;

    if ( i<N ) {
        c[i] = a[i]+b[i];
    }

}
```

Architecture et programmation d'accélérateurs matériels

Cours 2 : Programmation CUDA avancée

Julien Jaeger
julien.jaeger@cea.fr

Programmation avancée CUDA

Plan - Programmation

- Langage de programmation
 - Mots clés
 - Fonctions disponibles
 - Exemples
- Optimisations de *kernel*
- Multi-GPUs

Programmation CUDA

- Noyau de calcul
 - Basé sur la norme C99
 - Quelques restrictions
 - Quelques ajouts
- Extensions
 - Mots clés
 - Variables définies par défaut
 - Fonctions
- Voir l'appendice B de la documentation *CUDA C Programming Guide*

Noyau de base

- **Syntaxe de base pour un noyau**
 - Fonction qui ne renvoie rien (retour de type void)
 - Attribut définissant la fonction comme s'exécutant sur le *device*
 - `__global__`
 - Arguments en entrée

- **Exemple**

```
__global__ void vecAddKernel( double *a,  
                             double *b, double *c, int N ) {  
    int i ;  
    i = blockIdx.x * blockDim.x + threadIdx.x ;  
    if ( i < N ) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Extensions – Mots clés

- Définition de nouveaux mots clés
- Catégories
 - Attributs de fonctions
 - Attributs de variables
 - Types
- Ensemble de variables définies par défaut
 - Utilisant les nouveaux types de données

Attributs de fonction

- Mot clé à ajouter dans la déclaration et la définition de la fonction
 - Ajout entre le type de retour et le nom de la fonction
- Fonction s'exécutant sur le device et callable depuis l'hôte
 - `__global__`
- Fonction dédiée sur l'hôte ou le device (combinable)
 - `__host__`
 - `__device__`
- Par défaut, équivalent à `__host__`

Restrictions des fonctions

- **Fonction déclarée `__global__`**
 - Type de retour `void`
 - Appel avec un contexte d'exécution (nombre de blocs, nombre de threads par bloc, ...)
 - Appel asynchrone
 - Impossible de capturer son pointeur
- **Fonction s'exécutant sur le *device***
 - Pas de variable statique
 - Pas de nombre d'arguments variable
 - Récursion restreinte (uniquement pour les fonctions déclarées `__device__`)

Type de données

- Nouveau types

- Vecteur
- Entiers multi-dimensions

- Vecteurs

- Type de base + nombre de données
- Exemple : int2, float4
- Besoin de respecter les règles d'alignements
- Fonctions associées pour construire un tel type
 - Exemple : `int2 make_int2(int x, int y);`

Type de données (suite)

- Entiers 3 dimensions

- `dim3`
- Equivalent au type de vecteur `uint3`
- Accès aux composantes par les champs `x`, `y` et `z`
- Par défaut, initialisé à 1
- Type de données utilisées pour les coordonnées et dimensions de la grille, des blocs et des threads CUDA.

- Exemple

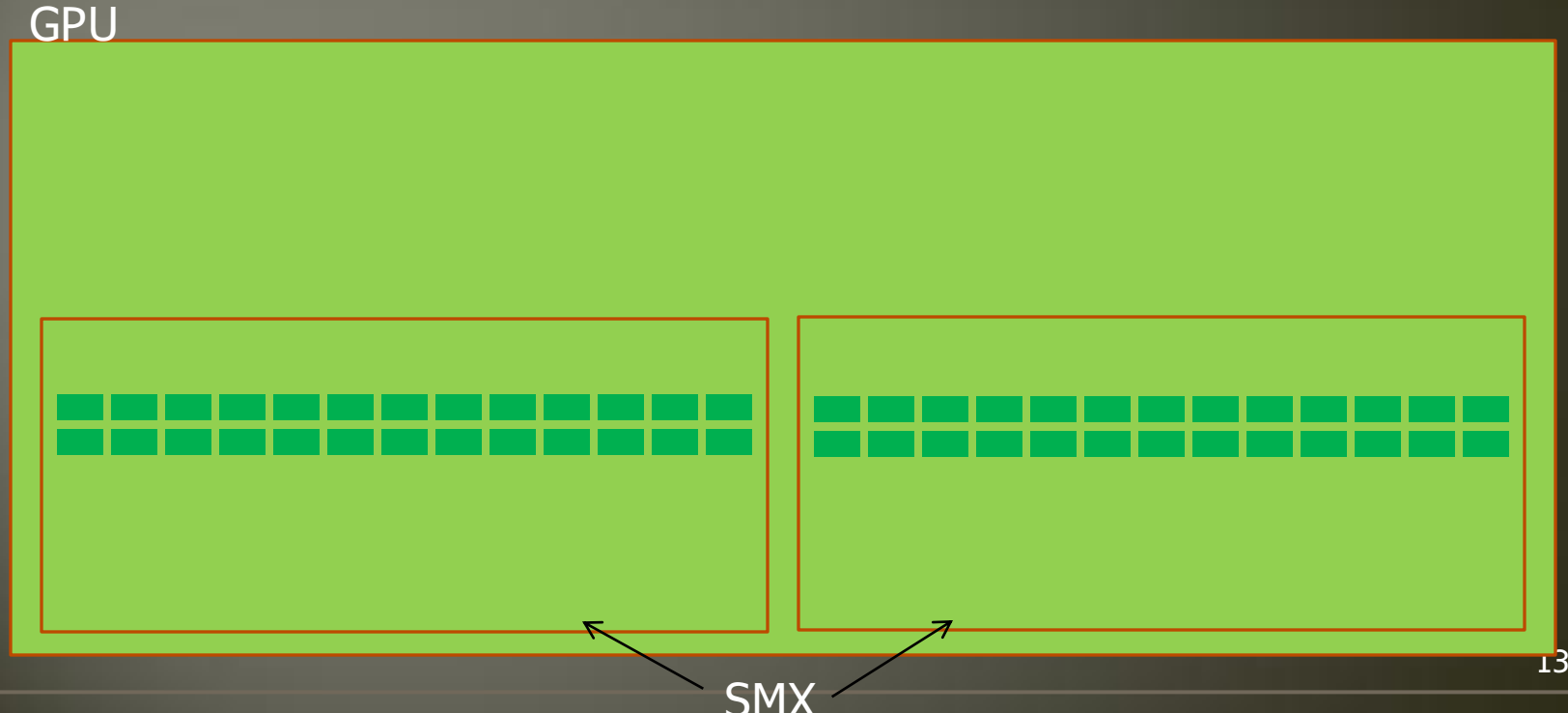
```
dim3 a ;  
a.x = 4 ;
```

Gestion de la mémoire

Hiérarchie mémoire d'un GPU: résumé

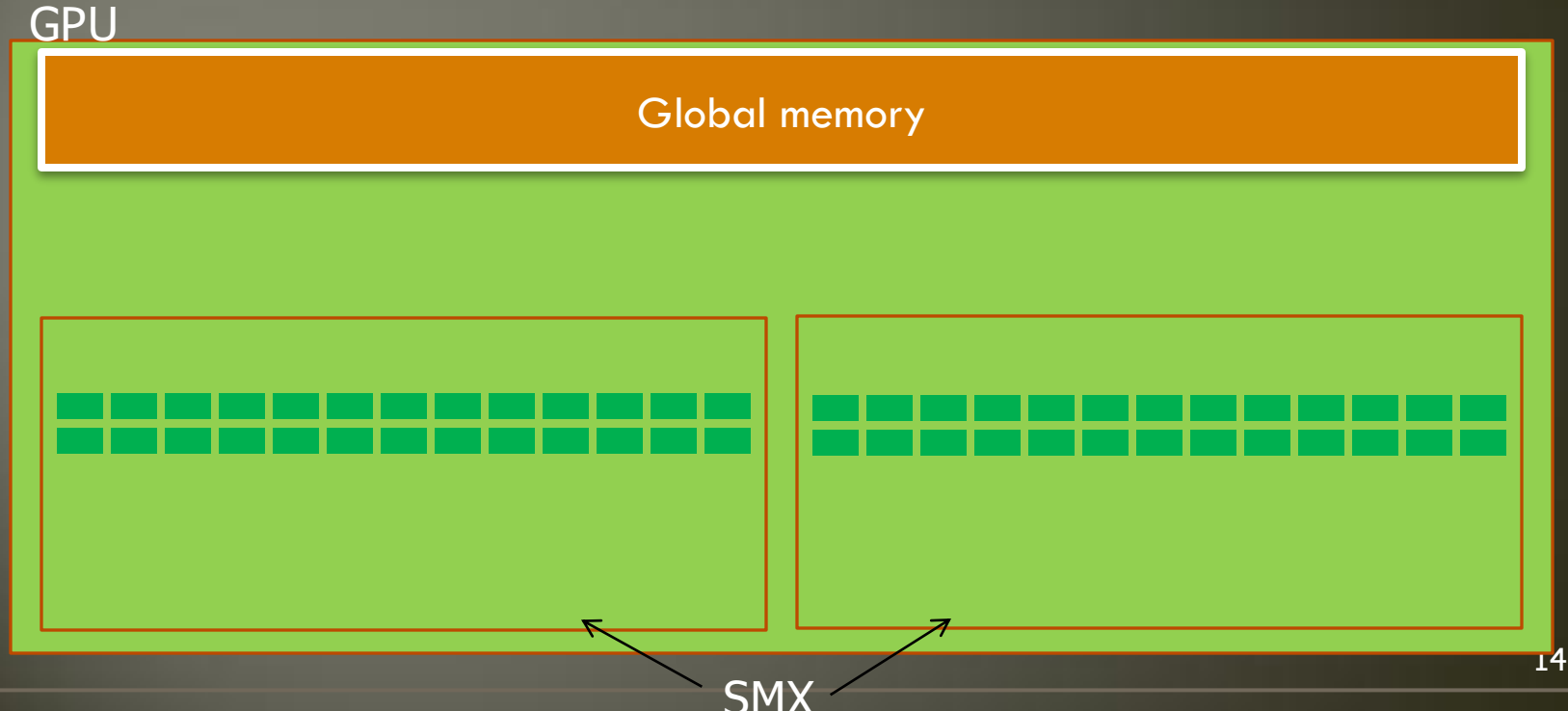
- Les GPUs Nvidias actuels ont une hiérarchie mémoire complexe
 - Plusieurs mémoires....
 - ... avec des systèmes d'accès différents...
 - ... avec des localités d'accès différentes...
 - ... manipulées de façon distinctes...
 - ... la plupart du temps directement par l'utilisateur

Hiérarchie mémoire d'un GPU



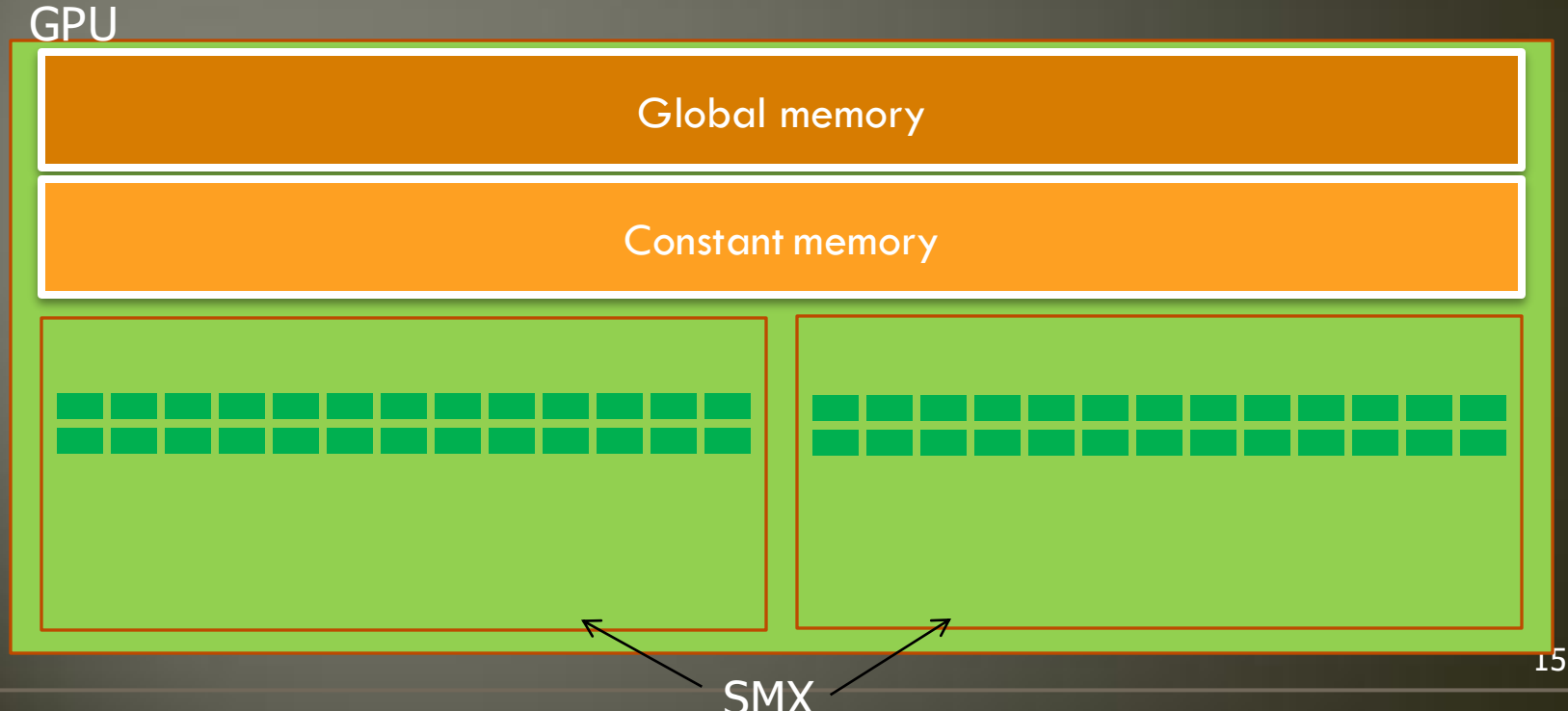
Hiérarchie mémoire d'un GPU

Mémoire globale



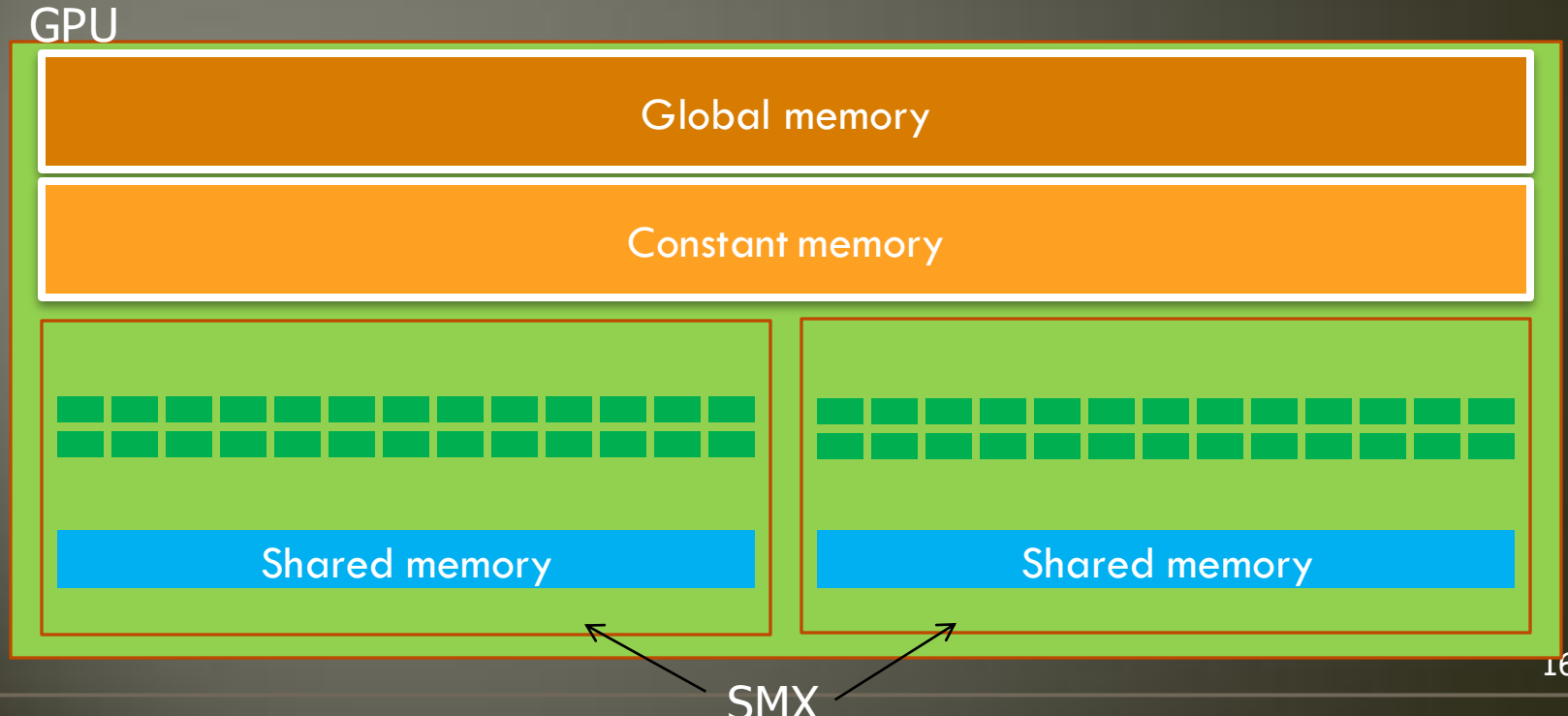
Hiérarchie mémoire d'un GPU

Mémoire constante



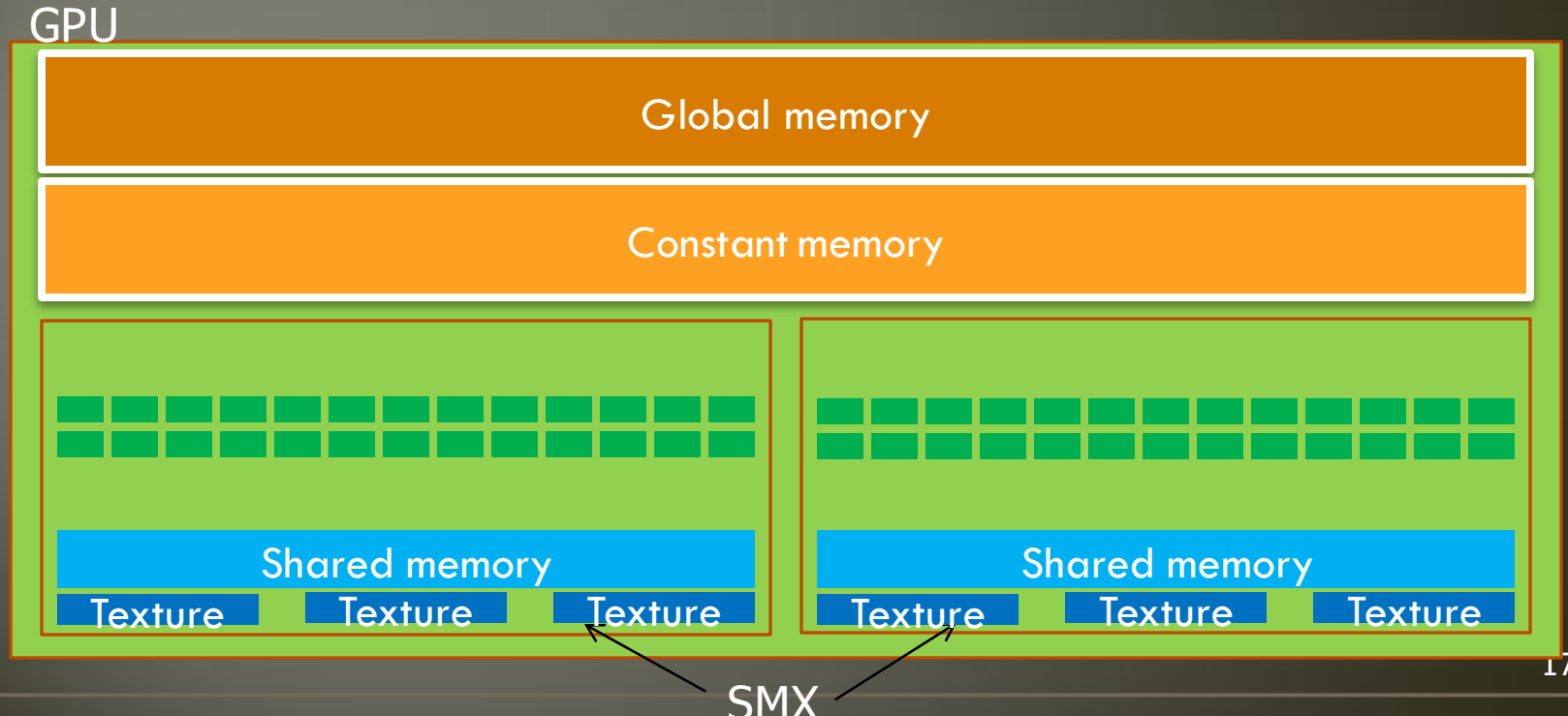
Hiérarchie mémoire d'un GPU

Mémoire partagée



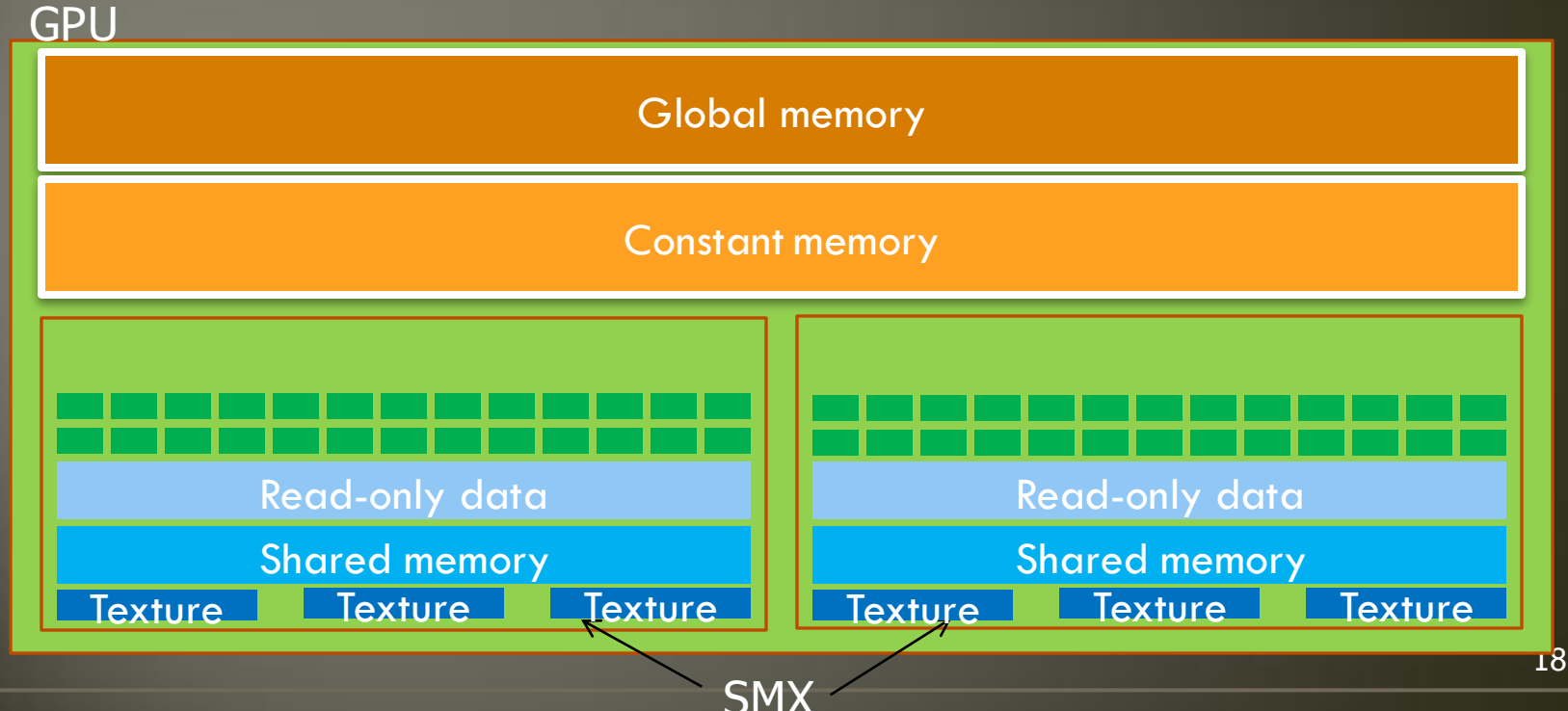
Hiérarchie mémoire d'un GPU

Mémoire de texture



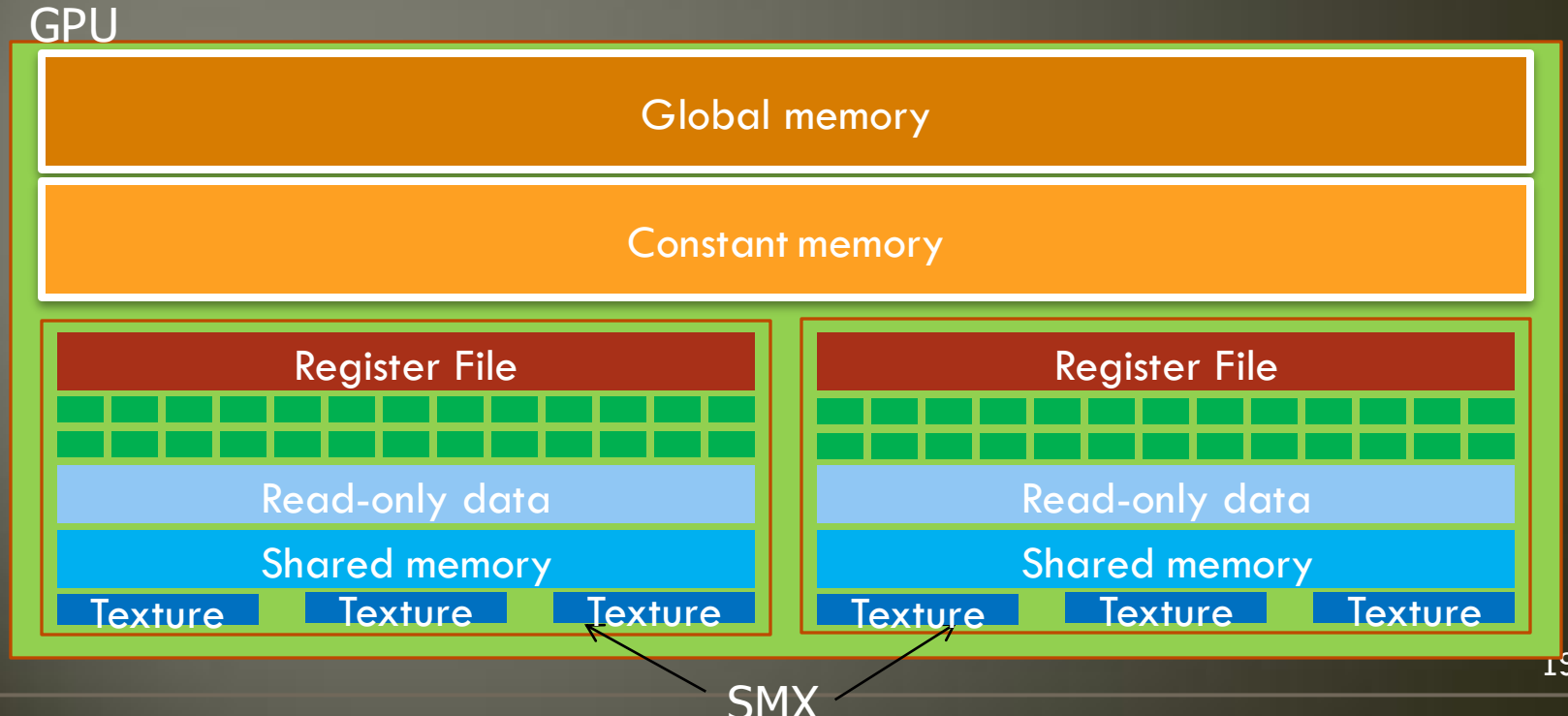
Hiérarchie mémoire d'un GPU

Mémoire data lecture-seule



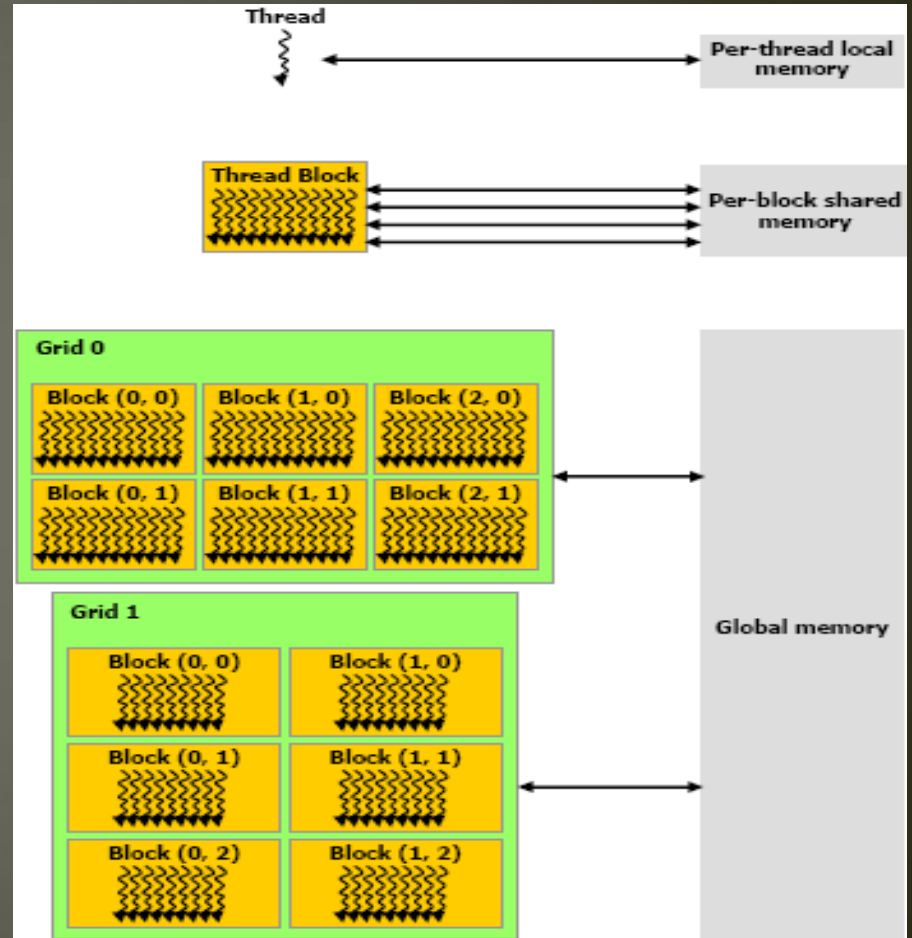
Hiérarchie mémoire d'un GPU

Ensemble de Registres

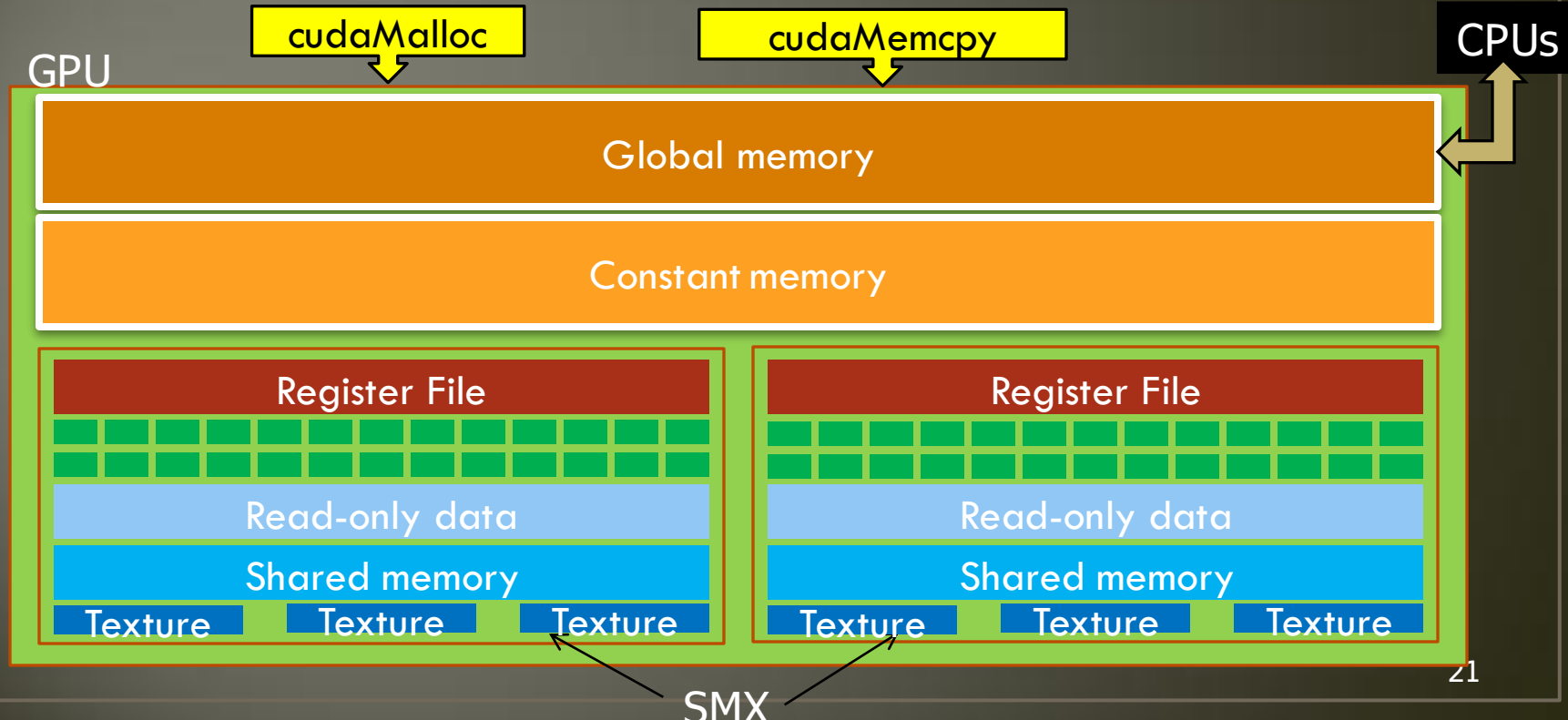


Hiérarchie mémoire

- Vision par entité de calcul
- Thread
 - Accès mémoire local
 - Banc de registres privé
- Bloc
 - Shared memory privé
- Grille
 - Mémoire globale



Alloc et transfert: Global Mem.



Allocation de base sur le GPU

- `__host__ __device__ cudaError_t cudaMalloc (void
** ptr, size_t size)`

Allocation de base sur le GPU

- `__host__ __device__ cudaError_t cudaMalloc (void ** ptr, size_t size)`
 - Alloue de la mémoire sur le dévise
 - Alloue size octets, dans la mémoire globale du device

Allocation de base sur le GPU

- `__host__ __device__ cudaError_t cudaMalloc (void ** ptr, size_t size)`
 - Alloue de la mémoire sur le dévise
 - Alloue size octets, dans la mémoire globale du device
 - Donne l'adresse du pointeur ptr
 - Le runtime CUDA s'occupe de l'allocation et récupère l'adresse de la zone mémoire
 - L'adresse est renvoyée et stockée dans ptr
 - L'adresse n'est pas utilisable sur le host!!!

Allocation de base sur le GPU

- `__host__ __device__ cudaError_t cudaMalloc (void ** ptr, size_t size)`
 - Alloue de la mémoire sur le dévise
 - Alloue size octets, dans la mémoire globale du device
 - Donne l'adresse du pointeur ptr
 - Le runtime CUDA s'occupe de l'allocation et récupère l'adresse de la zone mémoire
 - L'adresse est renvoyée et stockée dans ptr
 - L'adresse n'est pas utilisable sur le host!!!

```
int * d_a = NULL;  
cudaMalloc(&d_a, ( sizeof(int) * 1024));
```

Allocation avancée sur le GPU

- `__host__ cudaError_t cudaMallocPitch (void ** ptr,
size_t * pitch,
size_t width, size_t height)`
 - Alloue de la mémoire 2D sur le device
 - Alloue au moins width x height

Allocation avancée sur le GPU

- `__host__ cudaError_t cudaMallocPitch (void ** ptr, size_t * pitch, size_t width, size_t height)`
 - Alloue de la mémoire 2D sur le device
 - Alloue au moins width x height
 - Les allocations mémoires subissent des contraintes d'alignement
 - Peut avoir un impact sur les allocations 2D et 3D
 - Chaque ligne doit être correctement alignée
 - Possible qu'un padding en fin de ligne soit nécessaire
 - La taille réelle d'une ligne (width+padding) est renvoyée dans la variable pitch

Allocation avancée sur le GPU

- `__host__ cudaError_t cudaMalloc3D (struct
 cudaPitchPtr * pitchedDevPtr,
 struct cudaExtent extent)`

Allocation avancée sur le GPU

- `__host__ cudaError_t cudaMalloc3D (struct
 cudaPitchPtr * pitchedDevPtr,`

Struct cudaExtent extent)

- Spécifie le minimum d'octets à allouer
- La structure `cudaExtent` contient trois champs
 - `size_t depth`
 - `size_t height`
 - `size_t width`
- Alloue au minimum `depth x height x width` octets

Allocation avancée sur le GPU

- `__host__ cudaError_t cudaMalloc3D ( struct cudaPitchPtr * pitchedDevPtr,`
 - Récupère l'adresse de la mémoire allouée sur le device
 - Plus quelques infos stockée dans la structure liées aux contraintes d'alignement
 - `size_t size` et `size_t ysize` : correspondent aux champs `width` et `height` de la structure `extent` passée à l'allocation
 - `size_t pitch` : la taille réelle de la zone mémoire allouée (avec le padding nécessaire dans chaque dimension)`struct cudaExtent extent)`

Copie de base sur le GPU

- `__host__ cudaError_t cudaMemcpy`
`(void * dst, const void * src,`
`size_t count, enum cudaMemcpyKind kind)`

Copie de base sur le GPU

- `__host__ cudaError_t cudaMemcpy`
(`void * dst`, `const void * src`,
`size_t count`, `enum cudaMemcpyKind kind`)
 - Copie `count` octets de la mémoire pointée par `src` vers la mémoire pointée par `dst`

Copie de base sur le GPU

- `__host__ cudaError_t cudaMemcpy`
`(void * dst, const void * src,`
`size_t count, enum cudaMemcpyKind kind)`
 - Copie count octets de la mémoire pointée par src vers la mémoire pointée par dst
 - **Kind** permet de donner la direction de la copie
 - `cudaMemcpyHostToDevice`
 - `cudaMemcpyDeviceToHost`
 - `cudaMemcpyHostToHost`
 - `cudaMemcpyDeviceToDevice`

Copie de base sur le GPU

- `__host__ cudaError_t cudaMemcpy`
(`void * dst, const void * src,`
`size_t count, enum cudaMemcpyKind kind`)
 - Copie count octets de la mémoire pointée par src vers la mémoire pointée par dst
 - Kind permet de donner la direction de la copie
 - `cudaMemcpyHostToDevice`
 - `cudaMemcpyDeviceToHost`
 - `cudaMemcpyHostToHost`
 - `cudaMemcpyDeviceToDevice`

```
cudaMemcpy(d_a, h_a, ( sizeof(int) * 1024), cudaMemcpyHostToDevice);
```

Copie avancée sur le GPU

- `__host__ cudaError_t cudaMemcpy2D`
`(void * dst, size_t dpitch,`
`const void * src, size_t spitch,`
`size_t width, size_t height,`
`enum cudaMemcpyKind kind)`
 - Copie **width x height** octets de la mémoire pointée par **src** vers la mémoire pointée par **dst**

Copie avancée sur le GPU

- `__host__ cudaError_t cudaMemcpy2D`
(void * dst, **size_t dpitch**,
const void * src, **size_t spitch**,
size_t width, size_t height,
enum cudaMemcpyKind kind)

- Copie width x height octets de la mémoire pointée par src vers la mémoire pointée par dst
- Permet de spécifier le padding pour les deux zones mémoires, source et destination
 - Compatible avec les zones mémoires allouées avec `cudaMallocPitch`

Copie avancée sur le GPU

- `__host__ cudaError_t cudaMemcpy2D`
`(void * dst, size_t dpitch,`
`const void * src, size_t spitch,`
`size_t width, size_t height,`
`enum cudaMemcpyKind kind)`

- Copie width x height octets de la mémoire pointée par src vers la mémoire pointée par dst
- Permet de spécifier le padding pour les deux zones mémoires, source et destination

- Compatible avec les zones mémoires allouées avec `cudaMallocPitch`

```
cudaMemcpy2D(d_a, pitch, h_a, ( sizeof(int) * 64), ( sizeof(int) * 64),  
            16, cudaMemcpyHostToDevice);
```

Copie avancée sur le GPU

- `__host__ cudaError_t cudaMemcpy3D (const struct cudaMemcpy3DParms * p)`

Copie avancée sur le GPU

- `__host__ cudaError_t cudaMemcpy3D (const struct cudaMemcpy3DParms * p)`
 - `struct cudaArray *srcArray;`
 - `struct cudaPos srcPos – (size_t x, size_t y, size_t z)`
 - `struct cudaPitchedPtr srcPtr;`
 - `struct cudaArray *dstArray;`
 - `struct cudaPos dstPos;`
 - `struct cudaPitchedPtr dstPtr;`
 - `struct cudaExtent extent;`
 - `enum cudaMemcpyKind kind;`

Example code 2D

```
__global__ void plus_one(int * a, int size, size_t pitch)
{
    int y = blockIdx.x;
    int x = threadIdx.x;
    int rp = (int)(pitch / sizeof(int));

    int test = blockIdx.x * blockDim.x + threadIdx.x;

    if(test < size)
    {
        a[y * rp + x] +=1;
    }
}

int main (int argc, char * argv[])
{
    int * h_a = NULL;
    h_a = (int *)malloc( sizeof(int) * 1024);

    int i;
    for(i=0; i<1024; i++)
    {
        h_a[i] = 10;
    }

    int * d_a = NULL;
    size_t pitch;
    cudaMallocPitch(&d_a, &pitch, ( sizeof(int) * 64), 16);

    cudaMemcpy2D(d_a, pitch, h_a, ( sizeof(int) * 64), ( sizeof(int) * 64), 16, cudaMemcpyHostToDevice);

    plus_one<<<16, 64>>>(d_a, 1024, pitch);

    cudaMemcpy2D(h_a, ( sizeof(int) * 64), d_a, pitch, ( sizeof(int) * 64), 16, cudaMemcpyDeviceToHost);

    for(i=0; i<1024; i++)
    {
        printf("[%d]", h_a[i]);
    }
    printf("\n");

    return 0;
}
```

Autres fonctions avancées

- Allocation

- `cudaMallocArray`
- `cudaMalloc3DArray`

- Copie

- `cudaMemcpyToArray`
- `cudaMemcpy2DToArray`
- `cudaMemcpyFromArray`
- `cudaMemcpy2DFromArray`
- `cudaMemcpyArrayToArray`
- `cudaMemcpy2DArrayToArray`

Gestion automatique de la mémoire (mémoire unifiée)

- Introduit avec CUDA 6.0
- Vue unifiée de la mémoire entre host et devices
 - Un seul pointeur est utilisée pour la mémoire sur le host ou sur le(s) GPU(s)
 - Les transferts sont réalisés automatiquement en fonction de l'utilisation des données sur le host ou sur un device

Gestion automatique de la mémoire (mémoire unifiée)

- Deux façons pour demander de la mémoire gérée automatiquement
 - `__host__ cudaMallocManaged( void** devPtr, size_t size, unsigned int flags)`
 - `cudaMemAttachGlobal`
 - Memory can be accessed from any devices
 - `cudaMemAttachHost`
 - Memory cannot be accessed from any devices
 - `__managed__` attribut devant le nom des variables

Mémoire unifiée: Exemple

- cudaMallocManaged
 - Default flag: cudaMemAttachGlobal

```
__global__ void AplusB(int *ret, int a, int b) {  
    ret[threadIdx.x] = a + b + threadIdx.x;  
}  
  
int main() {  
    int *ret;  
    cudaMallocManaged(&ret, 1000 * sizeof(int));  
    AplusB<<< 1, 1000 >>>(ret, 10, 100);  
}
```

Mémoire unifiée: Exemple

- Attribut `__managed__`

```
__device__ __managed__ int ret[1000];
__global__ void AplusB(int a, int b) {
    ret[threadIdx.x] = a + b + threadIdx.x;
}

int main() {
    AplusB<<< 1, 1000 >>>(10, 100);
    cudaDeviceSynchronize();
}
```

Gestion automatique de la mémoire (mémoire unifiée)

- Introduit avec CUDA 6.0
- Vue unifiée de la mémoire entre host et devices
 - Un seul pointeur est utilisée pour la mémoire sur le host ou sur le(s) GPU(s)
 - Les transferts sont réalisés automatiquement en fonction de l'utilisation des données sur le host ou sur un device

Gestion automatique de la mémoire (mémoire unifiée)

- Introduit avec CUDA 6.0
- Vue unifiée de la mémoire entre host et devices
 - Un seul pointeur est utilisée pour la mémoire sur le host ou sur le(s) GPU(s)
 - Les transferts sont réalisés automatiquement en fonction de l'utilisation des données sur le host ou sur un device
- /!\ Attention aux problèmes de performances en cas de ping-pong CPU-GPU

Mémoire unifiée

Perf issue example

■ Exemple:

- Réalise 2000 fois l'invocation de kernel et function pour update le meme tableau

```
__global__ void kernel(int * a)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    a[i]++;
}

__host__ void function(int * a)
{
    int i;
    for(i=0; i< 100000; i++) a[i]++;
}

cudaMallocManaged(&a, 100000*sizeof(int));

for(i=0; i<100000; i++) a[i] = i;

gettimeofday(&start1, NULL);
for(j=0; j<2000; j++)
{
    kernel<<<100,1000>>>(a);
    cudaDeviceSynchronize();
    function(a);
}
gettimeofday(&stop1, NULL);

gettimeofday(&start2, NULL);
for(j=0; j<2000; j++)
{
    kernel<<<100,1000>>>(a);
    cudaDeviceSynchronize();
}
for(j=0; j<2000; j++)
{
    function(a);
}
gettimeofday(&stop2, NULL);
```

Mémoire unifiée

Perf issue example

■ Exemple:

- Réalise 2000 fois l'invocation de kernel et function pour update le meme tableau
- Ping-pong: 2000 fois kernel + function

```
__global__ void kernel(int * a)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    a[i]++;
}

__host__ void function(int * a)
{
    int i;
    for(i=0; i< 100000; i++) a[i]++;
}

cudaMallocManaged(&a, 100000*sizeof(int));

for(i=0; i<100000; i++) a[i] = i;

gettimeofday(&start1, NULL);
for(j=0; j<2000; j++)
{
    kernel<<<100,1000>>>(a);
    cudaDeviceSynchronize();
    function(a);
}
gettimeofday(&stop1, NULL);

gettimeofday(&start2, NULL);
for(j=0; j<2000; j++)
{
    kernel<<<100,1000>>>(a);
    cudaDeviceSynchronize();
}
for(j=0; j<2000; j++)
{
    function(a);
}
gettimeofday(&stop2, NULL);
```

Mémoire unifiée

Perf issue example

■ Exemple:

- Réalise 2000 fois l'invocation de kernel et function pour update le meme tableau
- Ping-pong: 2000 fois kernel + function
- Grouped : 2000 fois kernel puis 2000 fois function

```
__global__ void kernel(int * a)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    a[i]++;
}

__host__ void function(int * a)
{
    int i;
    for(i=0; i< 100000; i++)      a[i]++;
}

cudaMallocManaged(&a, 100000*sizeof(int));

for(i=0; i<100000; i++) a[i] = i;

gettimeofday(&start1, NULL);
for(j=0; j<2000; j++)
{
    kernel<<<100,1000>>>(a);
    cudaDeviceSynchronize();
    function(a);
}
gettimeofday(&stop1, NULL);

gettimeofday(&start2, NULL);
for(j=0; j<2000; j++)
{
    kernel<<<100,1000>>>(a);
    cudaDeviceSynchronize();
}
for(j=0; j<2000; j++)
{
    function(a);
}
gettimeofday(&stop2, NULL);
```

Mémoire unifiée

Perf issue example

■ Exemple:

- Réalise 2000 fois l'invocation de kernel et function pour update le meme tableau
- Ping-pong: 2000 fois kernel + function
- Grouped : 2000 fois kernel puis 2000 fois function

■ Temps mesurés:

```
$ ./managed.pgr  
time ping-pong = 1.25968003 | time grouped = 0.39084899
```

```
__global__ void kernel(int * a)  
{  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    a[i]++;  
}  
  
__host__ void function(int * a)  
{  
    int i;  
    for(i=0; i< 100000; i++) a[i]++;  
}  
  
cudaMallocManaged(&a, 100000*sizeof(int));  
  
for(i=0; i<100000; i++) a[i] = i;  
  
gettimeofday(&start1, NULL);  
for(j=0; j<2000; j++)  
{  
    kernel<<<100,1000>>>(a);  
    cudaDeviceSynchronize();  
    function(a);  
}  
gettimeofday(&stop1, NULL);  
  
gettimeofday(&start2, NULL);  
for(j=0; j<2000; j++)  
{  
    kernel<<<100,1000>>>(a);  
    cudaDeviceSynchronize();  
}  
for(j=0; j<2000; j++)  
{  
    function(a);  
}  
gettimeofday(&stop2, NULL);
```

Accès mémoire

- Exécution synchrone dans un warp
 - Un thread exécutant un chargement mémoire ralentit les autres
 - Plusieurs accès mémoire simultanées peuvent être sérialisés
 - Plus l'accès est long, plus il nous faudra de threads pour recouvrir cet accès
- Optimisations possibles
 - *Load coalescing*
 - Eviter les conflits de bancs

Load coalescing

- Accès à la mémoire globale
- Accès concurrent émis par les threads d'un même warp
 - Tous les threads d'un même warp exécute la même instruction au même instant
- Les requêtes sont sérialisés par paquets de 128 octets (taille de la ligne de cache)
 - Optimisation si ces accès sont contigus !

Load coalescing et mémoire shared

- Utilisation de la mémoire shared
 - Nécessite de déclarer des *buffers* résidant dans la mémoire shared
 - Transferts des données en début de noyau
 - Mise à jour de la mémoire globale à la fin du noyau
- Optimisation : profiter de ce premier transferts (global → shared) pour faire des accès contigus aux données
 - Même si toutes les données ne sont pas nécessaires !
- Attention aux conflits de bancs

Attributs de variables (1)

- Variable résidente sur le *device*
 - `__device__`
 - Par défaut dans la mémoire globale, accessible par tous les threads, pendant toute la durée de l'application
- Variable résidente dans la mémoire constante
 - `__constant__`
 - Durée de vie de l'application
 - Ne peut pas être défini sur le *device*

Attributs de variables (2)

- Variable dans la mémoire *shared*
 - `__shared__`
 - Partagée entre tous les threads d'un même bloc
 - Une copie par bloc
 - Durée de vie du bloc
- Par défaut une variable déclarée sur le *device* est stockée dans un registre

Variable volatile

- Synchronisation des données communes accédées de façon concurrente

- Exemple d'accès concurrents

```
// myArray is an array of non-zero integers
// located in global or shared memory
__global__ void MyKernel(int* result) {
    int tid = threadIdx.x;
    int ref1 = myArray[tid] * 1;
    myArray[tid + 1] = 2;
    int ref2 = myArray[tid] * 1;
    result[tid] = ref1 * ref2;
}
```

- Que vaut `result[tid]` ?

- `myArray[tid]` est dans un registre, donc `ref1==ref2`

- Par contre, si déclaré *volatile*, alors ok (ou alors mettre une barrière mémoire - *memory fence*)

- Mais cela ne garantie pas l'ordre d'exécution

Restrictions des variables

- Gestion dynamique des variables *shared*

```
extern __shared__ char array[];
__device__ void func() {
    short* array0 = (short*)array;
    float* array1 = (float*)&array[128];
    int* array2 = (int*)&array[64];
}
```

- Besoin de gérer à la main l'allocation des données si on décide d'utiliser la mémoire *shared* de façon dynamique
 - Respect des règles d'alignements

Allocation de registres

- Chaque noyau de calcul a besoin d'utiliser plusieurs registres
- En fonction des instructions présentes dans le noyau
 - Transformations/optimisations du compilateur
 - Allocation de registres
- Mais
 - Le nombre de registres est limité
 - Les registres sont partagés entre les threads s'exécutant sur un même *Streaming Multiprocessor*
- Relation avec le nombre de threads ?

Allocation de registres

- Option pour définir une borne au compilateur
 - `-maxrregcount=N`
- Attribut pour donner une indication sur le nombre maximum de threads et de blocs

```
__global__ void  
__launch_bounds__ (maxThreadsPerBlock,  
__minBlocksPerMultiprocessor)  
MyKernel (...)  
{  
    ...  
}
```

Asynchronisme

Exécution asynchrone en CUDA

- Les kernels CUDA sont asynchrones
- Au retour de l'invocation d'un kernel, celui-ci n'a pas forcément déjà été exécuté
 - Lors de l'invocation de kernel, celui-ci n'est pas lancé immédiatement
 - C'est comme si on avait donné « l'ordre » au GPU d'exécuter le kernel
 - Le kernel peut être exécuté plus tard
- Pour s'assurer de l'exécution du kernel, il faut synchroniser le device

Synchronisation globale

- `__host__ __device__
cudaDeviceSynchronize();`
 - Attend que les opérations sur le device soient terminées
 - Si appelée depuis le host, alors synchronise le host et le device
 - En fonction du flag de synchronisation mis en place pour ce device

Copie asynchrone

- Possibilité de faire des copies asynchrone
 - `__host__` `cudaMemcpyAsync`
 - `__host__` `cudaMemcpyPitchAsync`
 - `__host__` `cudaMemcpy3DAsync`
- Permet d'éviter les synchronisations host / device pour les copies de données

Copie asynchrone

- Possibilité de faire des copies asynchrone
 - `__host__ cudaMemcpyAsync`
 - `__host__ cudaMemcpyPitchAsync`
 - `__host__ cudaMemcpy3DAsync`
- Permet d'éviter les synchronisations host / device pour les copies de données

```
cudaMemcpyAsync(h_a, d_a, ( sizeof(int) * 1024), cudaMemcpyDeviceToHost);  
cudaDeviceSynchronize();  
  
for(i=0; i<1024; i++)  
{  
    printf("[%d]", h_a[i]);  
}
```

> host

Asynchronisme

- Par défaut certaines fonctions *rendent la main* au programme hôte
 - Exécution de kernel
 - Copies *device* vers *device*
 - Initialisation de la mémoire
- Possibilité d'attendre la fin de l'exécution à un instant donné
`cudaDeviceSynchronize()` ;
- Appels consécutifs à notre noyau `vecAdd`
 - Si les vecteurs sont différents ?
 - S'il existe une dépendance RAW dans notre calcul ?

Asynchronisme

- Pas besoin de synchronisation globale en cas de dépendances entre des kernels appelés consécutivement
- Même si la main est rendue à l'hôte, la sémantique reste séquentielle
 - Les noyaux de calcul sont exécutés dans l'ordre
 - La gestion des dépendances est implicites

Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),
                cudaMemcpyHostToDevice);

kernel1<<<16, 64>>>(d_a, size);
kernel2<<<32, 32>>>(d_a, size);
kernel3<<<64, 16>>>(d_a, size);

cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),
                cudaMemcpyDeviceToHost);

cudaDeviceSynchronize();

kernel4<<<8, 128>>>(d_a, size);
```

Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),  
                cudaMemcpyHostToDevice);  
  
kernel1<<<16, 64>>>(d_a, size);  
kernel2<<<32, 32>>>(d_a, size);  
kernel3<<<64, 16>>>(d_a, size);  
  
cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),  
                cudaMemcpyDeviceToHost);  
  
cudaDeviceSynchronize();  
  
kernel4<<<8, 128>>>(d_a, size);
```

CPU

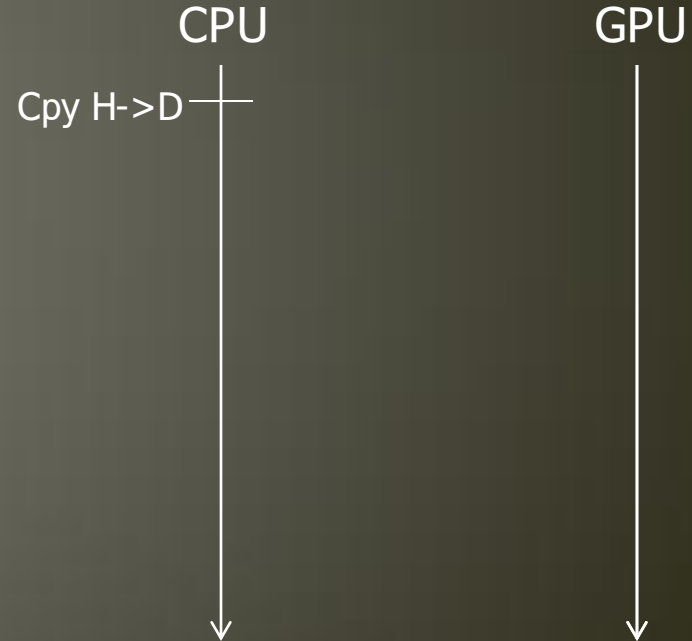


GPU



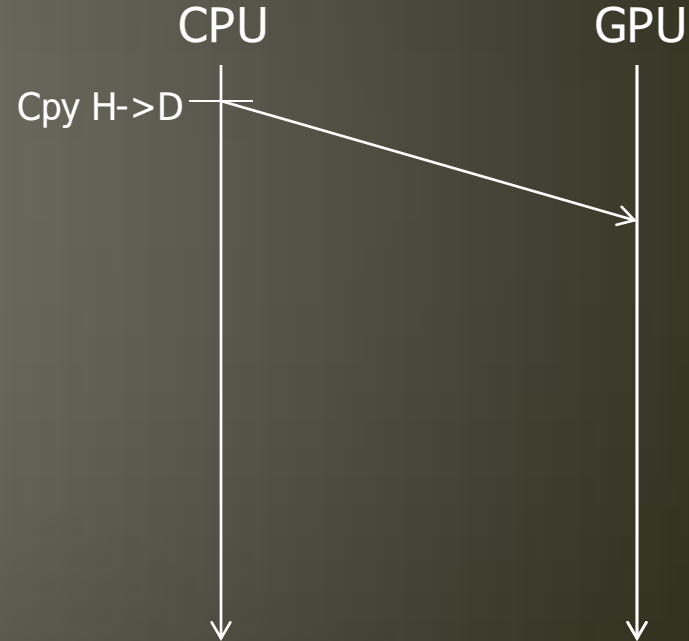
Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),  
                cudaMemcpyHostToDevice);  
  
kernel1<<<16, 64>>>(d_a, size);  
kernel2<<<32, 32>>>(d_a, size);  
kernel3<<<64, 16>>>(d_a, size);  
  
cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),  
                cudaMemcpyDeviceToHost);  
  
cudaDeviceSynchronize();  
  
kernel4<<<8, 128>>>(d_a, size);
```



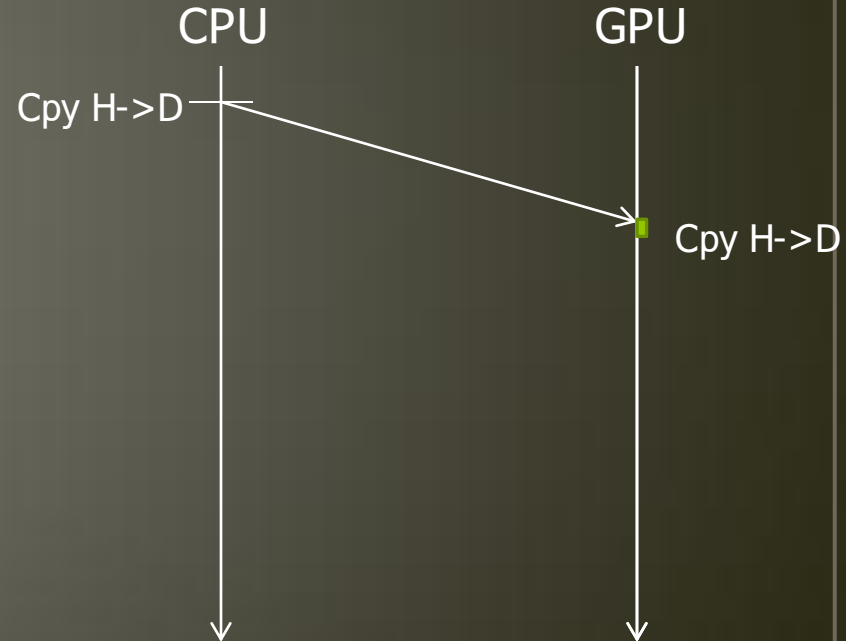
Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),  
                cudaMemcpyHostToDevice);  
  
kernel1<<<16, 64>>>(d_a, size);  
kernel2<<<32, 32>>>(d_a, size);  
kernel3<<<64, 16>>>(d_a, size);  
  
cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),  
                cudaMemcpyDeviceToHost);  
  
cudaDeviceSynchronize();  
  
kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),  
                cudaMemcpyHostToDevice);  
  
kernel1<<<16, 64>>>(d_a, size);  
kernel2<<<32, 32>>>(d_a, size);  
kernel3<<<64, 16>>>(d_a, size);  
  
cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),  
                cudaMemcpyDeviceToHost);  
  
cudaDeviceSynchronize();  
  
kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

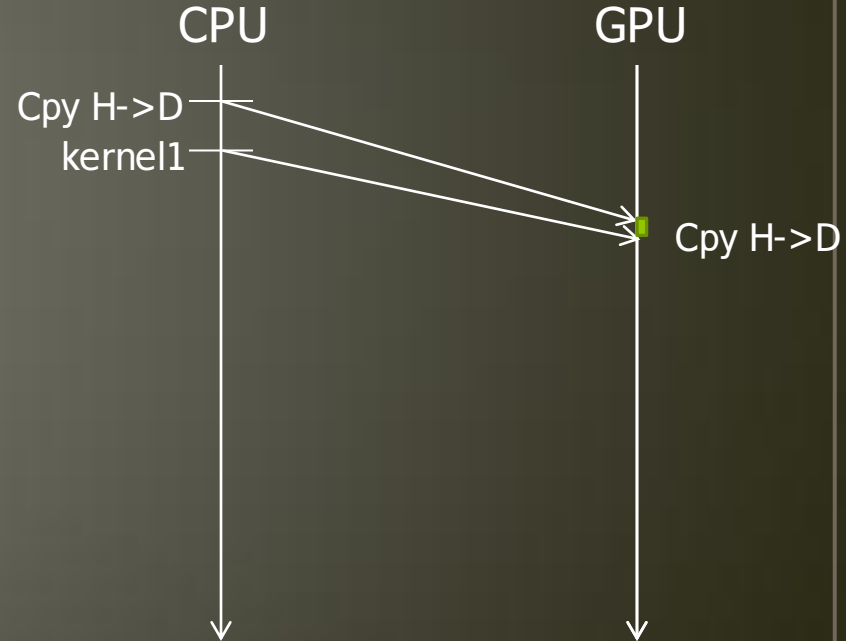
```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),
                cudaMemcpyHostToDevice);

kernel1<<<16, 64>>>(d_a, size);
kernel2<<<32, 32>>>(d_a, size);
kernel3<<<64, 16>>>(d_a, size);

cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),
                cudaMemcpyDeviceToHost);

cudaDeviceSynchronize();

kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

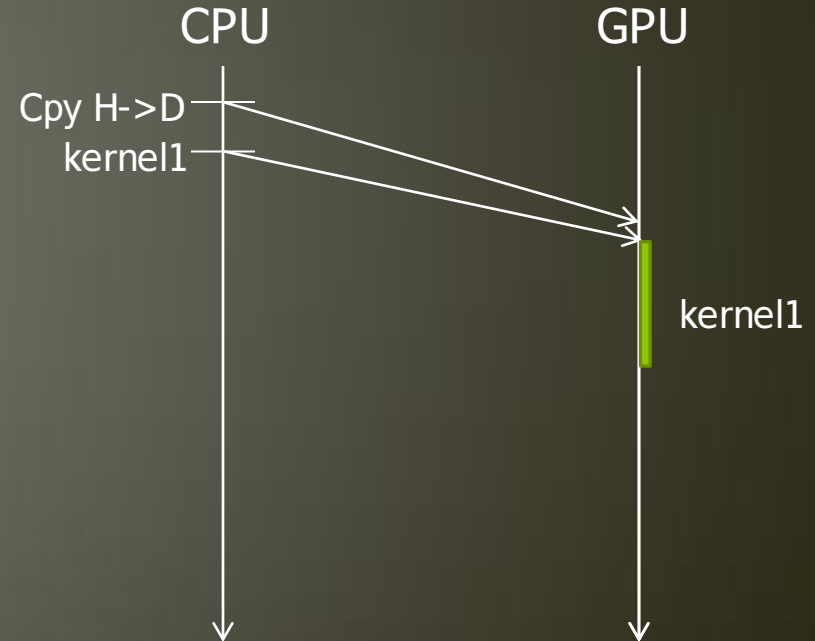
```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),
                cudaMemcpyHostToDevice);

kernel1<<<16, 64>>>(d_a, size);
kernel2<<<32, 32>>>(d_a, size);
kernel3<<<64, 16>>>(d_a, size);

cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),
                cudaMemcpyDeviceToHost);

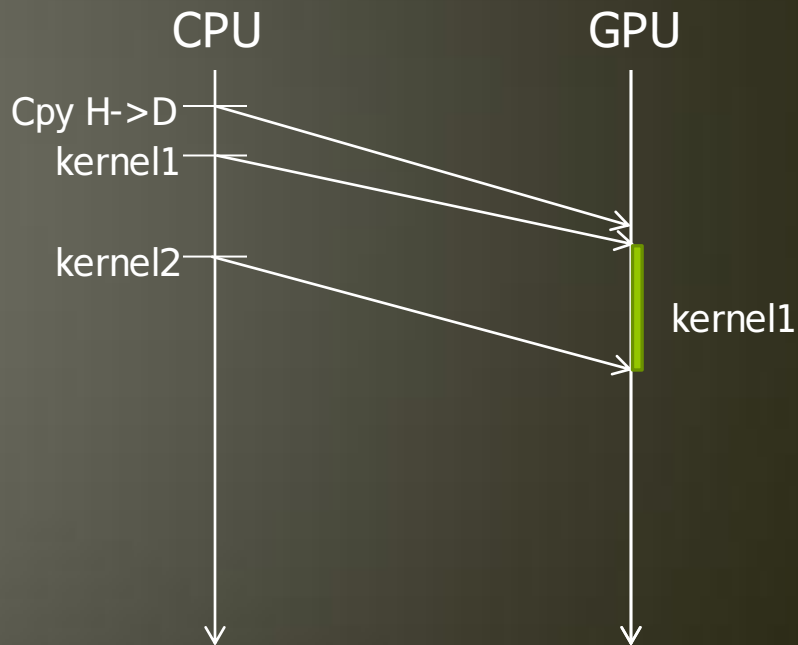
cudaDeviceSynchronize();

kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),  
                cudaMemcpyHostToDevice);  
  
kernel1<<<16, 64>>>(d_a, size);  
kernel2<<<32, 32>>>(d_a, size);  
kernel3<<<64, 16>>>(d_a, size);  
  
cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),  
                cudaMemcpyDeviceToHost);  
  
cudaDeviceSynchronize();  
  
kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

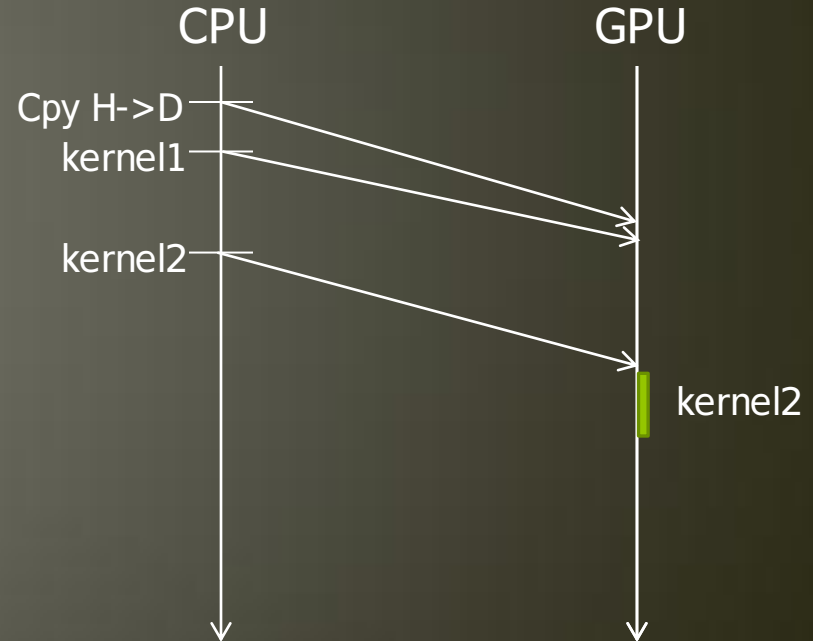
```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),
                cudaMemcpyHostToDevice);

kernel1<<<16, 64>>>(d_a, size);
kernel2<<<32, 32>>>(d_a, size);
kernel3<<<64, 16>>>(d_a, size);

cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),
                cudaMemcpyDeviceToHost);

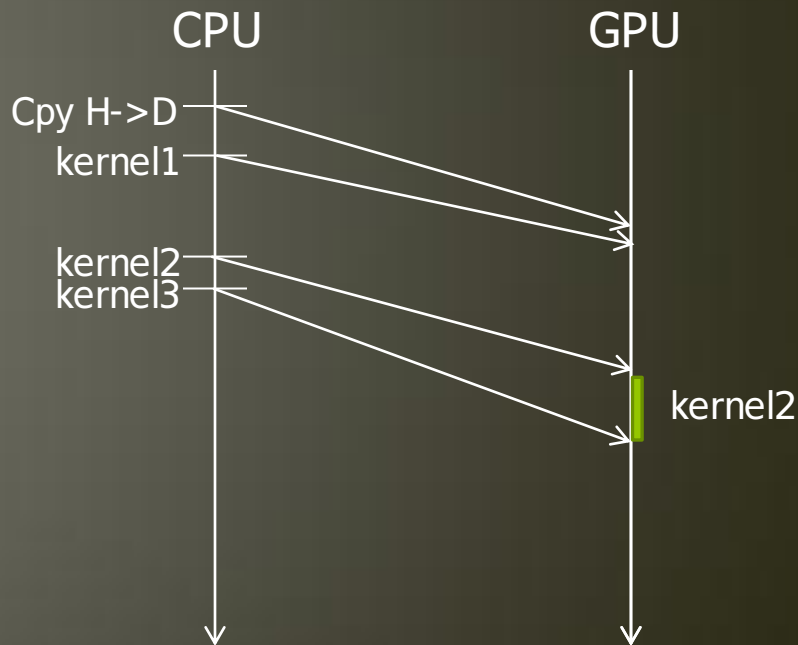
cudaDeviceSynchronize();

kernel4<<<8, 128>>>(d_a, size);
```



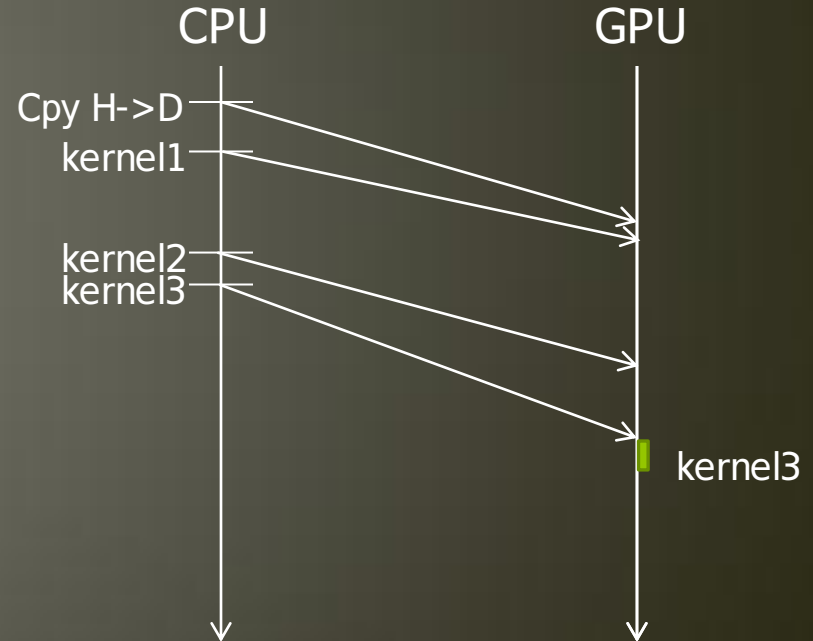
Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),  
                cudaMemcpyHostToDevice);  
  
kernel1<<<16, 64>>>(d_a, size);  
kernel2<<<32, 32>>>(d_a, size);  
kernel3<<<64, 16>>>(d_a, size);  
  
cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),  
                cudaMemcpyDeviceToHost);  
  
cudaDeviceSynchronize();  
  
kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),  
                cudaMemcpyHostToDevice);  
  
kernel1<<<16, 64>>>(d_a, size);  
kernel2<<<32, 32>>>(d_a, size);  
kernel3<<<64, 16>>>(d_a, size);  
  
cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),  
                cudaMemcpyDeviceToHost);  
  
cudaDeviceSynchronize();  
  
kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

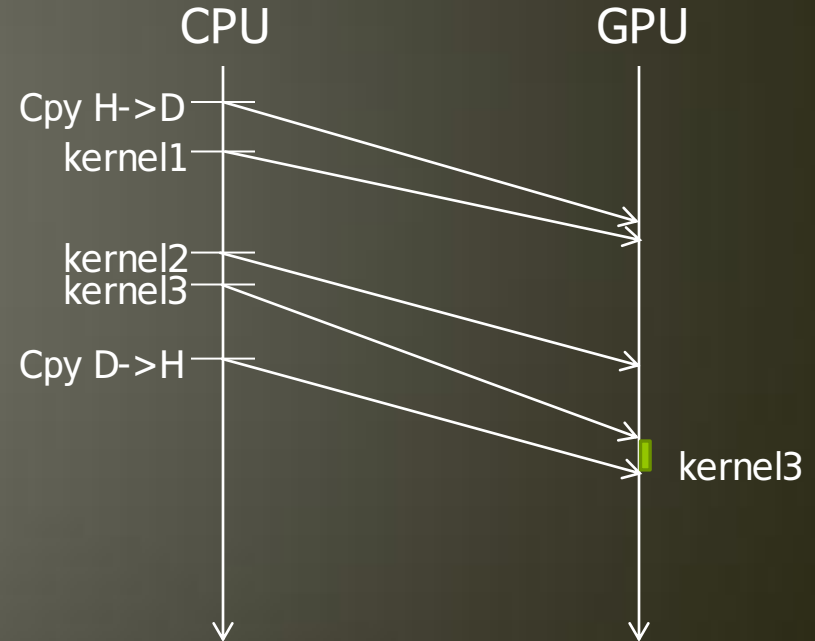
```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),
                cudaMemcpyHostToDevice);

kernel1<<<16, 64>>>(d_a, size);
kernel2<<<32, 32>>>(d_a, size);
kernel3<<<64, 16>>>(d_a, size);

cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),
                cudaMemcpyDeviceToHost);

cudaDeviceSynchronize();

kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

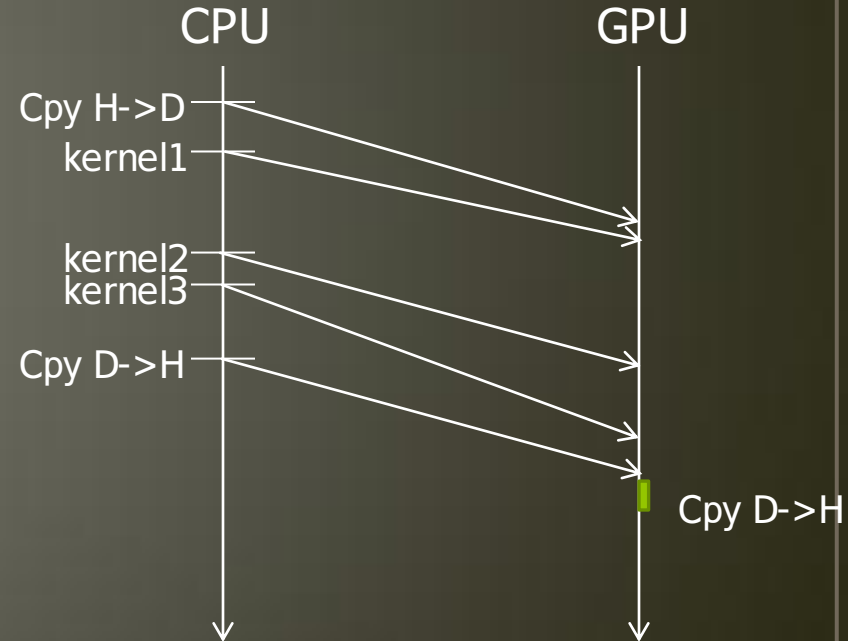
```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),
                cudaMemcpyHostToDevice);

kernel1<<<16, 64>>>(d_a, size);
kernel2<<<32, 32>>>(d_a, size);
kernel3<<<64, 16>>>(d_a, size);

cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),
                cudaMemcpyDeviceToHost);

cudaDeviceSynchronize();

kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

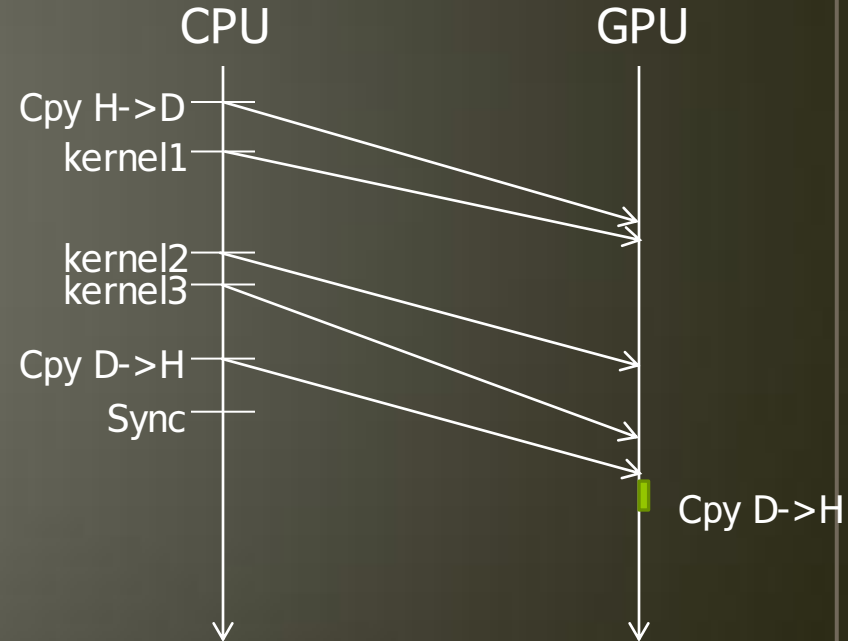
```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),
                cudaMemcpyHostToDevice);

kernel1<<<16, 64>>>(d_a, size);
kernel2<<<32, 32>>>(d_a, size);
kernel3<<<64, 16>>>(d_a, size);

cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),
                cudaMemcpyDeviceToHost);

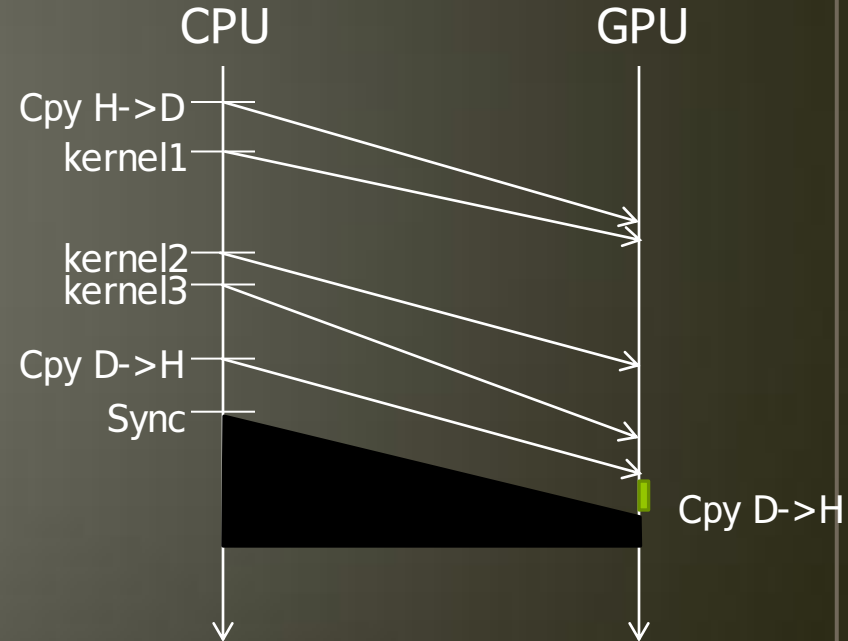
cudaDeviceSynchronize();

kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),  
                cudaMemcpyHostToDevice);  
  
kernel1<<<16, 64>>>(d_a, size);  
kernel2<<<32, 32>>>(d_a, size);  
kernel3<<<64, 16>>>(d_a, size);  
  
cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),  
                cudaMemcpyDeviceToHost);  
  
cudaDeviceSynchronize();  
  
kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme

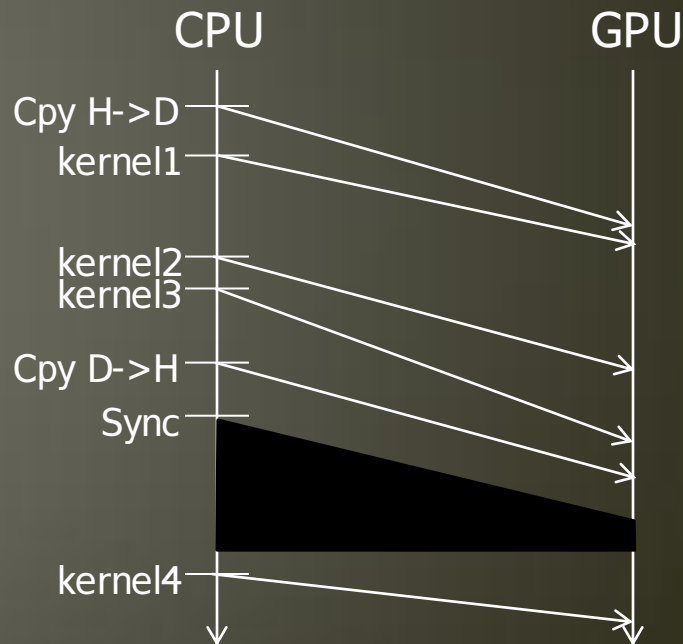
```
cudaMemcpyAsync(d_a, h_a, (sizeof(int) * (1024)),
                cudaMemcpyHostToDevice);

kernel1<<<16, 64>>>(d_a, size);
kernel2<<<32, 32>>>(d_a, size);
kernel3<<<64, 16>>>(d_a, size);

cudaMemcpyAsync(h_a, d_a, (sizeof(int) * (1024)),
                cudaMemcpyDeviceToHost);

cudaDeviceSynchronize();

kernel4<<<8, 128>>>(d_a, size);
```



Asynchronisme – mémoire host

- Problème: entre l'appel à `cudaMemcpyAsync` et la réalisation de la copie, l'adresse physique sur le host peut changer
 - En raison de la séparation espace d'adressage virtuelle / espace d'adressage physique
- Besoin d'une fonction pour fixer l'adresse physique de l'host pour les copies async.
 - `cudaMallocHost`

Asynchronisme

- Pas besoin de synchronisation globale en cas de dépendances entre des kernels appelés consécutivement
- Même si la main est rendue à l'hôte, la sémantique reste séquentielle
 - Les noyaux de calcul sont exécutés dans l'ordre
 - La gestion des dépendances est implicites
- Comment obtenir du *vrai* asynchronisme ?

Asynchronisme avancé

- Motivations :
 - Pouvoir transférer des données pendant l'exécution d'un kernel
 - Pouvoir exécuter plusieurs noyaux de calculs
 - Si la carte graphique le permet (Fermi)
 - S'il n'y a pas de dépendances entre les noyaux
- Solution : *streaming*

Asynchronisme avancé

- Déclaration d'un ou plusieurs *streams*
 - Chaque interaction avec le *device* se fait sur un stream en particulier
 - Le driver sait alors ce qui peut être parallélisé ou non

```
cudaMemcpy(d_c, c, N*sizeof(double),  
cudaMemcpyHostToDevice, stream[0]);  
my_kernel<<<Dg, Db, 0, stream[0]>>>  
                                (arg1, arg2, arg3);
```


Asynchronisme avancé

- Déclaration d'un ou plusieurs *streams*
 - Chaque interaction avec le *device* se fait sur un stream en particulier
 - Le driver sait alors ce qui peut être parallélisé ou non

```
cudaMemCpy(d_c, c, N*sizeof(double),  
cudaMemcpyHostToDevice, stream[0]);  
my_kernel<<<Dg, Db, 0, stream[1]>>>  
                                (arg1, arg2, arg3);
```

Asynchronisme avancé

- **Structure stream**

- `cudaStream_t`

- **Creation d'un stream**

- `cudaStream_t stream;`
 - `cudaStreamCreate (&stream) ;`

- **Destruction d'un stream**

- `cudaStreamDestroy (stream) ;`

- **Synchronization d'un stream**

- `cudaStreamSynchronize (stream) ;`
 - `cudaStreamWaitEvent (stream, event, flag)`

Synchronisation dans un bloc

- `void __syncthreads();`
 - Synchronisation entre tous les threads d'un même bloc
 - Permet également une synchronisation des données
 - Attention au flot de contrôle !
- **Pour les cartes compatibles 2.0**
 - `int __syncthreads_count(int predicate);`
 - `int __syncthreads_and(int predicate);`
 - `int __syncthreads_or(int predicate);`

Barrière mémoire

- Points de synchronisations au niveau des transactions mémoire
 - L'ordre des écritures peut ne pas être respecté du point de vue d'un autre thread
- But : permettre d'avoir une garantie que les accès mémoire lancés sont effectivement visibles par un ensemble de threads
- Différents niveaux de barrière
 - Bloc : `void __threadfence_block();`
 - Device : `void __threadfence();`
 - Device + hôte : `void __threadfence_system();`

Autres fonctionnalités

Mathématiques

- Ensemble de fonctions mathématiques optimisées pour GPU
 - Ex: `sin(float)`, `cos(double)`, ...
- Option de compilation pour utiliser les fonctions optimisées
 - `-use_fast_math`
 - Seulement pour les calculs simple précisions

Opérations atomiques

- Instructions assurant une atomicité
 - Ex: `int atomicAdd(int* address, int val);`
- Fonctions disponibles
 - Opérations: `atomicAdd`, `atomicSub`
 - Echange: `atomicExch`
 - Min/Max: `atomicMin`
 - Incrément/Décrément: `atomicInc`
 - CAS, ...
- Restrictions
 - Fonctionne sur:
 - les entiers, flottants,
 - 16-bits, 32-bits, 64-bits
 - Depends de la compute capability

Print

- **Sortie formatée**

- `int printf(const char *format[, arg, ...]);`
- Cartes supportant les capacités 2.0
- Fonction par thread
- Format final fait sur l'hôte

Timing et suivi du programme

- **Mesure de temps (profiling)**
 - Utilisation des événements définis par CUDA
- **Type principal : `cudaEvent_t`**
- **Création**
 - `cudaEventCreate(cudaEvent_t * e)`
- **Activation**
 - `cudaEventRecord(cudaEvent_t e, cudaStream_t s)`
- **Attente de l'activation des événements**
 - `cudaEventSynchronize(cudaEvent_t e);`
- **Calcul du temps passé**
 - `cudaEventElapsedTime(float * ms, cudaEvent_t start, cudaEvent_t stop);`

Gestion des erreurs

- En CUDA, (presque) toutes les fonctions retournent un code d'erreur
 - Retour d'un type `cudaError_t`
- Si tout se passe bien, il s'agit alors de `cudaSuccess`
- Sinon, il est possible d'obtenir une description
 - `const char * cudaGetErrorString (cudaError_t error);`
- Pour les fonctions ne retournant pas une telle info (par exemple, un appel à un *kernel*)
 - Appel à `cudaGetLastError()`
 - Retourne un type `cudaError_t`

Debugging

- Comment déboguer un code ?
 - Utilisation de *printf* directement possible mais pas infallible
 - Ajout de synchronisation
 - Vérifier le retour de chaque fonction CUDA
 - Récupérer les `cudaError` aussi pour les kernels

Optimisation

- **Priorité haute**

- Penser parallèle
- Minimiser les transferts hôte/device
- Nombre de blocs au moins égal au nombre de SM
 - Et nombre de threads au moins égal au nombre de cœurs
- Accès coalescés à la mémoire globale
- Utilisation de la mémoire shared
- Eviter de multiplier les chemins d'exécution dans le code

- **Priorité moyenne/basse**

- Eviter les conflits de banc de la mémoire shared
- Avoir un grand nombre de threads par blocs (multiple de 32)
- Utilisation des fonctions mathématiques optimisées

Architecture et programmation d'accélérateurs matériels

Cours 3 : APIs CUDA et Programmation Multi-GPUs

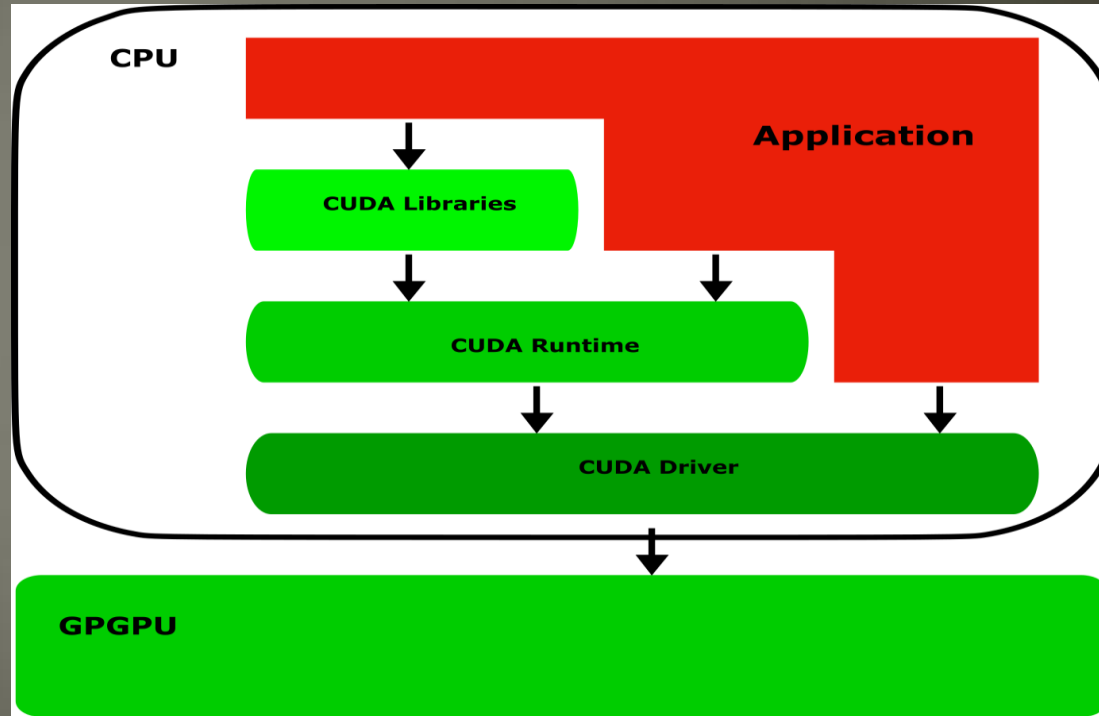
Julien Jaeger
julien.jaeger@cea.fr

Plan du cours

- Les différentes APIs pour CUDA
- Notion de contextes CUDA
- Programmation Multi-GPUs

Les différentes APIs pour CUDA

Les différents niveaux de programmation CUDA



Plan du cours

- Les différentes APIs pour CUDA
 - API Runtime
 - API Diver
- Notion de contextes CUDA
- Programmation Multi-GPUs

API Runtime (1)

- C'est l'API des utilisateurs finaux
 - Les exemples du cours et les fonctions utilisées en TDs jusqu'à présent font parties de l'API Runtime
- Fonctions préfixées par « cuda »
- Elle fournit les fonctions de base pour la programmation CUDA
 - cudaMalloc, cudaFree, cudaMemcpy, ...

API Runtime (2)

- API de haut niveau
- Niveau d'abstraction élevé
- Permet de ne pas exposer tous les mécanismes internes de CUDA aux utilisateurs

Plan du cours

- Les différentes APIs pour CUDA
 - API Runtime
 - API Driver
- Notion de contextes CUDA
- Programmation Multi-GPUs

API Driver (1)

- API de bas niveau
- C'est l'API des développeurs de bibliothèques
 - Besoin d'une expertise plus importante pour l'utiliser
 - Permet d'isoler les développements CUDA dans une bibliothèque du reste du programme.

API Driver (2)

- Fonctions préfixées par « cu »
- Elle fournit des fonctions équivalentes à celle de l'API Runtime
 - `cudaMalloc` → `cuMemAlloc`
 - `cudaMemcpy` → `cuMemcpyDtoH`
`cuMemcpyHtoD`
`cuMemcpyHtoH`
`cuMemcpyDtoD`

Contextes CUDA

Contextes (1)

- Structure interne de CUDA attaché à un GPU
 - Lorsqu'une opération demandant le concours d'un GPU est réalisée, le runtime CUDA « regarde » dans cette structure pour savoir quel GPU doit être utilisé
- Créé lorsqu'on demande à s'attacher à un nouveau GPU
 - Si un contexte n'existe pas déjà pour ce GPU

Contextes (2)

- Encapsule tous les objets relatifs au bon fonctionnement de CUDA comme:
 - Toutes les allocations mémoires avec son propre espace d'adressage.
 - Ainsi, seuls les threads partageant le même contexte partagent le même espace d'adressage
 - Les streams CUDA.
 - Les événements CUDA.
 - ...

Contextes (3)

- Deux types de contextes
 - Selon le niveau CUDA supporté
 - Primaire et « classique »
- Chacun accessible depuis une API différente
 - Primaire: API Runtime
 - « classique »: API Driver

Plan du cours

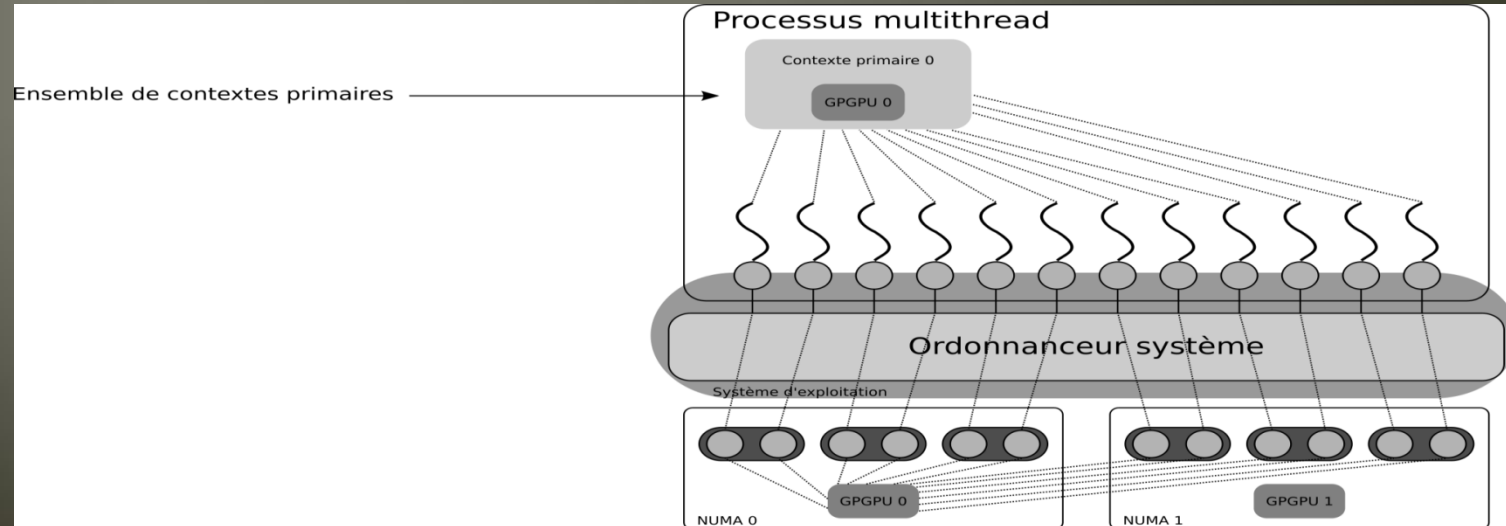
- Les différentes APIs pour CUDA
- Notion de contextes CUDA
 - Description des contextes
 - Contexte Primaire
 - Contexte « classique »
- Programmation Multi-GPUs

Contexte Primaire (1)

- Contexte le plus récent dans l'API CUDA
 - Disponible depuis CUDA 4.0
- Contexte commun pour tous les threads d'un processus ciblant le même GPU
 - Des données écrites par un thread dans la mémoire d'un GPU peuvent être lues par un thread associé au même GPU
- Création d'un nouveau contexte primaire à la première association d'un thread à un GPU
 - Si un contexte pour le GPU demandé n'est pas déjà disponible dans la pile des contextes primaire

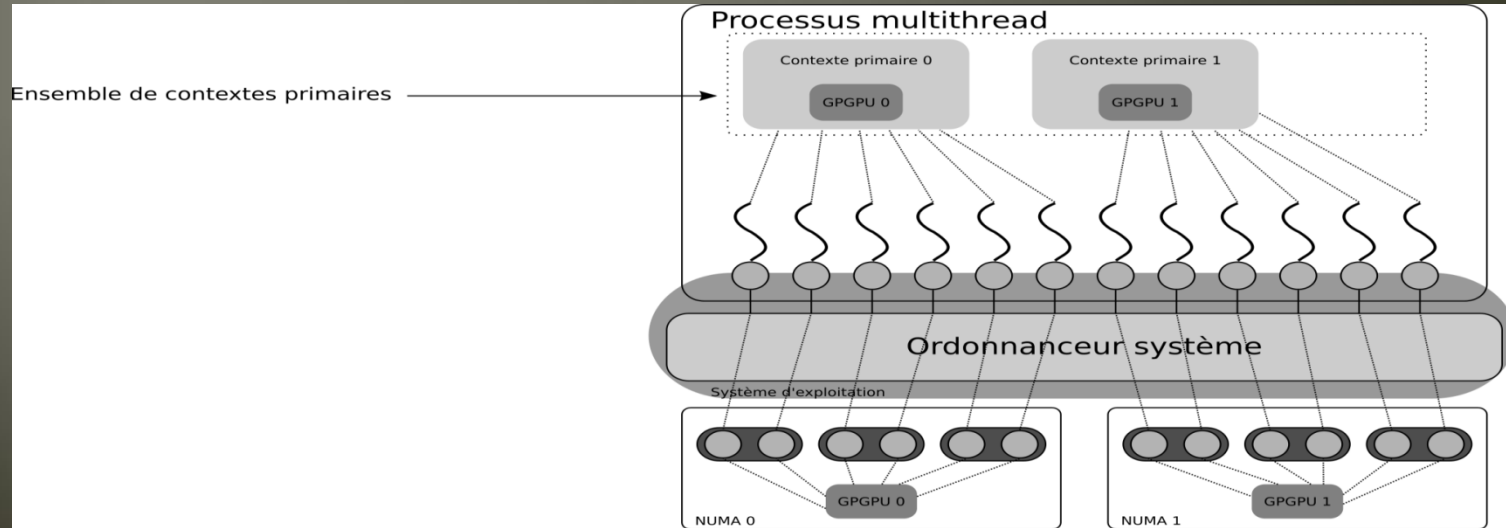
Contexte Primaire (2)

- Un contexte primaire par GPU par processus OS



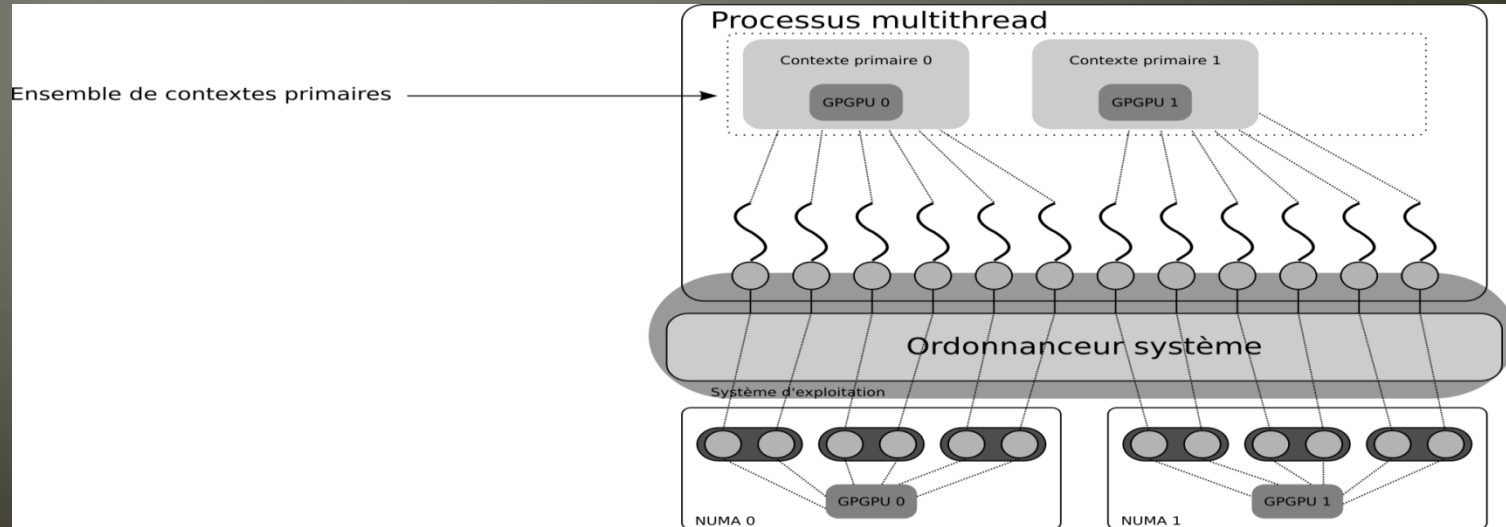
Contexte Primaire (3)

- Différents threads d'un même processus OS peuvent être associés à différents GPUs via différents contextes primaires



Contexte Primaire (4)

- Les threads partageant un même contexte primaire partagent aussi le même espace d'adressage
 - Une zone mémoire allouée ou initialisée par un thread peut être lue ou écrite par un autre thread associé au même contexte primaire



Plan du cours

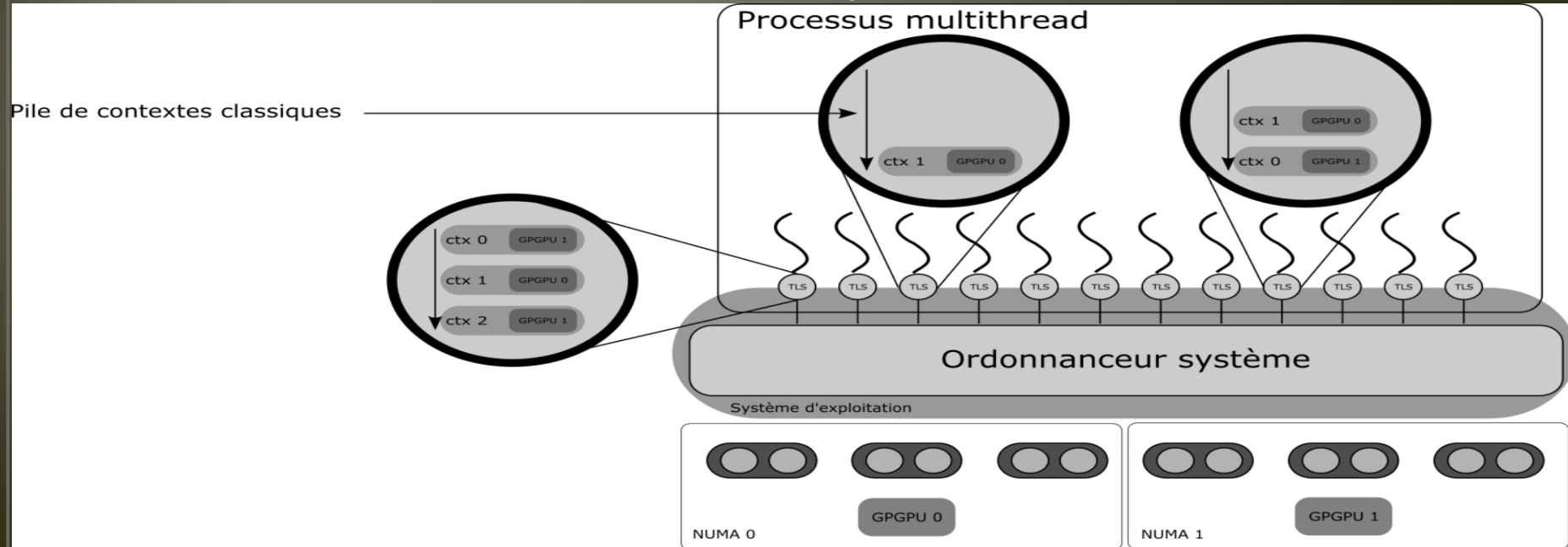
- Les différentes APIs pour CUDA
- Notion de contextes CUDA
 - Description des contextes
 - Contexte Primaire
 - Contexte classique
- Programmation Multi-GPUs

Contexte « classique » (1)

- Premier contexte de l'API CUDA
 - Contexte par défaut avant CUDA 4.0
- Contexte privé à chaque thread
- Création d'un nouveau contexte à l'appel de `cuCtxCreate(...)`
 - Peu importe le contenu des piles de contextes « classiques » ou primaires.
- Dans les versions récentes de CUDA, seules les contextes primaires sont disponibles

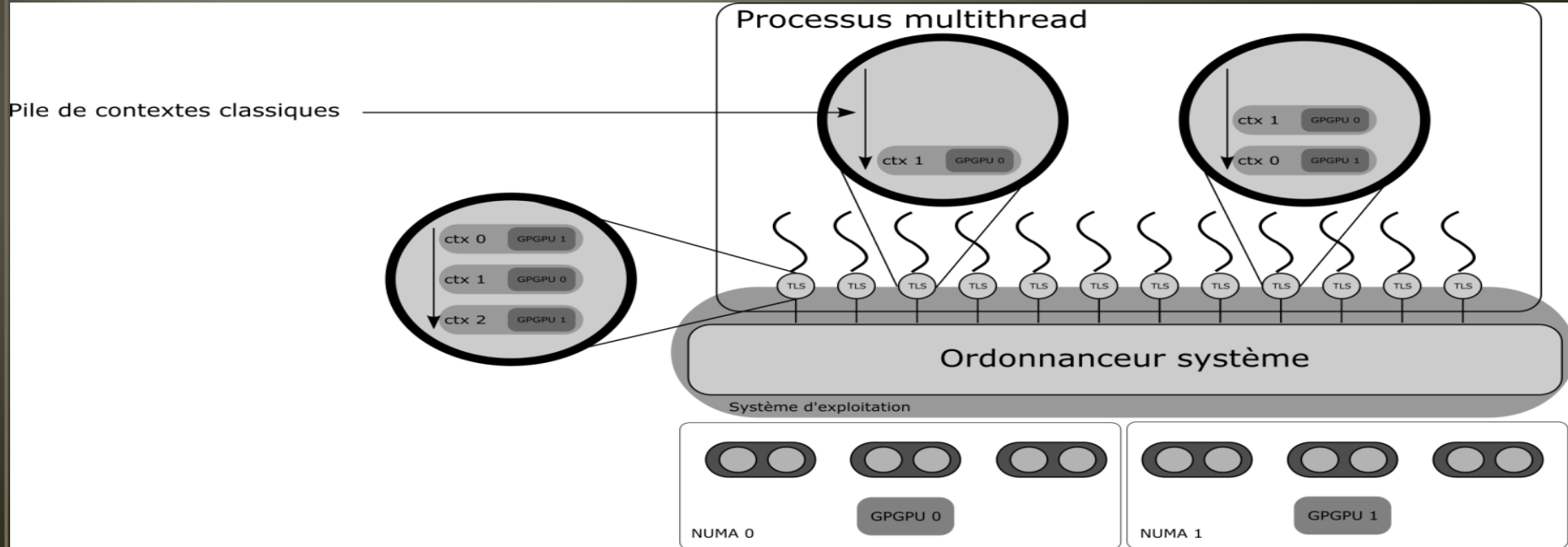
Contexte « classique » (2)

- Une pile de contextes par thread
 - Pile stockée dans la TLS du thread noyau



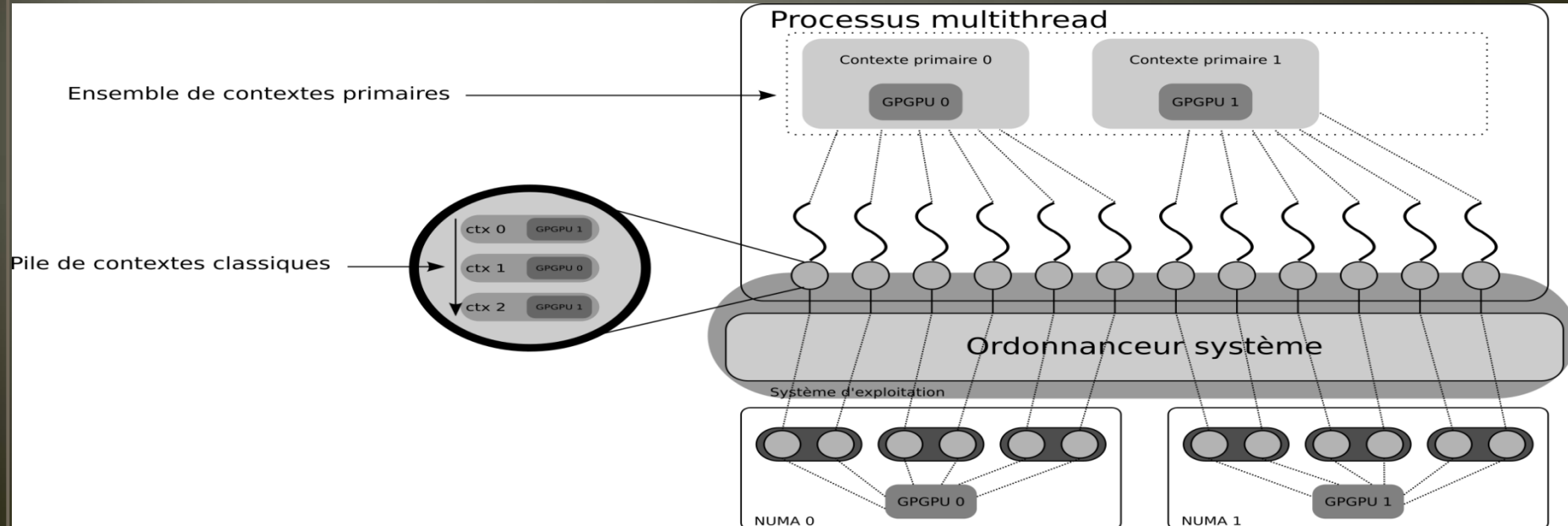
Contexte « classique » (3)

- Problèmes avec threads utilisateurs
 - Gestion des contextes à faire « à la main »



Contexte « classique » (4)

- Précédence sur les contextes primaires
 - Ctx « classique » choisit avant ctx primaire



Programmation Multi-GPUs

Plan du cours

- Les différentes APIs pour CUDA
- Notion de contextes CUDA
- **Programmation Multi-GPUs**
 - Sélection d'un GPU dans CUDA
 - UVA
 - MPI+CUDA
 - OpenMP + CUDA
 - NCCL

Sélection d'un GPU en CUDA

- Il existe deux fonctions pour choisir le GPU sur lequel les prochaines opérations hétérogènes vont être exécutées
 - API Runtime: `cudaSetDevice(...)`
 - API Driver: `cuCreateCtx(...)`

API Runtime: cudaSetDevice

- `__host__ cudaError_t cudaSetDevice (int device)`
 - Set device to be used for GPU executions.
- Parameters
 - device
 - Device on which the active host thread should execute the device code.

Utilisation de cudaSetDevice

```
Int main()
{
    ...
    cudaMalloc(&d_a, ...) // malloc sur le device par défaut
    (device 0)
    cudaSetDevice(1); // Sélectionne le device 1 pour les
    prochaines opérations CUDA
    cudaMalloc(&d_b, ...) // malloc sur le device 1
    ...
}
```

cudaSetDevice: détails (1)

- `cudaSetDevice` vérifie si un contexte est déjà associé au device demandé
 - Si il existe une pile de contexte « classique »
 - Sélectionne le contexte correspondant
 - Crée un nouveau contexte « classique » associé à ce device
 - Sinon, sélectionne le contexte primaire correspondant, ou en crée un nouveau

cudaSetDevice: détails (1)

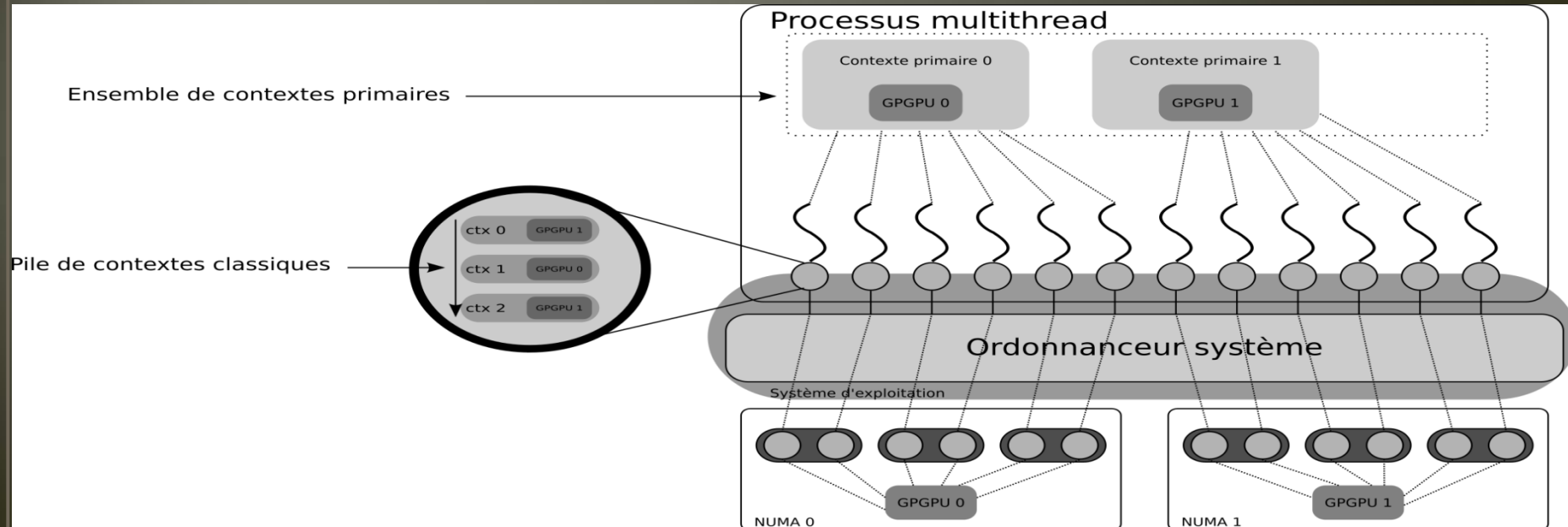
- `cudaSetDevice` vérifie si un contexte est déjà associé au device demandé
 - Si il existe une pile de contexte « classique »
 - Sélectionne le contexte correspondant
 - Crée un nouveau contexte « classique » associé à ce device
 - Sinon, sélectionne le contexte primaire correspondant, ou en crée un nouveau
 - Si oui, il s'associe au context primaire correspondant
 - Sinon, il crée un contexte primaire associé au GPU demandé puis s'y associe

cudaSetDevice: détails (2)

- Précédence sur les contextes primaires
 - Ctx « classique » choisit avant ctx primaire

Contexte « classique » (4)

- Précédence sur les contextes primaires
 - Ctx « classique » choisit avant ctx primaire



cudaSetDevice: détails (3)

- Si le device demandé n'existe pas (device 3 alors qu'il n'y a que 2 devices sur le nœud)
 - Pas d'erreur ni de segfault
 - Les prochains codes CUDA seront exécutés sur le device par défaut (device 0)

API Runtime: cudaGetDeviceCount

- `__host__ __device__ cudaError_t cudaGetDeviceCount`
`(int *count)`
 - Returns the number of compute-capable devices.
- Parameters
 - Count
 - Returns the number of devices with compute capability greater or equal to 2.0

Utilisation de cudaSetDevice

```
Int main()
{
    ...
    cudaMalloc(&d_a, ...);
    cudaGetDeviceCount(&nbGPUs); // Récupère le nombre de GPUs
disponibles
    cudaSetDevice(1%nbGPUs); // Modulo sur le nombre de GPUs
permet de choisir un « vrai » GPU
    cudaMalloc(&d_b, ...); //
    ...
}
```


API Driver: cuCtxCreate

- *CUresult* **cuCtxCreate** (*CUcontext* *pctx, unsigned int flags, *CUdevice* dev)
 - Create a CUDA context.
 - Ce nouveau contexte est « pushé » en tête de pile du thread appelant
- Parameters
 - Pctx: Returned context handle of the new context
 - Flags: Context creation flags
 - Dev: Device to create context on

API Driver: cuCtxPopCurrent

- *CUresult* **cuCtxPopCurrent** (*CUcontext* *pctx)
 - Pops the current CUDA context from the current CPU thread.
- Parameters
 - Pctx
 - Returned new context handle

API Driver: cuCtxPushCurrent

- *CUresult* **cuCtxPushCurrent** (*CUcontext ctx*)
 - Pushes a context on the current CPU thread.
- Parameters
 - Ctx
 - Context to push

Utilisation de cuCtxCreate

```
Int main()
{
    ...
    cudaMalloc(&d_a, ...) // malloc sur le device par défaut (device 0)
    cuCtxCreate(&myctx, flags, 1); // Sélectionne le device 1 pour les
    prochaines opérations CUDA
    cudaMalloc(&d_b, size) // malloc sur le device 1
    cuMemAlloc(&d_c, size) // malloc sur le device 1
    ...
}
```

Utilisation de cuCtxPopCurrent et cuCtxPushCurrent

```
Int main()
{
    cuCtxCreate(&myctx, flags, 1);
    cudaMalloc(&d_a, ...); // malloc sur le device 1
    cuCtxPopCurrent(&tmpctx); // Retire myctx du dessus de la pile des
contextes classiques
    cudaMalloc(&d_b, ...); // malloc sur le device précédent
    cuCtxPushCurrent(tmpctx); // On remet le contexte sur le dessus de la
pile
    cudaMalloc(&d_c, ...); // malloc sur le device 1
}
```

cuCtxCreate: détails (1)

- `cuCtxCreate` va créer un nouveau contexte pour le GPU demandé quel que soit le contenu de la pile
 - Il est possible d'avoir dans la même pile de contextes « classiques » plusieurs contexte attaché au même GPU
 - Lors d'un appel à `cudaSetDevice`, il va sélectionner le premier contexte attaché au GPU demandé trouvé lors du parcours de la pile

cuCtxCreate: détails (1)

- `cuCtxCreate` va créer un nouveau contexte pour le GPU demandé quel que soit le contenu de la pile
 - Il est possible d'avoir dans la même pile de contextes « classiques » plusieurs contexte attaché au même GPU
 - Lors d'un appel à `cudaSetDevice`, il va sélectionner le premier contexte attaché au GPU demandé trouvé lors du parcours de la pile

API Driver: Autres fonctions

- `CUresult cuCtxGetCurrent (CUcontext *pctx)`
 - Returns the CUDA context bound to the calling CPU thread.
 - Ce contexte n'est pas « poppé »

API Driver: Autres fonctions

- `CUresult cuCtxGetCurrent (CUcontext *pctx)`
 - Returns the CUDA context bound to the calling CPU thread.
- `CUresult cuCtxGetDevice (CUdevice *device)`
 - Returns the device ID for the current context.

API Driver: Autres fonctions

- `CUresult cuCtxGetCurrent (CUcontext *pctx)`
 - Returns the CUDA context bound to the calling CPU thread.
- `CUresult cuCtxGetDevice (CUdevice *device)`
 - Returns the device ID for the current context.
- `CUresult cuCtxSetCurrent (CUcontext ctx)`
 - Binds the specified CUDA context to the calling CPU thread.
 - En fait, effectue un remplacement (pop la tête de la pile, push ctx en tête de pile)

Gestion fine des contextes « classiques » (1)

- Pour éviter de créer plusieurs contextes « classique » attachés au même GPU:
 - Parcours « à la main » de la pile de contextes « classiques »
 - Avant de créer un nouveau contexte, parcours de toute la pile en « poppant » les contextes.
 - A chaque étape, un appel à `cuCtxGetDevice()` vous donnera l'ID du device auquel le contexte courant est attaché

Gestion fine des contextes « classiques » (2)

- Si c'est le GPU voulu, « poppez » le, « poussez » tous les contextes avec le GPU voulu en dernier
- Si vous ne trouvez pas le GPU voulu, alors faites votre appel à `cuCtxCreate`
- Ou laisser faire l'API Runtime
 - Créer un premier contexte « classique » pour votre thread
 - Ensuite chaque appel à `cudaSetDevice` cherchera dans la pile de contexte « classique » un contexte correspondant, ou bien créera un nouveau contexte « classique »

Plan du cours

- Les différentes APIs pour CUDA
- Notion de contextes CUDA
- **Programmation Multi-GPUs**
 - Sélection d'un GPU dans CUDA
 - UVA
 - MPI+CUDA
 - OpenMP + CUDA
 - NCCL

Utilisation de la même adresse sur des devices différents

```
Int main()
{
...
    cudaMalloc(&d_a, ...) //
    •
    cudaSetDevice(1); //
    •
    cudaMemcpy(&d_a, ...) //
    •
...
}
```

Utilisation de la même adresse sur des devices différents

```
Int main()  
{  
...  
    cudaMalloc(&d_a, ...) //  
    cudaSetDevice(1); //  
    .  
    cudaMemcpy(&d_a, ...) //  
    .  
    ...  
}
```

Que se pass-t-il dans ce cas ?

Utilisation de la même adresse sur des devices différents

```
Int main()
{
    ...
    cudaMalloc(&d_a, ...) // malloc sur le device par défaut
                           (device 0)
    cudaSetDevice(1); // Sélectionne le device 1 pour les
                     prochaines opérations CUDA
    cudaMemcpy(&d_a, ...) // Erreur! d_a est alloué sur le device
                           0, et ne veut rien dire pour le device 1
    ...
}
```


Utilisation de la même adresse sur des devices différents

```
Int main()
{
    ...
    cudaMalloc(&d_a, ...) // malloc sur le device par défaut
                           (device 0)
    cudaSetDevice(1); // Sélectionne le device 1 pour les
                     prochaines opérations CUDA
    cudaMemcpy(&d_a, ...) // Erreur! d_a est alloué sur le device
                           0, et ne veut rien dire pour le device 1
    ...
}
```

UVA: Unified Virtual Addressing

- UVA est arrivée avec CUDA 4.0
- UVA est un espace mémoire unifié virtuel
 - Les mémoires de l'hôte et des GPUs sont indépendants et ont des plages d'adresse mémoire différentes
 - UVA présente ces mémoires dans le meme espace d'adressage
- Le host ou un device peut interpréter correctement une adresse mémoire venant d'un autre device
 - Il peut savoir de quel device est issue cette adresse mémoire

Ce qui se passe avec UVA

```
Int main()
```

```
{
```

```
...
```

```
    cudaMalloc(&d_a, ...) //
```

•

```
    cudaSetDevice(1); //
```

•

```
    cudaMemcpy(&d_a, ...) //
```

•

•

```
...
```

```
}
```

Ce qui se passe avec UVA

```
Int main()
{
...
    cudaMalloc(&d_a, ...) // malloc sur le device par défaut (device 0)
    cudaSetDevice(1); //
    cudaMemcpy(&d_a, ...) //
    .
    .
...
}
```

Ce qui se passe avec UVA

```
Int main()
{
...
    cudaMalloc(&d_a, ...) // malloc sur le device par défaut (device 0)
    cudaSetDevice(1); // Sélectionne le device 1 pour les prochaines
opérations CUDA
    cudaMemcpy(&d_a, ...) //
.
...
}
```

Ce qui se passe avec UVA

```
Int main()
{
...
    cudaMalloc(&d_a, ...) // malloc sur le device par défaut (device 0)
    cudaSetDevice(1); // Sélectionne le device 1 pour les prochaines
opérations CUDA
    cudaMemcpy(&d_a, ...) // Le runtime reconnaît que d_a est une
adresse sur le GPU 0, et va donc copier les données sur ce GPU
...
}
```

UVA: Unified Virtual Addressing

- Il est tout a fait possible de copier des données vers, ou depuis, un autre GPU que celui auquel on est associé
 - Le runtime peut reconnaître les localisations des adresses
 - Plus besoin de spécifier le sens de la copie

UVA: Unified Virtual Addressing

- Il est tout a fait possible de copier des données vers, ou depuis, un autre GPU que celui auquel on est associé
 - Le runtime peut reconnaître les localisations des adresses
 - Plus besoin de spécifier le sens de la copie
 - `CudaMemcpyHostToDevice`
 - `CudaMemcpyDeviceToHost...`

UVA: Unified Virtual Addressing

- Il est tout a fait possible de copier des données vers, ou depuis, un autre GPU que celui auquel on est associé
 - Le runtime peut reconnaître les localisations des adresses
 - Plus besoin de spécifier le sens de la copie
 - `cudaMemcpyHostToDevice`
 - `cudaMemcpyDeviceToHost...`
 - `cudaMemcpyDefault`
 - Le runtime détermine seul le sens de la copie au fonction des adresses passées en argument de `cudaMemcpy`

Plan du cours

- Les différentes APIs pour CUDA
- Notion de contextes CUDA
- **Programmation Multi-GPUs**
 - Sélection d'un GPU dans CUDA
 - UVA
 - MPI+CUDA
 - OpenMP + CUDA
 - NCCL

MPI+CUDA (1)

```
Int main()
{
    MPI_Init(...)//
    ...
    cudaMalloc(...); // Tous les processus MPI utilisent le même
GPU par défaut
    ...
    MPI_Finalize(...)
}
```

MPI+CUDA (2)

```
Int main()
```

```
{
```

```
    MPI_Init(...)//
```

```
...
```

```
    cudaSetDevice (1); // Assigne le GPU 1 à chaque processus
```

```
    cudaMalloc(...); // Tous les processus MPI utilisent le même
```

GPU 1

```
...
```

```
    MPI_Finalize(...)
```

```
}
```

MPI+CUDA (3)

```
Int main()
{
    MPI_Init(...)//
    MPI_Comm_rank(MCW, &rank);
    cudaSetDevice(rank); // Assigne un GPU unique à chaque processus
    MPI. Pb si rank>nb GPU.
    cudaMalloc(...); //

    ...

    MPI_Finalize(...)
}
```

MPI+CUDA (4)

```
Int main()
{
    MPI_Init(...)//
    MPI_Comm_rank(MCW, &rank);
    cudaGetDeviceCount(&nbGPU);
    cudaSetDevice(rank % nbGPU); // Assigne un GPU à chaque
processus MPI en Round-Robin.
    cudaMalloc(...);
    ...
    MPI_Finalize(...)
}
```

Affectation des GPUs en Round-Robin (1)

- Maximise la répartition des processus MPI sur les différents GPUs disponibles
 - Ne maximise par forcément l'occupation des ces GPUs
 - Il faudrait pour cela affecter les GPUs aux processus MPI en fonction de la charge de travail présente... très compliqué à mettre en place
- Les GPUs affectés aux processus MPI ne sont pas forcément les plus proches
 - Problèmes d'effet NUMA

Affectation des GPUs en Round-Robin (2)

- Les processus utilisant le même GPU ne partagent pas le même espace d'adressage
 - Il faudrait partager le même contexte... très compliqué à mettre en place
 - Process-based: bcast du contexte du GPU choisi aux processus impliqués -> non prévu par CUDA (la taille et le contenu d'un contexte n'est pas accessible aux utilisateurs de CUDA)
 - Thread-based: utilisation de contextes primaires

Plan du cours

- Les différentes APIs pour CUDA
- Notion de contextes CUDA
- **Programmation Multi-GPUs**
 - Sélection d'un GPU dans CUDA
 - UVA
 - MPI+CUDA
 - OpenMP + CUDA
 - NCCL

OpenMP+CUDA (1)

```
Int main()
{
    #pragma omp parallel
    {
        rank = omp_get_thread_num();
        cudaGetDeviceCount(&nbGPU);
        cudaSetDevice (rank % nbGPU); // Chaque thread choisit son GPU. Le
        GPU courant n'est modifié que pour le thread courant. Mémoire partagée pour les
        threads sélectionnant le même GPU (utilisation de contexte primaire).
        cudaMalloc(...);
    }
}
```

OpenMP+CUDA (2)

```
Int main()
{
    #pragma omp parallel
    {
        rank = omp_get_thread_num();
        cudaGetDeviceCount(&nbGPU);
        cuCtxCreate (rank % nbGPU); // Créer et assigne, un nouveau contexte
        Indépendant pour chaque thread associé en Round-Robin à un GPU disponibles.
        Mémoire du GPU non partagée.
        cudaMalloc(...);
    }
}
```

OpenMP+CUDA (2)

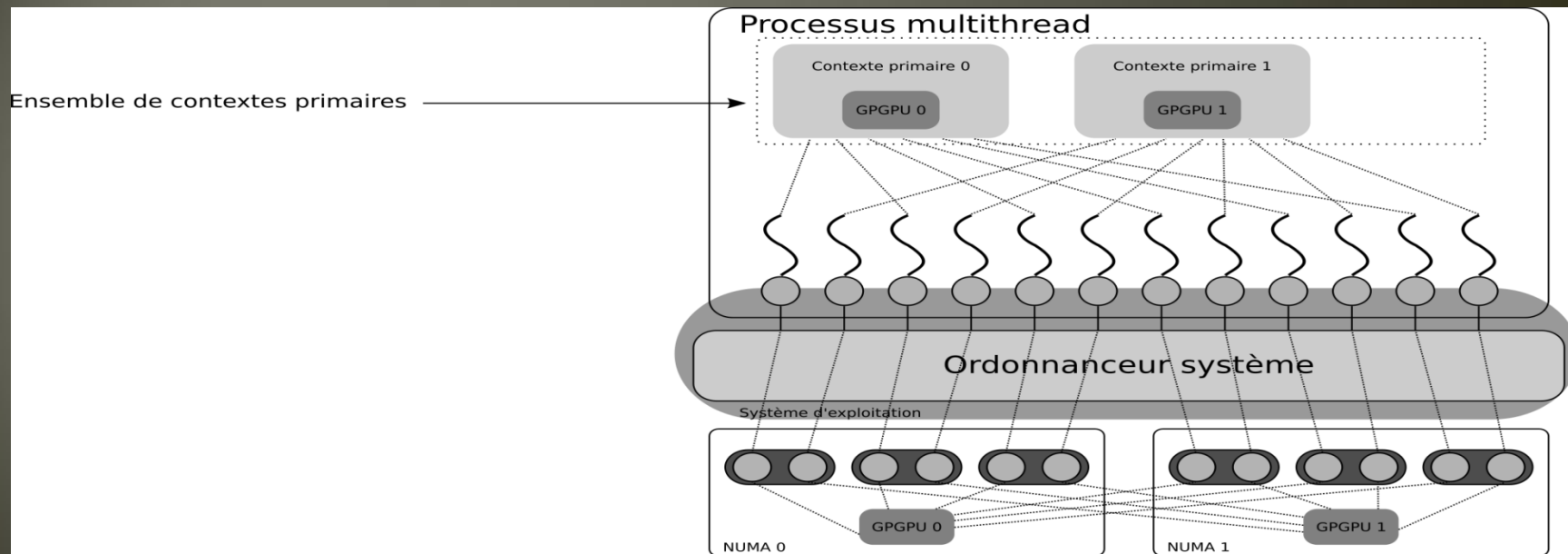
```
Int main()
{
    #pragma omp parallel
    {
        rank = omp_get_thread_num();
        cudaGetDeviceCount(&nbGPU);
        cuCtxCreate (&rank % nbGPU);
        cudaMalloc(&d_a,...);

        cudaGetDeviceCount(&nbGPU);
        cuCtxCreate (&rank % nbGPU);
        cudaMemcpy(&d_a,...); // Erreur! Même si c'est le même GPU, contexte différent du
        malloc, donc espace d'adressage différent!
    }
}
```

Affectation des GPUs en Round-Robin

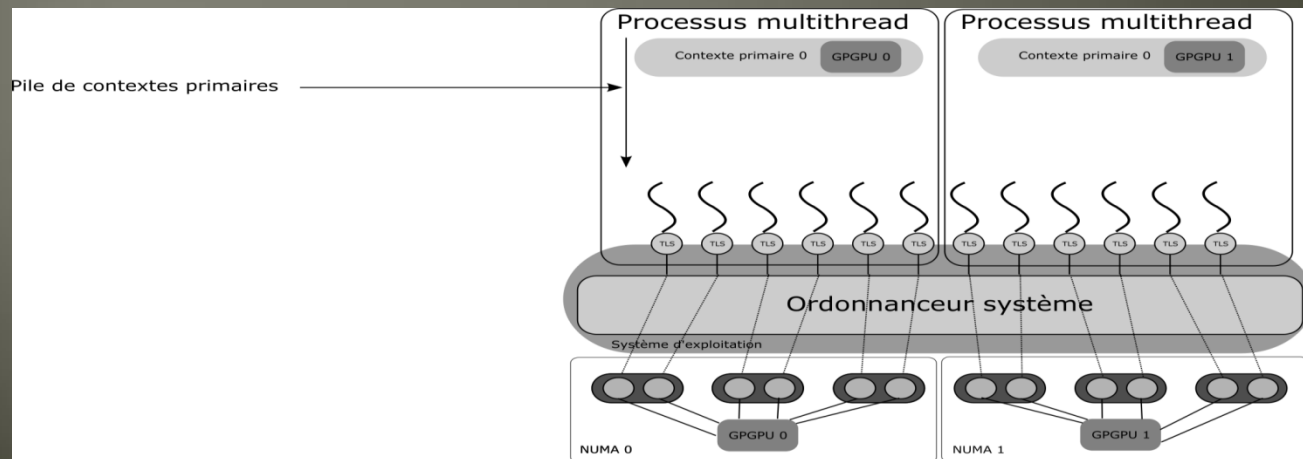
- Nous avons les même contraintes en OpenMP+CUDA (et en threads+CUDA en général) qu'en MPI+CUDA
 - Ne maximise par forcément l'occupation des GPUs
 - Les GPUs affectés aux threads ne sont pas forcément les plus proches
 - Les threads utilisant le même GPU ne partage pas forcément le même espace d'adressage
 - Cela dépend de l'utilisation de contexte classiques ou primaires

Affectation des GPUs en Round-Robin



MPI+OpenMP+CUDA (1)

- Meilleur cas ... presque (pour les GPUs)
 - 1 processus MPI/GPU disponible



MPI+OpenMP+CUDA (2)

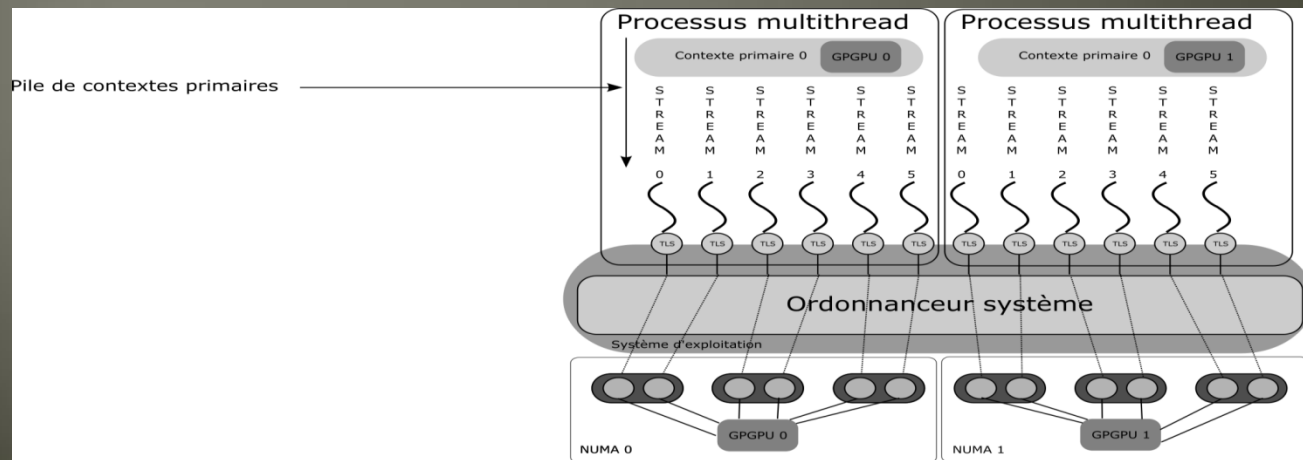
- Utilise tous les GPUs disponibles
- Tous les threads d'un même processus MPI partage le même espace d'adressage sur GPU
 - Possible de faire alloc+memcpy avant une région parallèle, puis chaque thread lance un kernel sur ses données
- Possible avec l'API Runtime
 - Pas besoin des fonctions compliquées de l'API Driver
- Fonctionne avec user-level threads

MPI+OpenMP+CUDA (3)

- `/!\` Chaque appel CUDA dans un même processus MPI sera sérialisé avec les autres appels des autres threads
 - Utilisation des streams: associer un stream unique à chaque threads
 - Les streams sont indépendants: pas de sérialisation, et il est alors possible pour deux threads utilisant seulement la moitié du GPU de l'utiliser au même moment
 - Attention: nombre de streams limités (16)

MPI+OpenMP+CUDA (4)

- Meilleur cas (pour les GPUs)
 - 1 processus MPI/GPU disponible + streams



MPI+OpenMP+CUDA (5)

```
Int main()
{
    MPI_Init(...)//
    MPI_Comm_rank(MCW, &mpirank);
    cudaGetDeviceCount(&nbGPU);
    cudaSetDevice (mpirank % nbGPU); // modulo nécessaire pour associer chaque rang au bon GPU
en multi-noeud
    #pragma omp parallel
    {
        omprank = omp_get_thread_num();
        cudaMalloc(..., omprank % 16); // appel à cudaMalloc sur le stream n° (omprank
%16).
    }
    MPI_Finalize(...)
}
```

MPI+OpenMP+CUDA (6)

- Pour de meilleures performances, il faut tenir compte de la position des GPUs pour placer les processus MPI et les threads générés de façon à éviter les effets NUMA
- Parfois, il n'est pas possible d'avoir une hiérarchie sans effets NUMA
 - Exemple: 2 sockets, avec les 2 GPUs attachés à la même socket

MPI+OpenMP+CUDA (7)

- /!\ La meilleure répartition processus MPI/threads OpenMP pour l'utilisation des GPUs n'est pas forcément celle apportant les meilleures performances sur CPUs
- /!\ Des bibliothèques externes peuvent manipuler les GPUs/les contextes CUDA et « casser » votre répartition optimale

Plan du cours

- Les différentes APIs pour CUDA
- Notion de contextes CUDA
- **Programmation Multi-GPUs**
 - Sélection d'un GPU dans CUDA
 - UVA
 - MPI+CUDA
 - OpenMP + CUDA
 - NCCL

NCCL (1)

- NCCL (« Nickel »): Nvidia Collective Communications Library
- But: permettre des échanges point-à-points et collectifs de données entre plusieurs GPUs présents sur un même nœud.
 - Possible de le faire entre GPUs sur nœuds distincts si ceux-ci sont reliés par les cartes réseaux Nvidia Mellanow Infiniband

NCCL (2)

- Très proche de la façon de faire de MPI
- Une « clique » regroupe les GPUs mis en jeu dans une collective (pas forcément tous)
- Un communicateur est initialisé pour chaque GPU dans la « clique » voulue
 - Soit un appel par GPU
 - Cette initialisation fait intervenir une barrière: il est nécessaire de faire cette initialisation en parallèle
 - Processus MPI différents ou threads différents
 - Soit un appel global

Fonctions d'initialisation

- *ncclResult_t* **ncclCommInitRank** (*ncclComm_t** comm, int nGPUs, *ncclUniqueId* cliqueld, int rank);
 - Initialise le rang « rank »
- **Parameters**
 - Comm: communicateur CUDA
 - nGPUs: nombre de GPU dans la clique
 - cliqueld: ID unique pour la clique
 - Un rang fait appel à *ncclGetUniqueId()* puis broadcast (MPI_Bcast, ...)
 - Rank: ID unique pour le GPU courant

Fonctions d'initialisation

- `ncclResult_t ncclCommInitAll (ncclComm_t* comms, int nGPUs, int* devList);`
 - Initialise directement les nGPUs GPUs
- **Parameters**
 - Comms: tableau de comm, un pour chaque GPU
 - nGPUs: nombre de GPU dans le communicateur
 - devList: quel CUDA device est associé à quel rang

Utilisation de ncclCommInitRank (MPI)

```
Int main()
{
    ...

    MPI_Init();

    MPI_Comm_rank(MCW, &rank);
    ncclCommInitRank(&gpucomm, nGPUs, cUID, getGPU(rank)); // Seuls les
rangs choisis initialisent leur communicateur GPU.
    MPI_Finalize();

    ...
}
```

Utilisation de ncclCommInitRank (OpenMP)

```
Int main()
{
    ...
        #pragma omp parallel
        {
            rank = omp_get_thread_num();
            ncclCommInitRank(&gpucomm, nGPUs, cUID, getGPU(rank)); // Seuls les
rangs choisis initialisent leur communicateur GPU.
        }
    ...
}
```

Utilisation de `ncclCommInitRank`

- `/!\` Si vous plus d'un processus MPI/thread affecté à un GPU, il faut faire bien attention de n'appeler qu'une seul fois `ncclCommInitRank` pour ce GPU.
- L'argument passé à `ncclCommInitRank` est l'identifiant du GPU, et non le rang du processus MPI/thread.

Utilisation de ncclCommInitAll (MPI)

```
int clique = {0,3,1,5}
```

```
Int main()
```

```
{
```

```
    ncclComm_t gpucomm [4]; // Autant de communicateurs nccl que de GPUs dans la  
clique
```

```
    MPI_Init();
```

```
    MPI_Comm_rank(MCW, &rank);
```

```
    if(rank == constante)
```

```
    {
```

```
        ncclCommInitAll(&gpucomm, nGPUs, clique); // on n'appelle cette fonction qu'une  
seul fois
```

```
    }
```

```
    MPI_Finalize();
```

```
}
```

Utilisation de ncclCommInitAll (OpenMP)

```
int clique = {0,3,1,5}
```

```
Int main()
{
    ncclComm_t gpucomm [4]; // Autant de communicateurs nccl que de GPUs dans la
    clique
    ncclCommInitAll(&gpucomm, nGPUs, clique); // On n'appelle cette fonction
    qu'une seul fois
    #pragma omp parallel
    {
        ...
    }
}
```

NCCL (3)

- Les fonctions de communications collectives ont des signatures quasiment équivalentes à MPI
- Chaque GPU doit faire appel à la même fonction
- Ce sont des appels asynchrones
 - Il est possible que le même processus/threads fassent tous les appels
 - Sélection du GPU + appel de fonction

Fonctions collective: allreduce

- `ncclResult_t ncclAllReduce (`
 `void* sendoff,`
 `void* recvbuff,`
 `int count,`
 `ncclDataType_t type,`
 `ncclRedOp_t op,`
 `ncclComm_t comm,`
 `cudaStream_t stream);`

Fonctions collective: allreduce

NCCL

```
■ ncclResult_t ncclAllReduce (  
    void* sendoff,  
    void* recvbuf,  
    int count,  
    ncclDataType_t type,  
    ncclRedOp_t op,  
    ncclComm_t comm,  
    cudaStream_t stream);
```

MPI

```
■ int MPI_Allreduce (  
    void *sendbuf,  
    void *recvbuf,  
    int count,  
    MPI_Datatype datatype,  
    MPI_Op op,  
    MPI_Comm comm);
```

Utilisation de fonction de communications coll. (MPI 1)

```
int clique = {0,3,1,5}
```

```
Int main()
```

```
{
```

```
    MPI_Init();
```

```
    MPI_Comm_rank(MCW, &rank);
```

```
    ... // Initialisation des communicateurs nccl
```

```
    if(inClique (getGPU(rank)) // Soit les rangs concernés font l'appel
```

```
    {
```

```
        ncclAllReduce(&sendbuf, &recvbuf, count, type, op, gpucomm);
```

```
    }
```

```
    MPI_Finalize();
```

```
}
```

Utilisation de fonction de communications coll. (MPI 1)

```
int clique = {0,3,1,5}
```

```
Int main()
```

```
{
```

```
    MPI_Init();
```

```
    MPI_Comm_rank(MCW, &rank);
```

```
    ... // Initialisation des communicateurs nccl
```

```
    if(inClique (getGPU(rank)) // Soit les rangs concernés font l'appel
```

```
    {
```

```
        ncclAllReduce(&sendbuf, &recvbuf, count, type, op,
```

```
gpucomm[getGPU(rank)]);
```

```
    }
```

```
    MPI_Finalize();
```

```
}
```

Utilisation de fonction de communications coll. (MPI 2)

```
int clique = {0,3,1,5}
Int main()
{
    MPI_Init();
    MPI_Comm_rank(MCW, &rank);
    ... // Initialisation des communicateurs nccl
    if(rank==0) // Soit un seul rang réalise les appels de tous les GPUs concernés
    {
        for(i=0; i<nbGPUs; i++)
        {
            if(inClique (i))
            {
                ncclAllReduce(&sendbuf, &recvbuf, count, type, op, gpucomm);
            }
        }
    }
    MPI_Finalize();
}
```

Utilisation de fonction de communications coll. (MPI 3)

// Si tous les GPUs sont concernés, il n'y a plus de sélection/vérification à faire: code plus simple

```
Int main()
{
    MPI_Init();
    MPI_Comm_rank(MCW, &rank);
    ... // Initialisation des communicateurs nccl
    if(rank==0)
    {
        for(i=0; i<nbGPUs; i++)
        {
            ncclAllReduce(&sendbuf, &recvbuf, count, type, op, gpucomm);
        }
    }
    MPI_Finalize();
}
```

Utilisation de ncclCommInitAll (OpenMP 1)

```
int clique = {0,3,1,5}
int main()
{
    ... // Initialisation des communicateurs nccl
    #pragma omp parallel
    {
        rank = omp_get_thread_num();
        if(inClique (getGPU(rank)) // Soit les rangs concernés font l'appel
        {
            ncclAllReduce(&sendbuf, &recvbuf, count, type, op,
                           gpucomm);
        }
    }
}
```

Utilisation de ncclCommInitAll (OpenMP 2)

```
int clique = {0,3,1,5}
int main()
{
    ... // Initialisation des communicateurs nccl
    for(i=0; i<nGPUs, i++)
    {
        if(inClique (i) // Soit les rangs concernés font l'appel
        {
            ncclAllReduce(&sendbuf, &recvbuf, count, type, op, gpucomm);
        }
    }
    #pragma omp parallel
    {
    }
}
```


Etat actuel de NCCL 2.19.3

- Collectives

- Broadcast
- All-Gather
- Reduce
- All-Reduce
- Reduce-Scatter

- Point-to-point

- Send / recv
- One-to-all (scatter)
- All-to-one (gather)
- All-to-all
- Neighbor exchange

- Key Features

- Single-node and multi-nodes
- Host-side API
- Asynchronous/non-blocking interface
- Multi-thread, multi-process support
- In-place and out-of-place operation
- Automatic Topology Detection
- NVLink & PCIe/QPI*support
- InfiniBand verbs, libfabric, RoCE and IP Socket internode communication

Architecture et programmation d'accélérateurs matériels

Cours 4 : Modèles avancés de programmation
accélérateurs

Julien Jaeger
julien.jaeger@cea.fr

Plan du cours

- Programmation OpenCL
- Programmation par directives
 - OpenMP
 - OpenACC
- Task-scheduling
 - StarPU

Programmation OpenCL

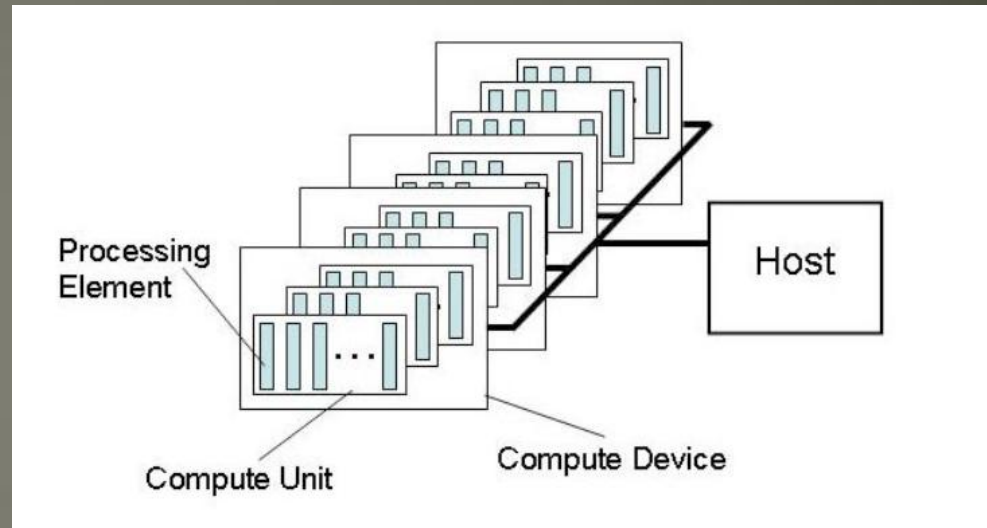
Plan – Programmation OpenCL

- Introduction
 - Vision globale
 - Modèle d'exécution
- Abstraction
- Comparaison avec CUDA

Introduction OpenCL

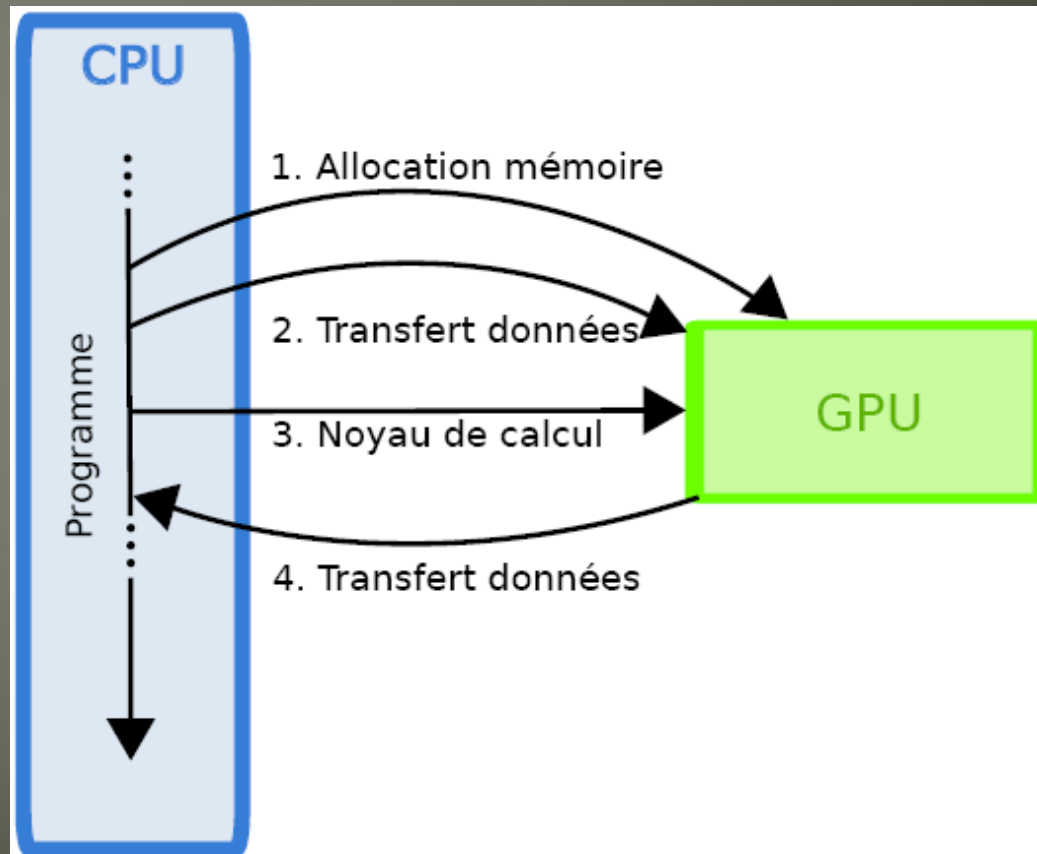
- Open Compute Language
 - <http://www.khronos.org/opensl>
 - Version courante 2.2-10
- **Modèle de programmation portable**
 - Utilisation des processeurs généralistes (CPUs)
 - Utilisation des cartes graphiques (GPUs)
 - Utilisation des accélérateurs (Cell)

Vision globale

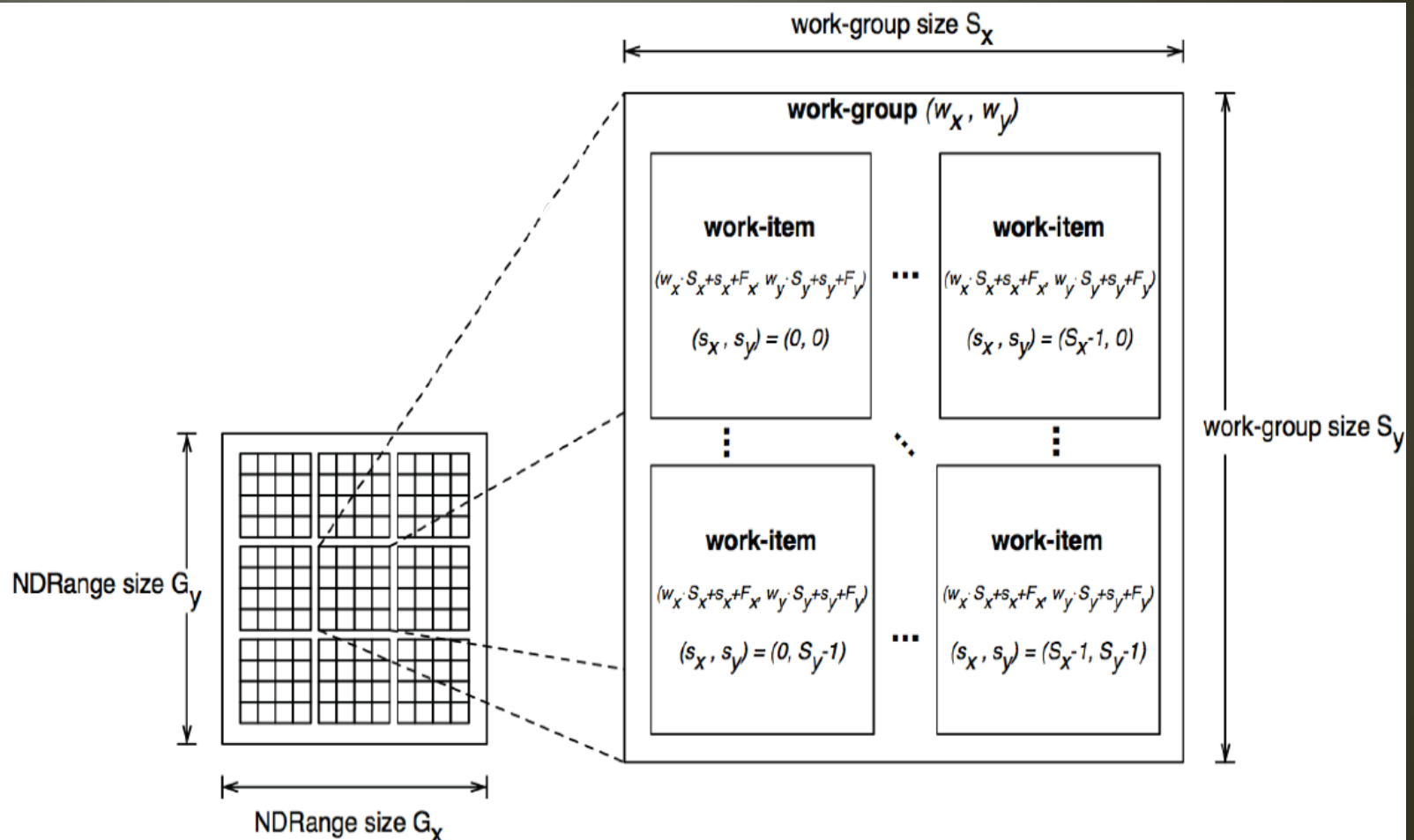


- Un hôte contrôle les accélérateurs
- Vision hiérarchique :
 - *Compute device*
 - *Compute unit*
 - *Processing element*

Principe d'exécution



Modèle d'exécution



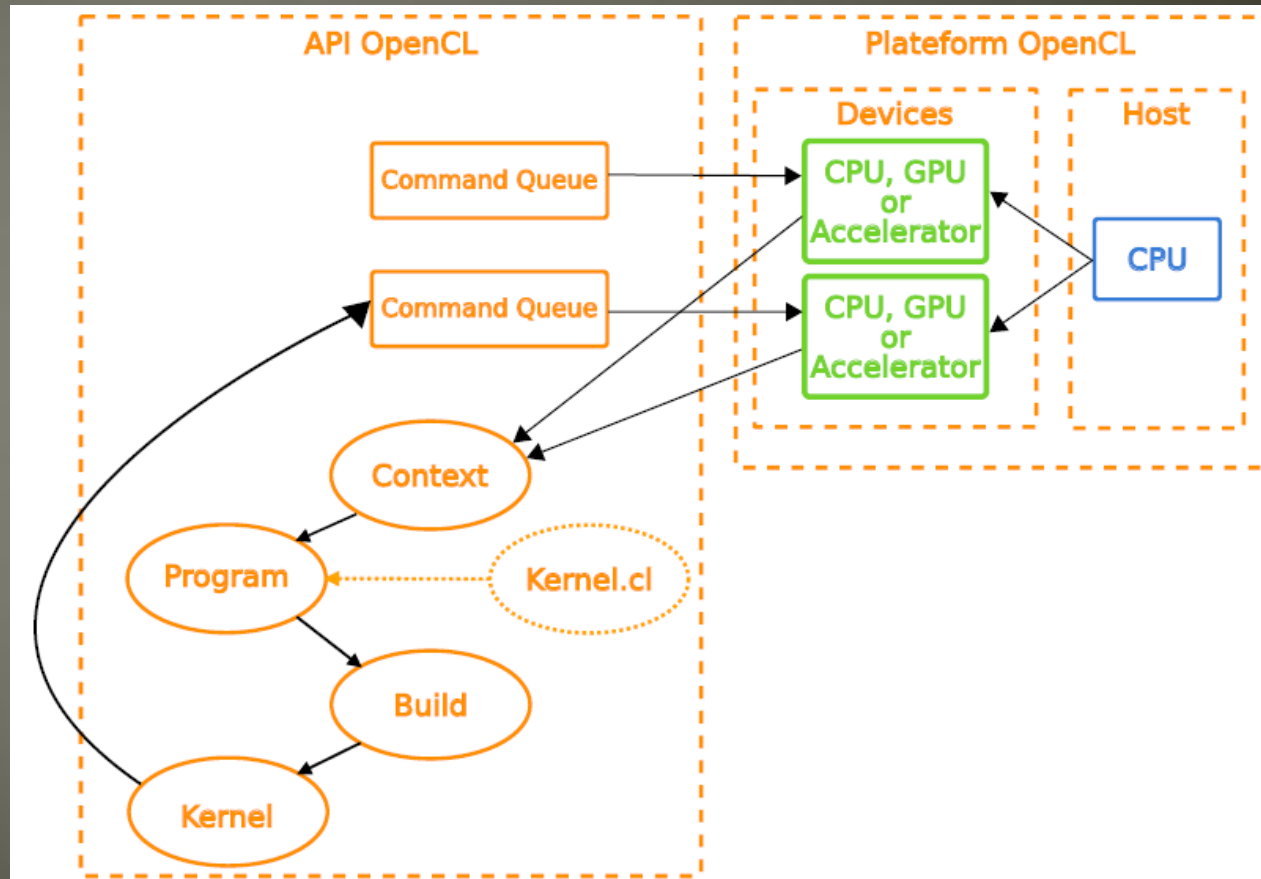
Plan – Programmation OpenCL

- Introduction
- **Abstraction**
 - Vision logique
 - File d'exécution
- Comparaison avec CUDA

Abstraction

- Vision de plusieurs devices de types différents
 - Les processeurs multicoeurs sont vus comme un device
- Le programme est compilé à la volée
 - Création d'un context, programme, arguments, ...
- Actions (exécution d'un noyau, transferts)
 - Command queue
 - Notion de dépendances (exécution dans le désordre)
 - Notion d'événements

Abstraction d'OpenCL



Plan – Programmation OpenCL

- Introduction
- Abstraction
- Comparaison avec CUDA
 - Hôte
 - Device

Hôte – CUDA

```
1  /******  
2  /*Programme hote  
3  /******  
4  main(){  
5  //1- Allocation memoire  
6  cudaMalloc(&C, sizeof(float)*niter);  
7  cudaMalloc(&A, sizeof(float)*niter);  
8  cudaMalloc(&B, sizeof(float)*niter);  
9  
10 //2- Transfert CPU -> GPU  
11 cudaMemcpy(A,hA,sizeof(float)*niter,  
    cudaMemcpyHostToDevice);  
12 cudaMemcpy(B,hB,sizeof(float)*niter,  
    cudaMemcpyHostToDevice);  
13  
14 //3- Execution du noyau de calcul  
15 addVectorKernel<<<64,128>>>(C,niter,A,B)  
    ;  
16  
17 //4- Transfert GPU -> CPU  
18 cudaMemcpy(hC,C,sizeof(reals)*niter,  
    cudaMemcpyDeviceToHost);  
19 }
```

Hôte – OpenCL

```
1  /******  
2  /*Programme hote  
3  /******  
4  main(){  
5  //Initialisation simplifiee plus de 200  
    lignes de codes.  
6  //Fonctions utilisees :  
7  // - clGetPlatformIDs  
8  // - clGetDeviceIDs  
9  // - clCreateContext  
10 // - clCreateCommandQueue  
11 // - clCreateProgramWithSource  
12 // - clBuildProgram  
13 // - clCreateKernel  
14  
15 //1- Allocations memoires  
16 A = clCreateBuffer(context,  
    CL_MEM_READ_ONLY,size,NULL,&errCl);  
17 B = clCreateBuffer(context,  
    CL_MEM_READ_ONLY,size,NULL,&errCl);  
18 C = clCreateBuffer(context,  
    CL_MEM_WRITE_ONLY,size,NULL,&errCl)  
    ;
```

```
19  
20 //2- Transfert CPU -> GPU  
21 clEnqueueWriteBuffer(cmdQueue, A,CL_TRUE  
    ,0,size,hA,0,NULL,&event[0]));  
22 clEnqueueWriteBuffer(cmdQueue, B,CL_TRUE  
    ,0,size,hB,0,NULL,&event[1]));  
23  
24 //3- Initialisation des arguments et  
    appel du noyau de calcul  
25 clSetKernelArg(kernel,0,size,&C);  
26 clSetKernelArg(kernel,1,size,&niter);  
27 clSetKernelArg(kernel,2,size,&A);  
28 clSetKernelArg(kernel,3,size,&B);  
29  
30 //3- Execution du noyau de calcul  
31 clEnqueueNDRangeKernel(cmdQueue,kernel  
    ,1,NULL,8192,128,2,event,&event[2]);  
32  
33 //4- Transfert GPU -> CPU  
34 clEnqueueReadBuffer(cmdQueue, C,CL_TRUE  
    ,0,size,hC,1,&event[2],&event[3]));  
35 }
```

Noyau – CUDA vs. OpenCL

```
1  /******  
2  /*Noyau de calcul  
3  /******  
4  __global__ void addVectorKernel(float *C  
    ,int niter,float *A,float * B)  
5  {  
6  //Indice globale du thread  
7  int i = blockIdx.x * blockDim.x +  
    threadIdx.x;  
8      C[i] = A[i]+B[i];  
9  }
```

```
1  /******  
2  /*Noyau de calcul  
3  /******  
4  __kernel void addVectorKernel(__global  
    float *C,int niter,float *A,  
    __global float * B)  
5  {  
6  //Indice globale du thread  
7  int i = get_global_id(0);  
8  
9      C[i] = A[i]+B[i];  
10 }
```


Conclusion – OpenCL

- Modèle de programmation unifié pour accélérateur parallèle
- Avantages :
 - Portabilité
 - Exécution dynamique (gestion des dépendances)
- Inconvénients
 - Verbeux
 - Portabilité en option pour chaque implémentation
 - Bas niveau

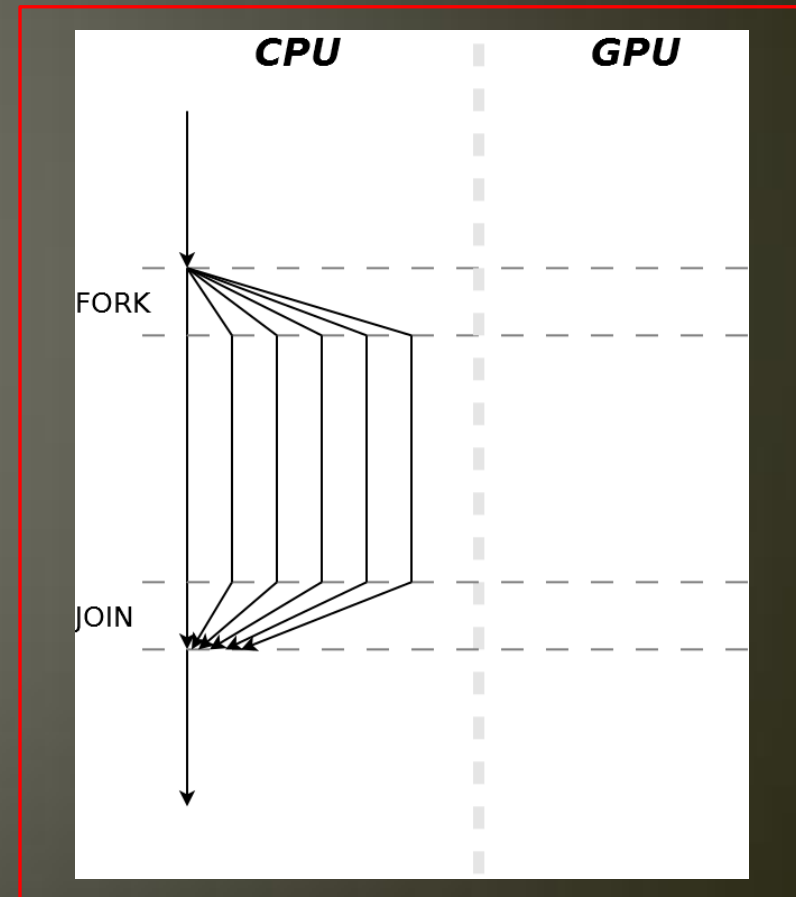
Programmation par directives

Plan du cours

- Programmation par directives
 - OpenMP
 - Directives simples
 - Directives de partages de travail
 - OpenACC
- Task-scheduling
 - StarPU

Rappel sur le principe d'OpenMP

- Ajout de directives OpenMP
 - Travail du programmeur
 - Possibilité d'ajout automatique dans certains cas précis
- À l'exécution du programme, le système d'exploitation construit une région parallèle sur le modèle «fork/join».
- À l'entrée d'une région parallèle, la tâche maître crée/active (fork) des threads « fils » (processus légers) qui disparaissent (ou s'assoupissent) en fin de région parallèle (join).



Partage du travail

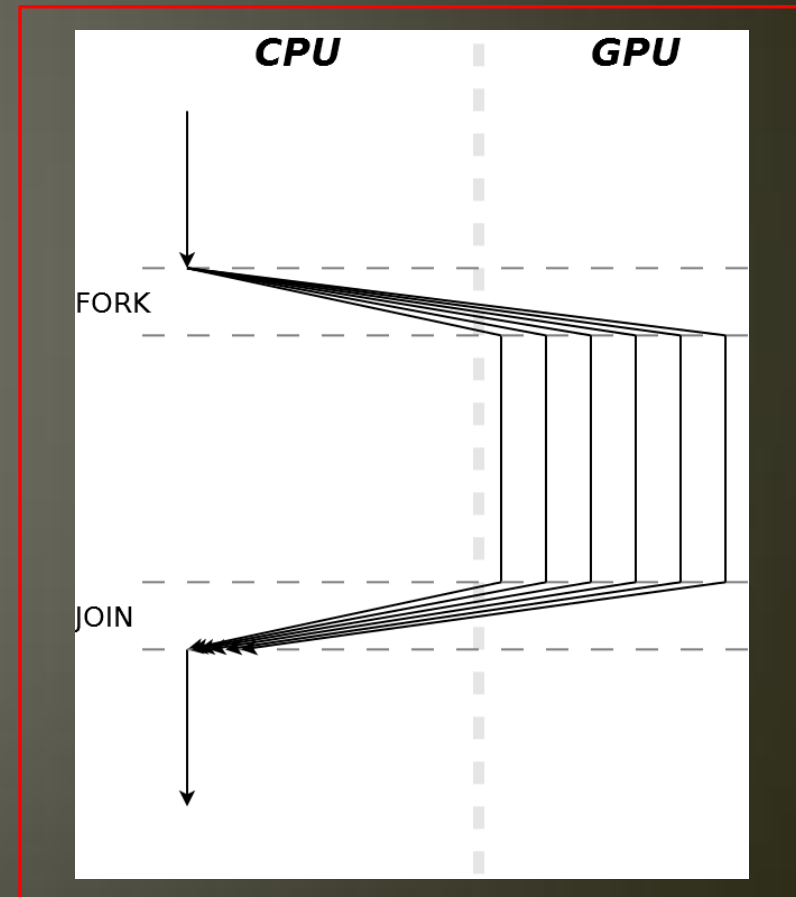
- Construction d'une région parallèle + fonctions OpenMP suffisants pour paralléliser un code
 - Répartition et équilibrage manuel du travail (données)
 - Synchronisation manuel
- OpenMP 4.5 propose des directives de partage de travail
 - FOR, SECTIONS et TASKLOOP : répartition de la charge de travail sur les threads
 - MASTER et SINGLE : exécution monothread d'une partie du travail
- Le partage du travail concerne des parties indépendantes de code.
 - Ni le compilateur, ni le runtime ne vérifient que ces parties sont bien parallélisables !

Update pragmas openmp

- **#pragma omp simd**
 - S'applique sur une boucle for
 - Permet de demander la vectorisation des données dans la boucle
- **Clause depend**
 - S'applique sur les clauses de tâches
 - Permet de spécifier des dépendances entre les données lues/produites par la tâche

OpenMP sur accélérateur

- **Nouvel ensemble de directives OpenMP**
 - En partie intégrée avec les directives « legacy » de OpenMP
 - Utile des clauses déjà existantes
- **À l'entrée de la région parallèle, la tâche maître crée (ou active) des threads « fils » s'exécutant sur l'accélérateur cible**
- **Le partage du travail concerne des parties indépendantes de code.**
 - Ni le compilateur, ni le runtime ne vérifient que ces parties sont bien parallélisables !



Plan du cours

- Programmation par directives
 - OpenMP
 - Directives simples
 - Directives de partages de travail
 - OpenACC
- Task-scheduling
 - StarPU

Structure d'un code accélérateur

```
int main()
{
    int * h_s, *h_d;
    int * d_s, *d_d;

    <... code cpu ...>

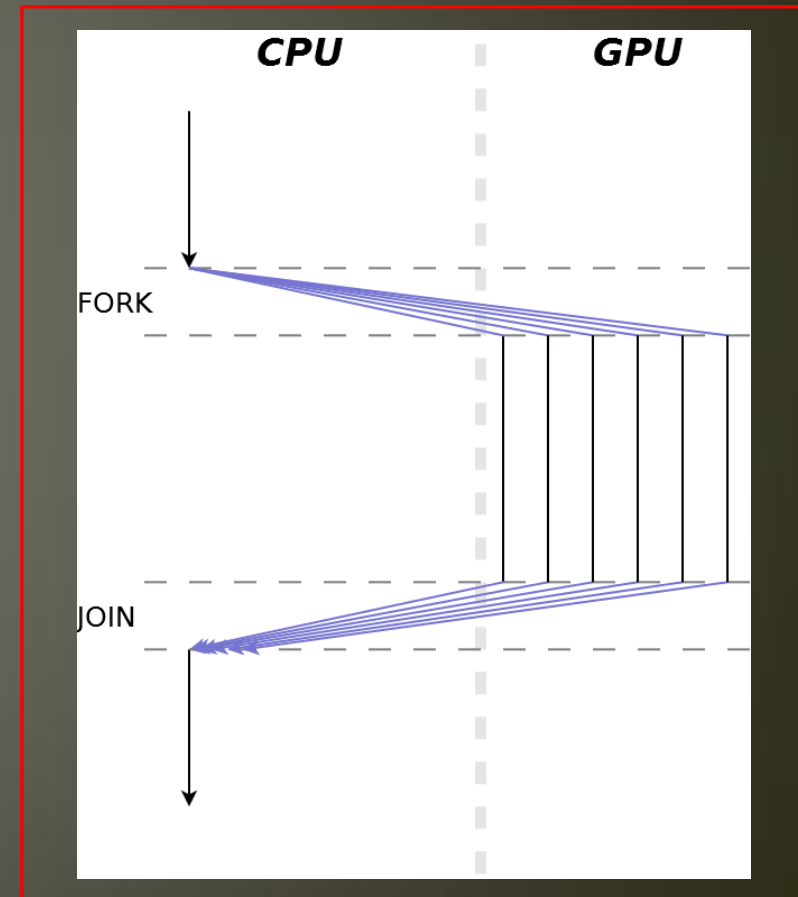
    cudaMalloc((void**)&d_s, size);
    cudaMalloc((void**)&d_d, size);

    cudaMemcpy(d_s, h_s, size, cudaMemcpyHostToDevice);

    mykernel <<<grid, block>>> (d_s, d_d);

    cudaMemcpy(h_d, d_d, size, cudaMemcpyDeviceToHost);

    return 0;
}
```



Transferts structurés de données

- **#pragma omp target data**
 - **if**([**target data** :] *scalar-expression*)
 - **device**(*integer-expression*)
 - **map**([[*map-type-modifier*[,]] *map-type*:] *list*)
 - **use_device_ptr**(*list*)

- Les directives **target data** doivent être parfaitement imbriquées
- Le bloc définissant le scope d'une directive **target data** s'appelle une *data region*

```
#pragma omp target data map(to:a[8:8])
{
    ...
    #pragma omp target data map(to:b[2:9])
    {
        ...
    }
    ...
}
```

Array sections

- Les données à transférer dans *list* sont représentées sous forme de *Array section*
 - `array_idenfier[ lower-bound : length ]`
 - `array_idenfier[ lower-bound : ]`
 - `array_idenfier[ : length ]`
 - `array_idenfier[ : ]`
- L'*array section* doit être un sous-ensemble du tableau
- *Lower-bound* et *length* sont des entiers positifs
- Il est possible d'utiliser des *array sections* sur des tableaux multidimensionnels
- Lorsque la taille du tableau n'est pas connu statiquement, il est obligatoire de spécifier *length*

Map clauses (1)

- **to:** transfère les données de *list* depuis le CPU vers le GPU
 - En début du bloc
 - Equivalent à `cudaMemcpy(..., cudaMemcpyHostToDevice)`
- **from:** transfère les données de *list* depuis le GPU vers le CPU
 - En fin de bloc
 - Equivalent à `cudaMemcpy(..., cudaMemcpyDeviceToHost)`
- **tofrom:** Equivalent à **to** et **from** sur les données de *list*

- Pour les clauses **to** et **tofrom**, les données sont transférées sur le GPU uniquement si les données ne sont pas déjà présentes, ou si **always** est utilisé comme *map-type-modifier*.

```
#pragma omp target data map(to:a[8:8])
{
    ...
    #pragma omp target data map(to:b[8:8])
    {
        ...
    }
    ...
}
```

Map clauses (2)

- **alloc:** alloue un espace pour les données de *list* sur le GPU
 - Ne réalise aucun transfert
 - Equivalent à `cudaMalloc(..., sizeof(list[i]))`
- **release:** relâche les données GPU pour la directive courante
 - Supprime les données si il n'y a plus de références aux données dans *list*
- **delete:** supprime l'allocation des données pour toutes les directives

```
#pragma omp target data map(to:a[8:8])
{
    ...
    #pragma omp target data map(to:b[8:8])
    {
        ...
    }
    ...
}
```

Peu de flexibilités

- Les transferts structurés de données sont simple à utiliser
- Cependant, les transferts structurés de données n'offrent pas la même liberté que la programmation cuda.

```
cudaMalloc(d_a,...) ;  
    Kernel<<<...,...>>>(d_a) ;  
  
cudaMalloc(d_b,...) ;  
    Kernel<<<...,...>>>(d_a, d_b) ;  
  
cudaFree(d_a) ;  
  
    Kernel<<<...,...>>>(d_b) ;  
cudaFree(d_b) ;
```

Peu de flexibilités

- Les transferts structurés de données sont simple à utiliser
- Cependant, les transferts structurés de données n'offrent pas la même liberté que la programmation cuda.

```
cudaMalloc(d_a,...) ;  
    Kernel<<<...,...>>>(d_a) ;  
  
cudaMalloc(d_b,...) ;  
    Kernel<<<...,...>>>(d_a, d_b) ;  
  
cudaFree(d_a) ;  
  
    Kernel<<<...,...>>>(d_b) ;  
cudaFree(d_b) ;
```

```
cudaMalloc(d_a,...) ;  
    Kernel<<<...,...>>>(d_a) ;  
  
cudaMalloc(d_b,...) ;  
    Kernel<<<...,...>>>(d_a, d_b) ;  
  
    Kernel<<<...,...>>>(d_b) ;  
cudaFree(d_b) ;  
cudaFree(d_a) ;
```

- **Solution: transferts non-structurés de données**
 - Directive *enter data* en début de *data region*
 - Directive *exit data* en fin de *data region*

Transferts non-structurés de données (1)

- **#pragma omp target enter data**
 - **if**([**target data** :] *scalar-expression*)
 - **device**(*integer-expression*)
 - **map**([[*map-type-modifier* [,]] *map-type*:] *list*)
 - **depend**(*dependence-type* : *list*)
 - **nowait**
- Seules les types **alloc** et **to** sont autorisés comme valeurs pour la clause map
- Cette directive se lance sous la forme d'une tâche
 - Il est possible de mettre une dépendance pour la réaliser une fois que les données requises sont produites par une autre tâche

Transferts non-structurés de données (2)

- **#pragma omp target exit data**
 - **if**([**target data** :] *scalar-expression*)
 - **device**(*integer-expression*)
 - **map**([[*map-type-modifier* [,]] *map-type*:] *list*)
 - **depend**(*dependence-type* : *list*)
 - **Nowait**
- Seules les types **from**, **delete** et **release** sont autorisés comme valeurs pour la clause **map**
- Cette directive ferme une directive *enter data*
- Cette directive se lance sous la forme d'une tâche

Mise à jour de données

- **#pragma omp target update**
 - **if**([**target update** :] *scalar-expression*)
 - **device**(*integer-expression*)
 - **nowait**
 - **depend** (*dependence-type* : *list*)
 - **to**(*list*) or **from**(*list*)
- Permet de mettre à jour des données déjà présentes dans une des *data region* en cours
- Cette directive se lance sous la forme d'une tâche

Structure d'un code accélérateur

```
int main()
{
    int * h_s, *h_d;
    int * d_s, *d_d;

    <... code cpu ...>

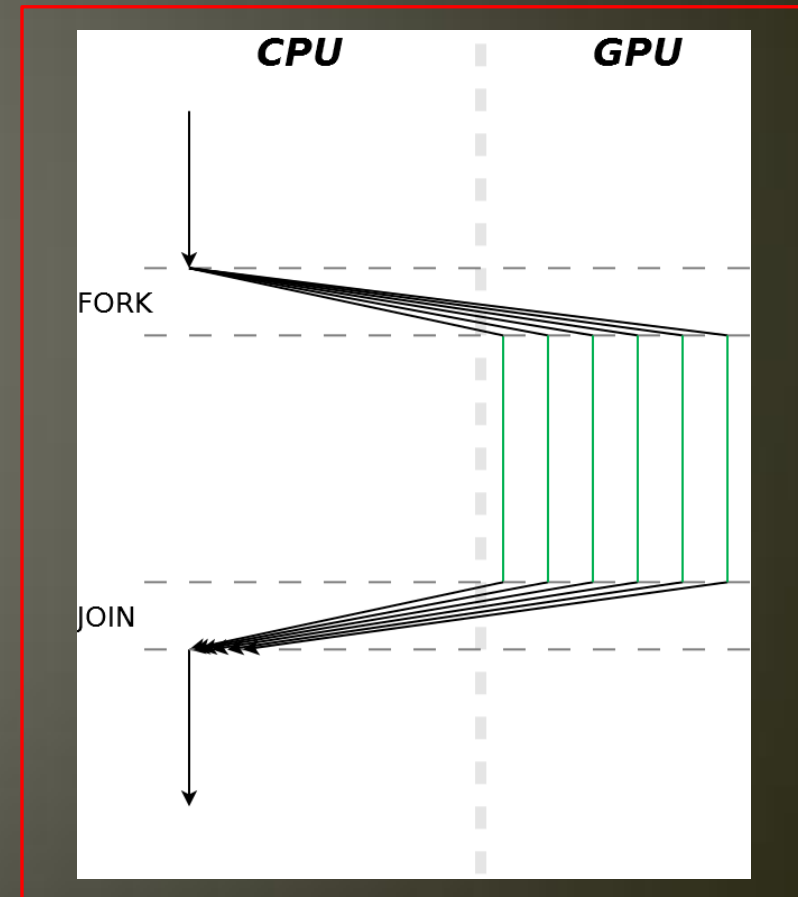
    cudaMalloc((void**)&d_s, size);
    cudaMalloc((void**)&d_d, size);

    cudaMemcpy(d_s, h_s, size, cudaMemcpyHostToDevice);

    mykernel <<<grid, block>>> (d_s, d_d);

    cudaMemcpy(h_d, d_d, size, cudaMemcpyDeviceToHost);

    return 0;
}
```



Lancement d'une région déportée (1)

- **#pragma omp target**
 - **if**([**target** :] *scalar-expression*)
 - **device**(*integer-expression*)
 - **private**(*list*)
 - **firstprivate**(*list*)
 - **map**([[*map-type-modifier*[,]] *map-type*:] *list*)
 - **is_device_ptr**(*list*)
 - **defaultmap**(*tofrom:scalar*)
 - **nowait**
 - **depend**(*dependence-type: list*)

Lancement d'une région déportée (2)

- Une directive target génère une *target region*
- Le scope de cette *target region* s'exécute sur l'accélérateur spécifié dans la clause device
- `/!\` La directive target ne crée pas de threads!
 - Il est nécessaire d'appeler ensuite des directives de créations de threads, tels que `parallel`
- Nous retrouvons les mêmes clauses que pour une ouverture de section parallèle
 - `Private`, `firstprivate`, `nowait`, `depend`
- Si une clause `map` est spécifié, la *target region* est aussi une *data region* structurée

Plan du cours

- Programmation par directives
 - OpenMP
 - Directives simples
 - Directives de partages de travail
 - OpenACC
- Task-scheduling
 - StarPU

Gestion du parallélisme sur un accélérateur

- Cuda propose trois niveaux de parallélisme:
 - Grille -> Bloc -> thread
- OpenMP ne propose, à la base, que deux niveaux de parallélisme...
 - Team -> thread
 - Pour un GPU Nvidia, team $\approx$ bloc
- ... le troisième niveau est implicite
 - C'est le nombre de teams
- Certaines implémentations voit le niveau SIMD comme un niveau supplémentaire représentant les warps
- Comment fait-on pour gérer correctement ces niveaux de parallélisme?

Equipes de threads: teams (1)

- **#pragma omp teams**
 - **num_teams**(*integer-expression*)
 - **thread_limit**(*integer-expression*)
 - **default**(**shared** | **none**)
 - **private**(*list*)
 - **firstprivate**(*list*)
 - **shared**(*list*)
 - **reduction**(*reduction-identifiser* : *list*)
- **Clause num_teams: nombre de teams demandé**
 - Equivalent à la taille de la grille en CUDA
- **Clause thread_limit: nombre maximum de threads de une team**
 - Le nombre de threads dans la team dépend de l'implémentation

Equipes de threads: teams (2)

- La directive teams crée un nombre de teams égal à la valeur dans la clause num_teams
- Chaque team est composée du même nombre de threads
- Le nombre de threads dans chaque team est inférieur ou égal à la valeur dans la clause thread_limit
- La directive teams accepte les mêmes clauses que la directive parallel
 - private
 - firstprivate
 - reduction

Equipes de threads: teams (3)

- `/!\` Un seul thread de chaque team exécute le code dans le scope de la directive
 - Il est le thread master de sa team
- Il est donc nécessaire de faire appel à d'autres directives pour répartir le travail sur tous les threads



Et la directive *parallel*?

- Jusqu'à présent, vous aviez l'habitude d'utiliser la directive *parallel* pour lancer une région parallèle OpenMP
 - Pourquoi ne fait-on pas la même chose en « mode » target?
- En fait, la directive *parallel* crée une seule team de threads
- De base, chacun de ces threads exécute la région parallèle
- Pour répartir le travail, il est nécessaire de lui associer d'autres directives
 - Directive *for* pour répartir les itérations d'une boucle sur les threads
 - Directive *sections* pour créer différentes sections de code à exécuter en parallèle
- La directive *parallel* offre moins de hiérarchie, et donc moins de contrôle, pour la répartition de la parallélisation

Répartitions des itérations d'une boucle

- De même, jusqu'à présent, vous aviez l'habitude d'utiliser les directives combinées *parallel for* pour répartir les itérations d'une boucle sur les différents threads
- Cette répartition se fait uniquement sur les threads d'une team
- Si nous avons plusieurs teams, il semble intéressant de pouvoir répartir le travail sur tous les threads de toutes les teams
- la directive *distribute* est là pour remplir une partie de ce rôle

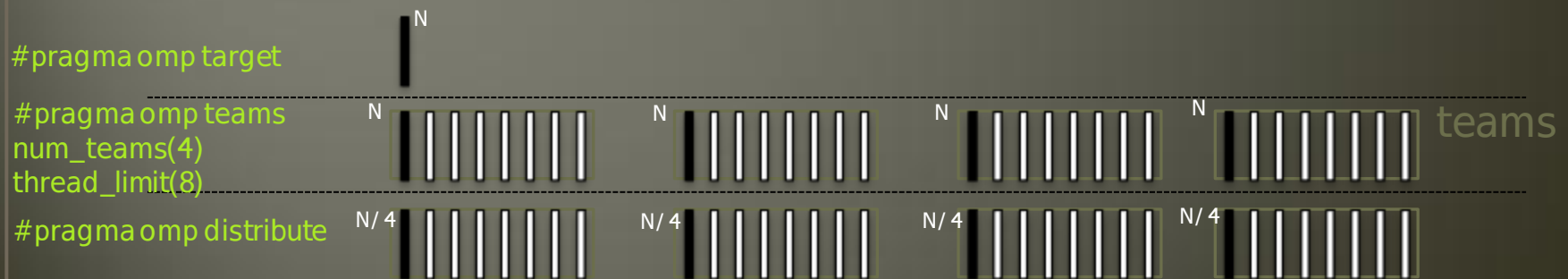


Distribution du travail sur accélérateur (1)

- **#pragma omp distribute**
 - **private(*list*)**
 - **firstprivate(*list*)**
 - **lastprivate(*list*)**
 - **collapse(*n*)**
 - **dist_schedule(*kind*[, *chunk_size*])**
- La directive *distribute* se place avant une boucle *for*
- Nous retrouvons les mêmes clauses que pour une directive *for*
 - Private, firstprivate, lastprivate, collapse

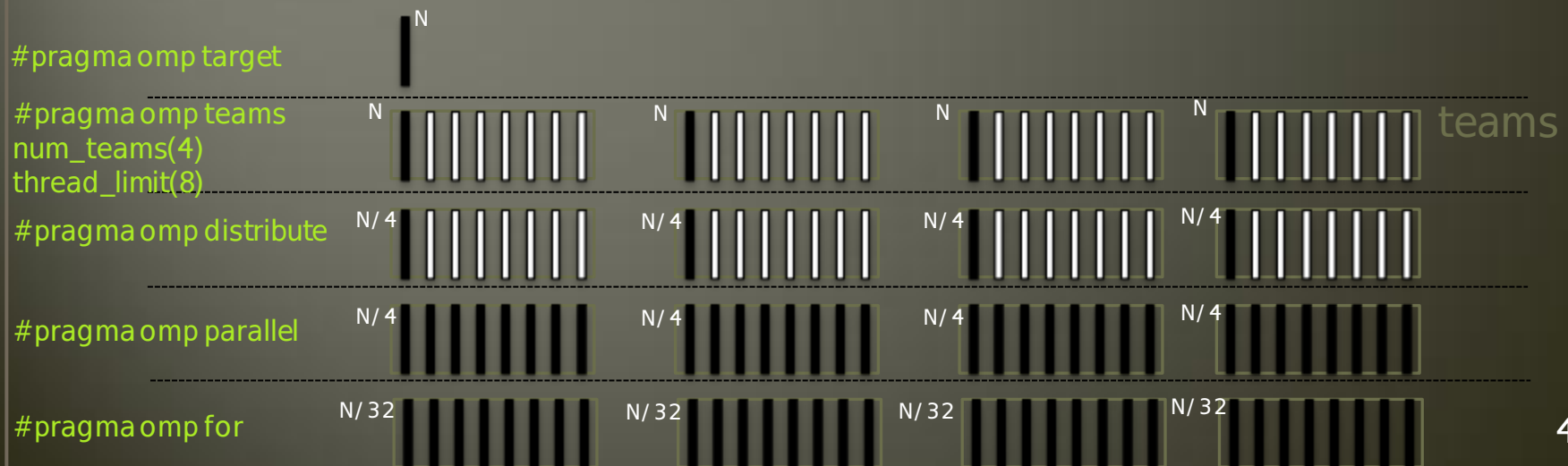
Distribution du travail sur accélérateur (2)

- La directive *distribute* permet de répartir les itérations de la boucle suivant la directive sur les threads masters des différentes teams créées par une directive *teams* englobante
- Il est encore nécessaire de faire appels à d'autres directives pour que chaque thread de chaque team réalise une part du travail



Distribution du travail sur accélérateur (2)

- Pour répartir les itérations recueillies par le thread master sur tous les threads de la team, il faut appeler les directives *parallel* et *for* (ou la directive combinée *parallel for*)



Distribution plus précis du travail

- Pour ne pas avoir à imbriquer de nombreuses directives OpenMP pour répartir les itérations d'une boucle sur tous les threads de toutes les teams, OpenMP 4.5 offre plusieurs directives imbriquées avec la directive *distribute*
- **#pragma omp distribute simd**
 - Réparti les itérations d'une boucle sur les master threads des teams et essaie de les vectoriser
- **#pragma omp distribute parallel for**
 - Réparti les itérations d'une boucle sur les tous les threads de toutes les teams
- **#pragma omp distribute parallel for simd**
 - Réparti les itérations d'une boucle sur les tous les threads de toutes les teams et essaie de les vectoriser

Exemple OpenMP target

```
int main ()
{
    int *a, *b, *c;
    int i;
    int j;

    a = malloc(sizeof(int) * 10000);
    b = malloc(sizeof(int) * 10000);
    c = malloc(sizeof(int) * 10000);

    #pragma omp target data map(to:a[:10000],b[:10000]) map(from:c[:10000]) device(1)
    {
        #pragma omp target device(1)
        {
            #pragma omp teams num_teams(20) thread_limit(5)
            {
                #pragma omp distribute parallel for
                for(i=0; i<10000; i++)
                {
                    a[i] = omp_get_thread_num() + omp_get_team_num() * omp_get_num_threads();
                    b[i] = 1;
                    c[i] = 0;
                }
            }
        }

        #pragma omp target update from(a[:10000]) device(1)

        printf("/***** A *****/\n");
        for(j=0; j< 10; j++)
        {
            printf("[%d]", a[j]);
        }
    }
}
```

Plan du cours

- Programmation par directives
 - OpenMP
 - OpenACC
- Task-scheduling
 - StarPU

Transferts structurés de données

- **#pragma acc data**
 - **if(condition)**
 - **copy(var-list)**
 - **copyin(var-list)**
 - **copyout(var-list)**
 - **create(var-list)**
 - **present(var-list)**
 - **deviceptr(var-list)**
- Les directives **acc data** doivent être parfaitement imbriquées
- Equivalent aux directive **omp target data**

```
#pragma acc data copy(a[8:8])
{
    ...
    #pragma acc data copyin(b[2:9])
    copy(c[3:20])
    {
        ...
    }
    ...
}
```

data clauses (1)

- OpenACC utilise aussi des *array sections* pour définir les zones de données à transférer
- Si une *array section* fait référence au même *array_identifieur* qu'une *data region* englobante, elle doit utiliser les mêmes données **strictement**
- **create:** alloue un espace pour les données de *list* sur le GPU
 - Ne réalise aucun transfert
 - Equivalent à `cudaMalloc(..., sizeof(list[i]))`
- **present:** spécifie que les données de *list* sont censées être déjà présentes sur le device à l'entrée de la *data region*
 - Si ce n'est pas le cas, cela génère une erreur à l'exécution

data clauses (2)

- **copyin:** transfère les données de *list* depuis le CPU vers le GPU
 - En début de bloc
 - Équivalent à `cudaMemcpy(..., cudaMemcpyHostToDevice)`
- **copyout:** transfère les données de *list* depuis le GPU vers le CPU
 - En fin de bloc
 - Équivalent à `cudaMemcpy(..., cudaMemcpyDeviceToHost)`
- **copy:** Équivalent à **copyin** et **copyout** sur les données de *list*
- Ces clauses ne sont pas équivalentes à **to**, **from** et **tofrom**
 - Les données sont transférées dans tous les cas
 - Existence des clauses `present_or_{create, copyin, copyout, copy}` équivalentes à `map(to: )`, `map(from: )` et `map(tofrom: )`
 - Il y a transfert ou allocation uniquement si les données ne sont pas déjà référencées

Transferts non-structurés de données (1)

- **#pragma acc enter data**
 - **if(condition)**
 - **async [(int-expr)]**
 - **wait [(int-expr-list)]**
 - **copyin(var-list)**
 - **create(var-list)**
- Directive équivalente à *omp target enter data*
- Cette directive se lance sous la forme d'une tâche
 - Il est possible de mettre une dépendance pour la réaliser une fois que les données requises sont produites par une autre tâche
 - Cette tâche peut être lancée de façon asynchrone

Transferts non-structurés de données (2)

- **#pragma acc exit data**
 - **if(condition)**
 - **async [(int-expr)]**
 - **wait [(int-expr-list)]**
 - **copyout(var-list)**
 - **delete(var-list)**
 - **finalize**
- **/!\ Attention aux définitions des clauses dans les deux APIs**
 - **delete** en OpenACC est équivalent à **release** en OpenMP
- Cette directive ferme une directive *enter data*
- Cette directive se lance sous la forme d'une tâche

Lancement d'une région déportée (1)

- **#pragma acc parallel**
 - **async** [(int-expr)]
 - **wait** [(int-expr-list)]
 - **num_gangs**(int-expr)
 - **num_workers**(int-expr)
 - **vector_length**(int-expr)
 - **device_type**(device-type-list)
 - **if**(condition)
 - **reduction**(operator:var-list)
 - **deviceptr**(var-list)
 - **private**(var-list)
 - **firstprivate**(var-list)
 - **default**(none | present)
 - ... et les clauses de data (create, ...)

Gestion du parallélisme sur un accélérateur

- Cuda propose trois niveaux de parallélisme:
 - Grille -> Bloc -> thread
- OpenMP ne propose, à la base, que deux niveaux de parallélisme...
 - Team -> thread
- OpenACC, plus proche d'une représentation GPU, propose trois niveaux de parallélisme
 - Gang -> worker -> vector
 - `num_gangs( int-expr )`
 - `num_workers( int-expr )`
 - `vector_length( int-expr )`

Lancement d'une région déportée (2)

- Une directive *parallel* génère une *accelerator parallel region*
- Le scope de cette *parallel region* s'exécute sur l'accélérateur spécifié dans la clause *device*
- La directive *parallel* crée le nombre de threads demandés dans les clauses *num_gangs*, *num_workers* et *vector_length*
 - $\text{Number of threads} = \text{num_gangs} * \text{num_workers} * \text{vector_length}$
- En rapprochant la hiérarchie de parallélisme OpenACC à un GPU Nvidia:
 - Gang = block, worker = warp, and vector = threads of a warp
 - /!\ L'interprétation des niveaux hiérarchiques dépend de l'implémentation du runtime OpenACC

Lancement d'une région déportée (3)

- Un seul *worker* de chaque *gang* exécute le code dans le scope de la directive
 - Chaque gang va exécuter de façon redondante le code dans la *accelerator parallel region*
- La directive *acc parallel* est donc équivalente à la combinaison des deux directives *omp target* et *omp teams*
- Avec OpenACC, il est donc aussi nécessaire de faire appel à d'autres directives pour répartir le travail sur tous les threads

Distribution du travail sur accélérateur (1)

- **#pragma acc loop**
 - collapse(n)
 - gang [(gang-arg-list)]
 - worker [([num:]int-expr)]
 - vector [([length:]int-expr)]
 - seq
 - auto
 - tile(size-expr-list)
 - device_type(device-type-list)
 - independent
 - private(var-list)
 - reduction(operator:var-list)

Distribution du travail sur accélérateur (2)

- La directive *loop* permet de répartir les itérations de la boucle suivant la directive *threads*
- Les clauses *gang*, *worker* et *vector* permettent de guider le découpage et la répartition des itérations
 - une directive *loop* imbriquée dans une directive *parallel* et avec une clause *gang* passe du mode « gang-redundant » en mode « gang-partitioned »
 - une directive *loop* avec une clause *worker* passe du mode « worker-single » en mode « worker-partitioned »
 - une directive *loop* avec une clause *vector* passe du mode « vector-single » en mode « vector-partitioned »
- Il est donc nécessaire de bien spécifier les trois niveaux de parallélismes dans la directive *loop* pour utiliser correctement l'intégralité de l'accélérateur.

Le plus de OpenACC: la directive *kernels* (1)

- **#pragma acc kernel**
 - **async** [(int-expr)]
 - **wait** [(int-expr-list)]
 - **num_gangs**(int-expr)
 - **num_workers**(int-expr)
 - **vector_length**(int-expr)
 - **device_type**(device-type-list)
 - **if**(condition)
 - **deviceptr**(var-list)
 - **default**(none | present)
 - ... et les clauses de data (create, ...)

Le plus de OpenAcc: la directive *kernels* (2)

- Le principe de la directive *kernels* : le compilateur sait mieux que vous ce qui est parallélisable, donc laissez le faire!
- Concrètement:
 - Un bloc est englobé dans la directive *kernels*
 - Comme pour une directive *acc parallel*
 - Le compilateur va évaluer la meilleure façon de paralléliser
 - Il peut créer plusieurs kernels distincts à partir du bloc contenu dans la directives
 - Comme si le programmeur avait placé plusieurs directives *parallel*
 - Typiquement, chaque boucle dans le bloc va devenir un kernel distinct

Le plus de OpenAcc: la directive *kernels* (3)

- Chacun des kernels générés peut avoir un nombre de gangs, workers et vectors différent
 - Permet d'adapter le parallélisme au code du kernel
- Les kernels créés à partir d'une seule directive kernels seront toujours exécutés dans l'ordre
- Il est possible d'appeler une directive loop dans un kernel

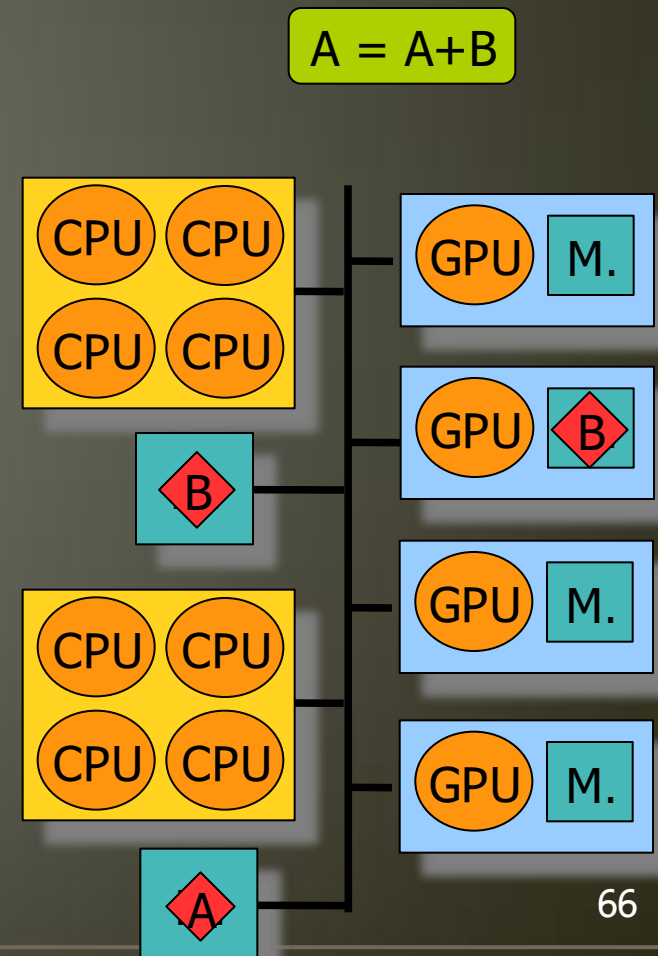
Task Scheduling

Plan du cours

- Les différentes APIs pour CUDA
- Notion de contextes CUDA
- Programmation Multi-GPUs
- Task-scheduling
 - StarPU

The StarPU task-based runtime system

- StarPU est un support exécutif pour les architectures hétérogènes
- Permet d'ordonnancer dynamiquement les tâches d'un graphe sur tous les processing units
 - Il reconnaît et gère un ensemble de ressources de calculs hétérogènes (cpus, gpus ...)
- StarPu évite les transferts inutiles
 - Mémoire partagée virtuelle pour les machines hétérogènes

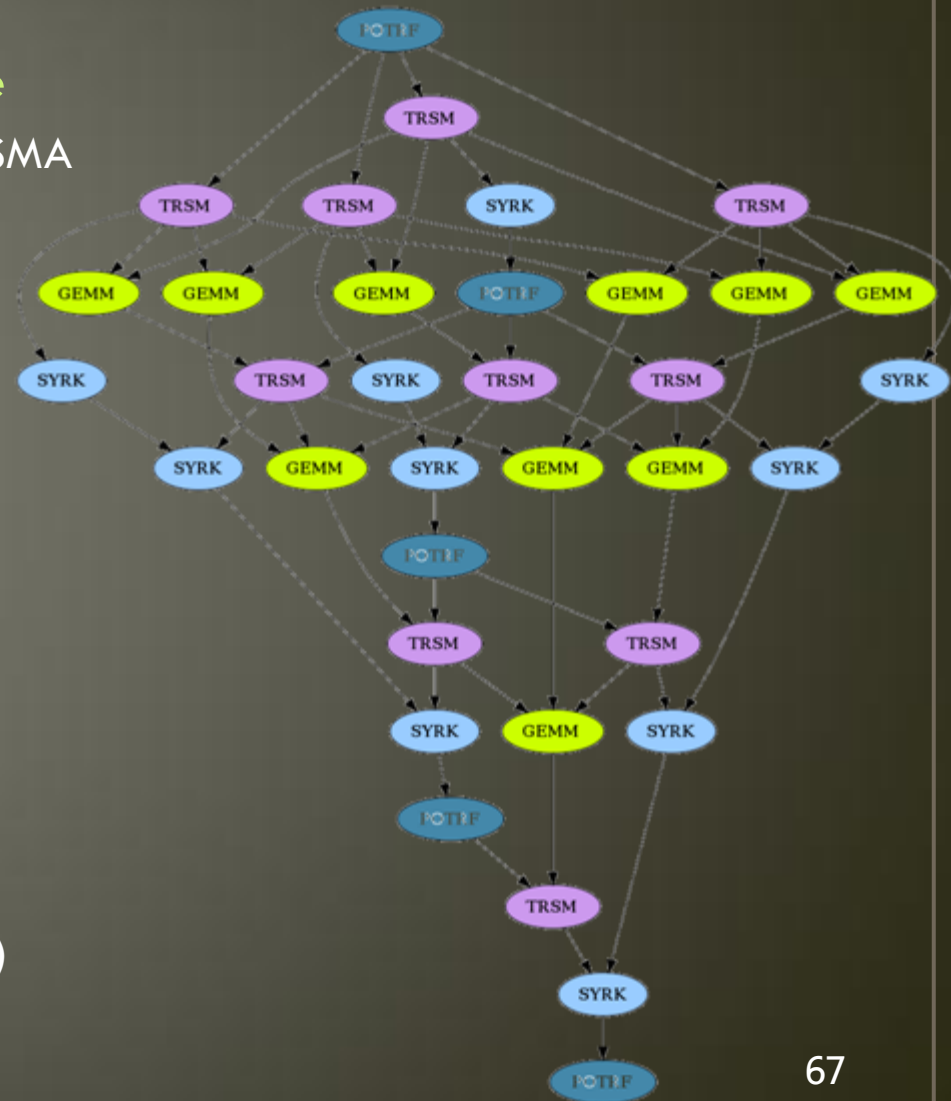


Un exemple de graphe de tâche

- Décomposition de Cholesky par tuile
 - Exemple tiré de la bibliothèque PLASMA

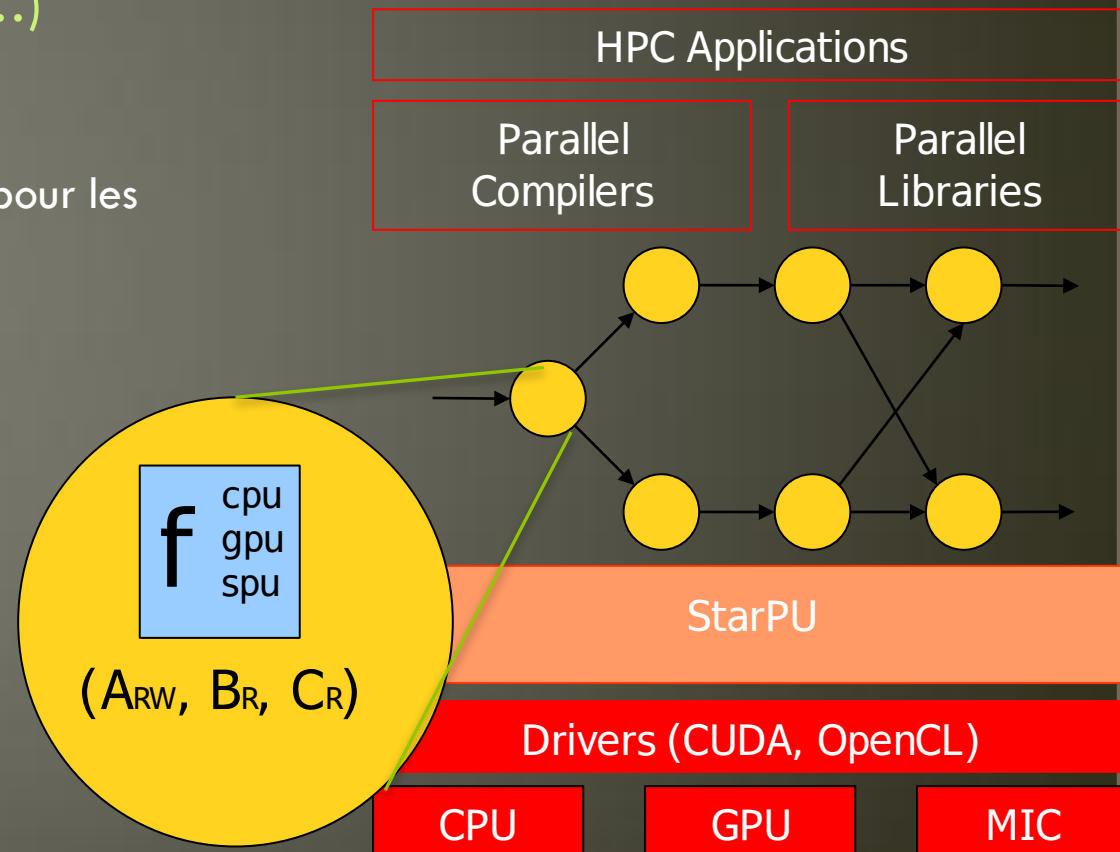
```

For (k = 0 .. tiles - 1) {
  POTRF(A[k,k])
  for (m = k+1 .. tiles - 1)
    TRSM(A[k,k], A[m,k])
  for (m = k+1 .. tiles - 1) {
    SYRK(A[m,k], A[m,m])
    For (n = k+1 .. m - 1)
      GEMM(A[m,k], A[n,k], A[m,n])
  }
}
    
```



Maximise l'utilisation des PU, minimise les transferts

- Les tâches peuvent avoir plusieurs implémentations (cpu, gpu, ...)
 - C'est la responsabilité de l'utilisateur de fournir les implémentations des tâches pour les différents types de PU disponibles

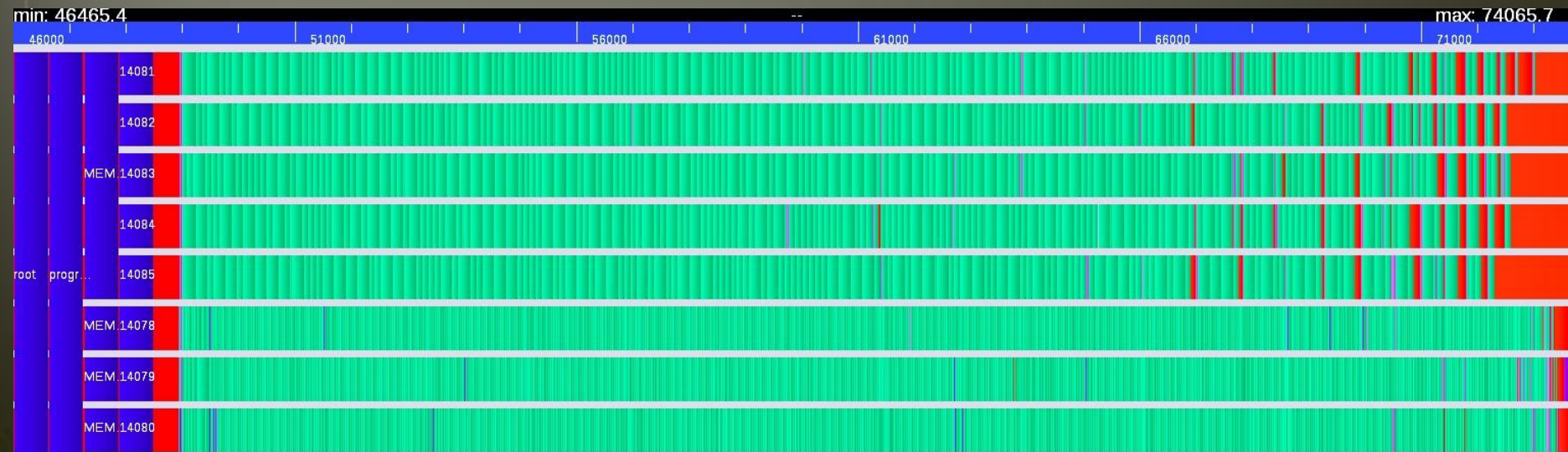
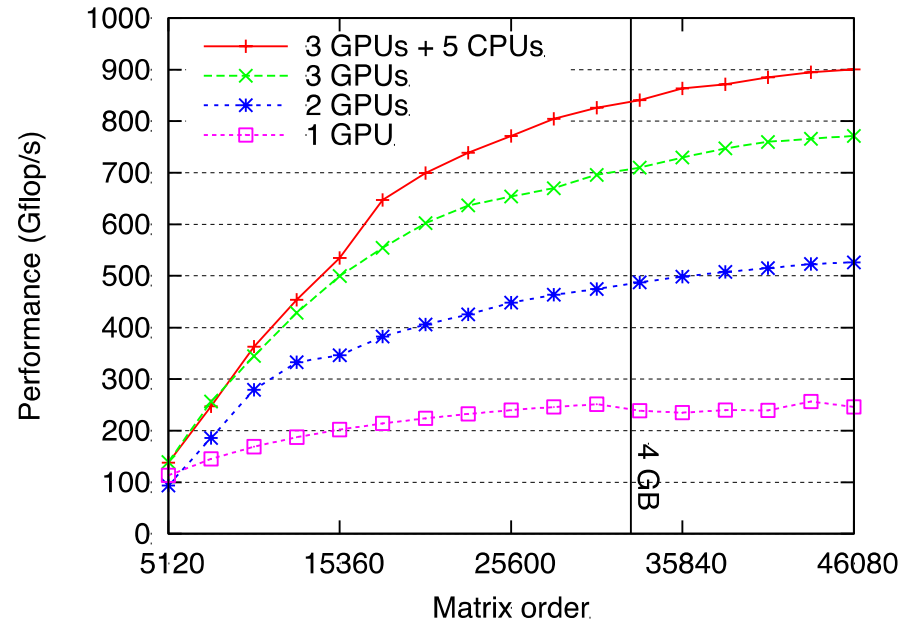


Fonction objective: réduire le temps de restitution

- Estimation du temps de complétion d'une tâche
 - Basé sur l'historique
 - Fonction de coût user-defined
 - Modèle de coût paramétrique
- Peut être utilisé pour améliorer l'ordonnancement
 - E.g. Minimum Completion Time
- Estimation du temps de transferts de données
 - Echantillonnage basé sur une calibration préalable
- Peut être utilisé pour améliorer la prédiction du temps d'exécution total
 - Minimise les mouvements de données pour éviter les surcoûts

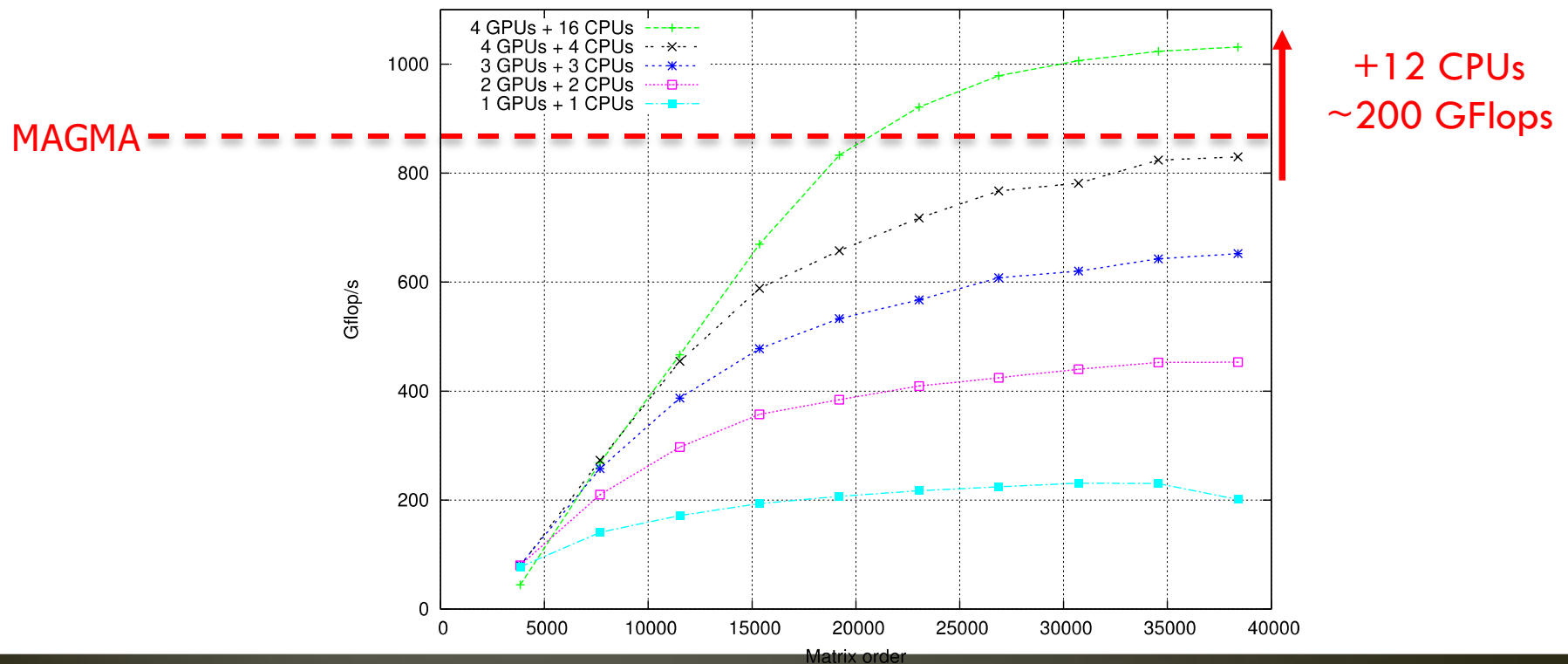
Résultats (1)

- Cholesky decomposition
- Mixing PLASMA and MAGMA with StarPU
 - 5 CPUs (Nehalem) + 3 GPUs (FX5800)
 - Efficiency > 100%



Résultats (2)

- QR decomposition
- Mixing PLASMA and MAGMA with StarPU
 - 16 CPUs (AMD) + 4 GPUs (C1060)



Fin du cours